

```

1  # ==== _gen_code_doc.py (41 行) ====
2  """生成代码.docx - 将所有源码按文件汇编"""
3  import docx, os, glob
4
5  doc = docx.Document()
6  style = doc.styles['Normal']
7  font = style.font; font.name = 'Consolas'; font.size = docx.shared.Pt(8)
8  style.paragraph_format.space_before = docx.shared.Pt(0)
9  style.paragraph_format.space_after = docx.shared.Pt(0)
10 style.paragraph_format.line_spacing = 1.0
11 for section in doc.sections:
12     section.top_margin = docx.shared.Cm(1.2)
13     section.bottom_margin = docx.shared.Cm(1.2)
14     section.left_margin = docx.shared.Cm(1.2)
15     section.right_margin = docx.shared.Cm(1.2)
16
17 base = os.path.dirname(os.path.abspath(__file__))
18 py_files = []
19 for root, dirs, files in os.walk(base):
20     dirs[:] = [d for d in dirs if d not in ('venv', '__pycache__', 'resources',
21     '.claude')]
22     for f in files:
23         if f.endswith('.py') and 'test' not in f.lower():
24             py_files.append(os.path.join(root, f))
25 py_files.sort()
26
27 total = 0
28 for fpath in py_files:
29     rel = os.path.relpath(fpath, base)
30     with open(fpath, 'r', encoding='utf-8') as f:
31         lines = f.readlines()
32     h = doc.add_paragraph()
33     run = h.add_run(f'# ==== {rel} ({len(lines)} 行) ====')
34     run.bold = True; run.font.size = docx.shared.Pt(9)
35     for line in lines:
36         p = doc.add_paragraph(line.rstrip('\n').rstrip('\r'))
37         p.style = doc.styles['Normal']
38         total += 1
39     doc.add_page_break()
40
41 out = os.path.join(base, '代码.docx')
42 doc.save(out)
43 print(f'代码.docx 已生成: {len(py_files)} 个文件, {total} 行代码, 预估
44 ~{total//50} 页')
45
46 # ==== core\__init__.py (1 行) ====
47 # 低代码 AI 平台 核心引擎层
48
49 # ==== core\algorithms.py (168 行) ====
50 """原创核心算法集 - 画布布局、特征分析、参数优化、数据适配"""

```

```

51 import math, random, hashlib, json
52 from typing import Dict, List, Tuple, Any, Optional
53 from dataclasses import dataclass, field
54
55 @dataclass
56 class LayoutNode:
57     nid: str; x: float = 0; y: float = 0; w: float = 150; h: float = 60
58     level: int = 0; children: List[str] = field(default_factory=list)
59
60 class ForceDirectedLayout:
61     """原创力导向布局算法 - 用于 AI 模型画布自动排列"""
62     def __init__(self, width: float = 2000, height: float = 1500):
63         self.width = width; self.height = height
64         self.spring_len = 180; self.spring_k = 0.06
65         self.repulsion = 8000; self.damping = 0.85
66         self.iterations = 200; self.min_dist = 100
67
68     def layout(self, nodes: List[LayoutNode], edges: List[Tuple[str, str]]) ->
69     List[LayoutNode]:
70         if not nodes: return []
71         node_map = {n.nid: n for n in nodes}
72         for n in nodes:
73             if n.x == 0 and n.y == 0:
74                 n.x = self.width / 2 + random.uniform(-50, 50)
75                 n.y = self.height / 2 + random.uniform(-50, 50)
76         velocities = {n.nid: [0.0, 0.0] for n in nodes}
77         for _ in range(self.iterations):
78             forces = {n.nid: [0.0, 0.0] for n in nodes}
79             for i, ni in enumerate(nodes):
80                 for j, nj in enumerate(nodes):
81                     if i >= j: continue
82                     dx, dy = ni.x - nj.x, ni.y - nj.y
83                     dist = max(1, math.sqrt(dx * dx + dy * dy))
84                     f = self.repulsion / (dist * dist)
85                     fx, fy = f * dx / dist, f * dy / dist
86                     forces[ni.nid][0] += fx; forces[ni.nid][1] += fy
87                     forces[nj.nid][0] -= fx; forces[nj.nid][1] -= fy
88             for src, tgt in edges:
89                 if src in node_map and tgt in node_map:
90                     ns, nt = node_map[src], node_map[tgt]
91                     dx, dy = ns.x - nt.x, ns.y - nt.y
92                     dist = max(1, math.sqrt(dx * dx + dy * dy))
93                     f = self.spring_k * (dist - self.spring_len)
94                     fx, fy = f * dx / dist, f * dy / dist
95                     forces[ns.nid][0] -= fx; forces[ns.nid][1] -= fy
96                     forces[nt.nid][0] += fx; forces[nt.nid][1] += fy
97             for n in nodes:
98                 vx, vy = velocities[n.nid]
99                 vx = (vx + forces[n.nid][0]) * self.damping
100                 vy = (vy + forces[n.nid][1]) * self.damping

```

```

101         velocities[n.nid] = [vx, vy]
102         n.x = max(20, min(self.width - 20, n.x + vx))
103         n.y = max(20, min(self.height - 20, n.y + vy))
104     return nodes
105
106 class FeatureAnalyzer:
107     """原创特征分析算法 - 数据特征提取、维度分析、分布计算"""
108     @staticmethod
109     def analyze(data: List[dict]) -> dict:
110         if not data: return {"error": "无数据"}
111         keys = list(data[0].keys())
112         analysis = {"total_samples": len(data), "dimensions": len(keys), "fields":
113         {}
114         for key in keys:
115             values = [d.get(key) for d in data if d.get(key) is not None]
116             if not values: continue
117             numeric_vals = [v for v in values if isinstance(v, (int, float))]
118             str_vals = [v for v in values if isinstance(v, str)]
119             field_info = {"type": "mixed", "count": len(values), "null_count":
120             len(data) - len(values)}
121             if len(numeric_vals) > len(values) * 0.8:
122                 field_info["type"] = "numeric"
123                 field_info.update(FeatureAnalyzer._numeric_stats(numeric_vals))
124             elif len(str_vals) > len(values) * 0.8:
125                 field_info["type"] = "categorical"
126                 field_info["unique"] = len(set(str_vals))
127                 freq = {}
128                 for v in str_vals: freq[v] = freq.get(v, 0) + 1
129                 field_info["top_values"] = sorted(freq.items(), key=lambda x:
130             -x[1])[:5])
131             analysis["fields"][key] = field_info
132         return analysis
133
134     @staticmethod
135     def _numeric_stats(vals: List[float]) -> dict:
136         n = len(vals); mean = sum(vals) / n
137         variance = sum((x - mean) ** 2 for x in vals) / n
138         sorted_vals = sorted(vals)
139         return {"mean": round(mean, 4), "std": round(math.sqrt(variance), 4),
140             "min": min(vals), "max": max(vals),
141             "median": sorted_vals[n // 2], "q1": sorted_vals[n // 4], "q3":
142             sorted_vals[3 * n // 4]}
143
144 class ParameterOptimizer:
145     """原创参数优化算法 - 网格搜索、随机搜索、贝叶斯优化框架"""
146     @staticmethod
147     def grid_search(param_grid: Dict[str, List], eval_fn, top_k: int = 5) ->
148     List[dict]:
149         import itertools
150         keys = list(param_grid.keys())

```

```

151     results = []
152     for combo in itertools.product(*[param_grid[k] for k in keys]):
153         params = dict(zip(keys, combo))
154         try:
155             score = eval_fn(params)
156             results.append({"params": params, "score": round(score, 6)})
157         except: pass
158     return sorted(results, key=lambda x: -x["score"])[:top_k]
159
160     @staticmethod
161     def random_search(param_dist: Dict[str, Tuple[float, float]], n_iter: int =
162 100,
163                       eval_fn=None, top_k: int = 5) -> List[dict]:
164         results = []
165         for _ in range(n_iter):
166             params = {}
167             for k, (lo, hi) in param_dist.items():
168                 params[k] = random.uniform(lo, hi)
169             try:
170                 score = eval_fn(params) if eval_fn else random.random()
171                 results.append({"params": params, "score": round(score, 6)})
172             except: pass
173         return sorted(results, key=lambda x: -x["score"])[:top_k]
174
175 class DataAdapter:
176     """统一数据适配器 - 多源数据接入标准化"""
177     @staticmethod
178     def from_csv(path: str) -> List[dict]:
179         import csv
180         with open(path, 'r', encoding='utf-8-sig') as f:
181             return list(csv.DictReader(f))
182
183     @staticmethod
184     def from_json(path: str) -> Any:
185         with open(path, 'r', encoding='utf-8') as f:
186             return json.load(f)
187
188     @staticmethod
189     def normalize(data: List[dict], fields: List[str] = None) -> List[dict]:
190         if not data: return data
191         keys = fields or list(data[0].keys())
192         numeric_fields = []
193         for k in keys:
194             vals = [d[k] for d in data if k in d and isinstance(d[k], (int, float))]
195             if len(vals) > len(data) * 0.5: numeric_fields.append(k)
196         result = []
197         for row in data:
198             norm_row = dict(row)
199             for k in numeric_fields:
200                 if k in norm_row and isinstance(norm_row[k], (int, float)):

```

```

201         norm_row[f"{k}_norm"] = round(norm_row[k], 4)
202         result.append(norm_row)
203     return result
204
205     @staticmethod
206     def to_training_format(data: List[dict], features: List[str], target: str) ->
207     Tuple[List, List]:
208         X, y = [], []
209         for row in data:
210             try:
211                 X.append([float(row.get(f, 0)) for f in features])
212                 y.append(float(row.get(target, 0)))
213             except: pass
214         return X, y
215
216 class HashValidator:
217     """数据完整性快速校验"""
218     @staticmethod
219     def checksum(data: Any) -> str:
220         try: return hashlib.sha256(json.dumps(data, sort_keys=True,
221 default=str).encode()).hexdigest()[:16]
222         except: return "INVALID"
223
224     @staticmethod
225     def compare(local: Any, remote: Any) -> bool:
226         return HashValidator.checksum(local) == HashValidator.checksum(remote)
227
228 # ==== core\auth.py (193 行) ====
229 """RBAC 认证授权引擎 - 多角色权限调度与会话状态流转"""
230 import hashlib, json, time, uuid
231 from enum import Enum
232 from dataclasses import dataclass, field
233 from typing import Dict, List, Set, Optional, Callable
234
235 class Role(Enum):
236     ADMIN = "管理员"
237     DEVELOPER = "模型开发者"
238     OPERATOR = "普通操作员"
239     GUEST = "访客"
240
241 class SessionState(Enum):
242     ACTIVE = "活跃"
243     IDLE = "空闲"
244     LOCKED = "锁定"
245     EXPIRED = "过期"
246
247 class OpAction(Enum):
248     VIEW = "查看"
249     CREATE = "创建"
250     EDIT = "编辑"

```

```

251     DELETE = "删除"
252     EXECUTE = "执行"
253     EXPORT = "导出"
254     ADMIN = "管理"
255
256     @dataclass
257     class Permission:
258         resource: str
259         actions: Set[OpAction] = field(default_factory=set)
260
261     @dataclass
262     class User:
263         uid: str; username: str; password_hash: str; role: Role
264         permissions: List[Permission] = field(default_factory=list)
265         login_attempts: int = 0; locked_until: float = 0; last_login: float = 0
266
267     class RBACEngine:
268         """原创 RBAC 权限调度引擎 - 支持粒度权限控制、批量分配、权限回收、操作审计"""
269         ROLE_DEFAULTS = {
270             Role.ADMIN: ["*"],
271             Role.DEVELOPER: ["dashboard", "model_designer", "lowcode_editor",
272 "model_training",
273 "vision_lab", "vision_3d", "data_manager",
274 "task_manager",
275 "project_manager", "component_market", "report_center"],
276             Role.OPERATOR: ["dashboard", "model_training", "vision_lab",
277 "data_manager", "task_manager", "report_center"],
278             Role.GUEST: ["dashboard", "report_center"],
279         }
280
281         def __init__(self):
282             self._users: Dict[str, User] = {}
283             self._sessions: Dict[str, dict] = {}
284             self._audit_log: List[dict] = []
285             self._perm_change_callbacks: List[Callable] = []
286             self._init_default_users()
287
288         def _hash(self, pwd: str) -> str:
289             return hashlib.sha256(f"AILab_Salt_{pwd}_2024".encode()).hexdigest()
290
291         def _init_default_users(self):
292             for role, (uname, pwd) in {
293                 Role.ADMIN: ("admin", "admin123"),
294                 Role.DEVELOPER: ("developer", "dev123"),
295                 Role.OPERATOR: ("operator", "op123"),
296                 Role.GUEST: ("guest", "guest123"),
297             }.items():
298                 self.register(uname, pwd, role)
299
300         def register(self, username: str, password: str, role: Role = Role.GUEST) ->

```

```

301 Optional[User]:
302     if any(u.username == username for u in self._users.values()):
303         return None
304     user = User(uid=str(uuid.uuid4())[8:], username=username,
305                 password_hash=self._hash(password), role=role)
306     user.permissions = self._build_permissions(role)
307     self._users[user.uid] = user
308     self._log("REGISTER", user.uid, f"用户注册: {username} 角色:
309 {role.value}")
310     return user
311
312     def authenticate(self, username: str, password: str) -> Optional[User]:
313         for user in self._users.values():
314             if user.username != username: continue
315             if time.time() < user.locked_until:
316                 self._log("LOGIN_BLOCKED", user.uid, "账号锁定中")
317                 return None
318             if self._hash(password) == user.password_hash:
319                 if user.login_attempts >= 5 and time.time() - user.last_login <
320 300:
321                     user.locked_until = time.time() + 600
322                     self._log("LOGIN_LOCKED", user.uid, "多次失败锁定 10 分钟")
323                     return None
324                 user.login_attempts = 0; user.last_login = time.time()
325                 sid = str(uuid.uuid4())[12:]
326                 self._sessions[sid] = {"uid": user.uid, "state":
327 SessionState.ACTIVE,
328 "created": time.time(), "last_active": time.time()}
329                 self._log("LOGIN_SUCCESS", user.uid, f"登录成功 sid={sid}")
330                 return user
331                 user.login_attempts += 1
332                 self._log("LOGIN_FAIL", user.uid, f"密码错误 尝试
333 #{user.login_attempts}")
334                 return None
335
336     def check_permission(self, user: User, resource: str, action: OpAction) ->
337 bool:
338         if user.role == Role.ADMIN: return True
339         for perm in user.permissions:
340             if perm.resource == resource or perm.resource == "*":
341                 return action in perm.actions or OpAction.ADMIN in perm.actions
342         return False
343
344     def grant_permission(self, admin: User, target_uid: str, resource: str, actions:
345 Set[OpAction]) -> bool:
346         if admin.role != Role.ADMIN: return False
347         if target_uid not in self._users: return False
348         target = self._users[target_uid]
349         for perm in target.permissions:
350             if perm.resource == resource:

```

```

351         perm.actions.update(actions); break
352     else:
353         target.permissions.append(Permission(resource=resource,
354 actions=actions))
355         self._log("PERM_GRANT", admin.uid, f"授权 {target_uid} -> {resource}:
356 {[a.value for a in actions]}")
357         self._notify_perm_change()
358         return True
359
360     def revoke_permission(self, admin: User, target_uid: str, resource: str) ->
361 bool:
362         if admin.role != Role.ADMIN: return False
363         if target_uid not in self._users: return False
364         self._users[target_uid].permissions = [p for p in
365 self._users[target_uid].permissions if p.resource != resource]
366         self._log("PERM_REVOKE", admin.uid, f"回收 {target_uid} -> {resource}")
367         self._notify_perm_change()
368         return True
369
370     def batch_grant(self, admin: User, target_uids: List[str], resources:
371 List[str], actions: Set[OpAction]) -> bool:
372         if admin.role != Role.ADMIN: return False
373         results = []
374         for uid in target_uids:
375             for res in resources:
376                 results.append(self.grant_permission(admin, uid, res, actions))
377         return all(results)
378
379     def get_audit_log(self, uid_filter: str = None, limit: int = 100) -> List[dict]:
380         logs = self._audit_log
381         if uid_filter:
382             logs = [l for l in logs if l.get("uid") == uid_filter]
383         return logs[-limit:]
384
385     def validate_session(self, sid: str) -> Optional[User]:
386         if sid not in self._sessions: return None
387         session = self._sessions[sid]
388         if time.time() - session["last_active"] > 3600:
389             session["state"] = SessionState.EXPIRED
390             return None
391         session["last_active"] = time.time()
392         return self._users.get(session["uid"])
393
394     def transition_session(self, sid: str, new_state: SessionState) -> bool:
395         if sid not in self._sessions: return False
396         valid_transitions = {
397             SessionState.ACTIVE: [SessionState.IDLE],
398             SessionState.IDLE: [SessionState.ACTIVE, SessionState.LOCKED,
399 SessionState.EXPIRED],
400             SessionState.LOCKED: [SessionState.ACTIVE, SessionState.EXPIRED],

```



```

401         SessionState.EXPIRED: [SessionState.ACTIVE],
402     }
403     current = self._sessions[sid]["state"]
404     if new_state in valid_transitions.get(current, []):
405         self._sessions[sid]["state"] = new_state
406         self._log("SESSION", sid, f"状态流转: {current.value} ->
407 {new_state.value}")
408         return True
409     return False
410
411     def _build_permissions(self, role: Role) -> List[Permission]:
412         resources = self.ROLE_DEFAULTS.get(role, [])
413         perms = []
414         for res in resources:
415             if res == "*":
416                 perms.append(Permission(resource="*",
417 actions={OpAction.ADMIN}))
418             else:
419                 perms.append(Permission(resource=res, actions={
420 OpAction.VIEW, OpAction.CREATE, OpAction.EDIT,
421 OpAction.EXECUTE}))
422         return perms
423
424     def _log(self, action: str, uid: str, detail: str):
425         self._audit_log.append({"ts": time.time(), "action": action, "uid": uid,
426 "detail": detail})
427
428     def _notify_perm_change(self):
429         for cb in self._perm_change_callbacks: cb()
430
431     def on_perm_change(self, callback: Callable):
432         self._perm_change_callbacks.append(callback)
433
434     def get_all_users(self) -> List[User]:
435         return list(self._users.values())
436
437     def get_user_by_uid(self, uid: str) -> Optional[User]:
438         return self._users.get(uid)
439
440 # 全局单例
441 auth_engine = RBACEngine()
442
443 # ==== core\data_consistency.py (143 行) ====
444 """数据一致性引擎 - 事务管理、快照回溯、MVCC 式数据版本控制"""
445 import time, uuid, copy, threading
446 from dataclasses import dataclass, field
447 from typing import Dict, List, Optional, Any, Callable
448 from enum import Enum
449
450 class TxnStatus(Enum):

```

```

451     ACTIVE = "活跃"
452     COMMITTED = "已提交"
453     ROLLED_BACK = "已回滚"
454
455     @dataclass
456     class DataSnapshot:
457         snap_id: str; flow_id: str; node_id: str; data: Any
458         timestamp: float; version: int; checksum: str = ""
459
460     class DataTransaction:
461         def __init__(self, txn_id: str, flow_id: str):
462             self.txn_id = txn_id; self.flow_id = flow_id
463             self.status = TxnStatus.ACTIVE; self.operations: List[dict] = []
464             self.created = time.time()
465
466     class DataConsistencyEngine:
467         """原创数据一致性引擎 - 事务锁定、异常回滚、快照管理、版本强一致性校验"""
468         def __init__(self):
469             self._data_store: Dict[str, dict] = {}
470             self._snapshots: Dict[str, List[DataSnapshot]] = {}
471             self._txns: Dict[str, DataTransaction] = {}
472             self._locks: Dict[str, str] = {}
473             self._lock = threading.RLock()
474             self._version_counter: Dict[str, int] = {}
475             self._integrity_rules: List[Callable] = []
476
477         def register_node(self, flow_id: str, node_id: str, initial_data: Any = None):
478             key = f"{flow_id}:{node_id}"
479             with self._lock:
480                 self._data_store[key] = {"data": initial_data, "version": 1,
481 "checksum": self._checksum(initial_data)}
482                 self._version_counter.setdefault(flow_id, 0)
483                 self._save_snapshot(flow_id, node_id, initial_data, 1)
484
485         def begin_transaction(self, flow_id: str) -> DataTransaction:
486             txn = DataTransaction(f"TXN_{uuid.uuid4().hex[:8]}", flow_id)
487             with self._lock: self._txns[txn.txn_id] = txn
488             return txn
489
490         def write(self, txn: DataTransaction, node_id: str, data: Any) -> bool:
491             if txn.status != TxnStatus.ACTIVE: return False
492             key = f"{txn.flow_id}:{node_id}"
493             with self._lock:
494                 if key in self._locks and self._locks[key] != txn.txn_id: return False
495                 self._locks[key] = txn.txn_id
496                 current = self._data_store.get(key, {"data": None, "version": 0})
497                 new_version = current["version"] + 1
498                 txn.operations.append({"node_id": node_id, "old":
499 copy.deepcopy(current["data"]),
500 "new": copy.deepcopy(data), "version": new_version, "key": key})

```

```

501         return True
502
503     def commit(self, txn: DataTransaction) -> bool:
504         if txn.status != TxnStatus.ACTIVE: return False
505         with self._lock:
506             try:
507                 for op in txn.operations:
508                     key = op["key"]
509                     if not self._validate_integrity(key, op["new"]): raise
510 ValueError("完整性校验失败")
511                     self._data_store[key] = {"data": op["new"], "version":
512 op["version"],
513                     "checksum": self._checksum(op["new"])}
514                     self._save_snapshot(txn.flow_id, op["node_id"], op["new"],
515 op["version"])
516                     self._locks.pop(key, None)
517                     txn.status = TxnStatus.COMMITTED
518                     return True
519             except Exception:
520                 self.rollback(txn); return False
521
522     def rollback(self, txn: DataTransaction) -> bool:
523         with self._lock:
524             for op in reversed(txn.operations):
525                 key = op["key"]
526                 if op["version"] > 1:
527                     self._data_store[key] = {"data": op["old"], "version":
528 op["version"] - 1,
529                     "checksum": self._checksum(op["old"])}
530                     self._locks.pop(key, None)
531                     txn.status = TxnStatus.ROLLED_BACK
532             return True
533
534     def get_snapshot(self, flow_id: str, node_id: str = None, version: int = None)
535 -> Any:
536         key = f"{flow_id}:{node_id}" if node_id else None
537         if key:
538             store = self._data_store.get(key)
539             return store["data"] if store else None
540         snapshots = self._snapshots.get(flow_id, [])
541         if version:
542             snapshots = [s for s in snapshots if s.version == version]
543         return snapshots[-1].data if snapshots else None
544
545     def export_snapshot(self, flow_id: str, version: int = None) -> dict:
546         snapshots = self._snapshots.get(flow_id, [])
547         if version: snapshots = [s for s in snapshots if s.version == version]
548         return {"flow_id": flow_id, "snapshots": [
549             {"node": s.node_id, "version": s.version, "data": s.data, "checksum":
550 s.checksum}

```

```

551         for s in snapshots]}
552
553     def load_snapshot(self, snapshot_data: dict) -> bool:
554         flow_id = snapshot_data["flow_id"]
555         with self._lock:
556             for snap in snapshot_data["snapshots"]:
557                 key = f"{flow_id}:{snap['node']}"
558                 self._data_store[key] = {"data": snap["data"], "version":
559 snap["version"],
560                                     "checksum": snap["checksum"]}
561             return True
562
563     def verify_consistency(self, flow_id: str) -> dict:
564         """强一致性校验 - 比对所有节点 checksum 确保数据完整性"""
565         results = {"flow_id": flow_id, "consistent": True, "issues": []}
566         for key, store in self._data_store.items():
567             if key.startswith(flow_id):
568                 actual = self._checksum(store["data"])
569                 if actual != store["checksum"]:
570                     results["consistent"] = False
571                     results["issues"].append({"node": key, "expected":
572 store["checksum"], "actual": actual})
573         return results
574
575     def add_integrity_rule(self, rule: Callable[[str, Any], bool]):
576         self._integrity_rules.append(rule)
577
578     def _validate_integrity(self, key: str, data: Any) -> bool:
579         return all(rule(key, data) for rule in self._integrity_rules) if
580 self._integrity_rules else True
581
582     def _save_snapshot(self, flow_id: str, node_id: str, data: Any, version: int):
583         snap = DataSnapshot(snap_id=uuid.uuid4().hex[:8], flow_id=flow_id,
584                             node_id=node_id, data=copy.deepcopy(data), timestamp=time.time(),
585                             version=version, checksum=self._checksum(data))
586         self._snapshots.setdefault(flow_id, []).append(snap)
587
588     @staticmethod
589     def _checksum(data: Any) -> str:
590         import hashlib, json
591         try:         return         hashlib.md5(json.dumps(data,
592 default=str).encode()).hexdigest()[:8]
593         except: return "00000000"
594
595     def get_node_history(self, flow_id: str, node_id: str) -> List[DataSnapshot]:
596         return [s for s in self._snapshots.get(flow_id, []) if s.node_id == node_id]
597
598 data_engine = DataConsistencyEngine()
599
600 # ==== core\rule_engine.py (192 行) ====

```

```

601 """业务规则引擎 - 图形化条件配置、规则触发、冲突检测"""
602 from enum import Enum
603 from dataclasses import dataclass, field
604 from typing import Dict, List, Optional, Callable, Any, Set
605 import time, uuid, itertools
606
607 class TriggerType(Enum):
608     DATA = "数据触发"
609     TIMER = "定时触发"
610     NODE = "流程节点触发"
611
612 class LogicOp(Enum):
613     AND = "与"; OR = "或"; NOT = "非"
614
615 class CompareOp(Enum):
616     EQ = "等于"; NE = "不等于"; GT = "大于"; LT = "小于"
617     GTE = "大于等于"; LTE = "小于等于"; IN = "包含"; RE = "正则匹配"
618
619 @dataclass
620 class Condition:
621     field: str; op: CompareOp; value: Any; logic: LogicOp = LogicOp.AND
622
623 @dataclass
624 class RuleAction:
625     action_type: str; params: dict = field(default_factory=dict)
626
627 @dataclass
628 class Rule:
629     rule_id: str; name: str; description: str = ""
630     trigger: TriggerType = TriggerType.DATA
631     conditions: List[Condition] = field(default_factory=list)
632     actions: List[RuleAction] = field(default_factory=list)
633     priority: int = 5; enabled: bool = True
634     template_id: str = ""; group: str = "默认"
635
636 class RuleEngine:
637     """原创规则引擎 - 条件组合、多触发方式、冲突检测、模板复用"""
638     def __init__(self):
639         self._rules: Dict[str, Rule] = {}
640         self._templates: Dict[str, Rule] = {}
641         self._trigger_handlers: Dict[TriggerType, List[Callable]] = {
642             t: [] for t in TriggerType
643         }
644         self._execution_log: List[dict] = []
645
646     def create_rule(self, name: str, trigger: TriggerType, conditions:
647         List[Condition], actions: List[RuleAction], priority: int = 5, group: str = "
648         默认") -> Rule:
649         rid = f"R{str(uuid.uuid4())[:8]}"
650         rule = Rule(rule_id=rid, name=name, trigger=trigger,

```

```

651 conditions=conditions,
652             actions=actions, priority=max(1, min(10, priority)),
653 group=group)
654     self._rules[rid] = rule
655     return rule
656
657     def evaluate(self, rule_id: str, context: dict) -> bool:
658         rule = self._rules.get(rule_id)
659         if not rule or not rule.enabled: return False
660         results = []
661         for cond in rule.conditions:
662             val = context.get(cond.field)
663             r = self._compare(val, cond.op, cond.value)
664             results.append(r)
665         if not results: return True
666         final = results[0]
667         for i, cond in enumerate(rule.conditions[1:], 1):
668             if cond.logic == LogicOp.AND:
669                 final = final and results[i]
670             elif cond.logic == LogicOp.OR:
671                 final = final or results[i]
672             elif cond.logic == LogicOp.NOT:
673                 final = final and not results[i]
674         return final
675
676     def evaluate_all(self, context: dict, trigger: TriggerType = None) ->
677 List[Rule]:
678         fired = []
679         for rule in sorted(self._rules.values(), key=lambda r: -r.priority):
680             if not rule.enabled: continue
681             if trigger and rule.trigger != trigger: continue
682             if self.evaluate(rule.rule_id, context):
683                 fired.append(rule)
684                 self._log(rule.rule_id, "FIRED", str(context)[:100])
685         return fired
686
687     def execute_actions(self, rule: Rule, context: dict) -> dict:
688         results = {}
689         for action in rule.actions:
690             try:
691                 handler = self._get_action_handler(action.action_type)
692                 results[action.action_type] = handler(context, action.params)
693             except Exception as e:
694                 results[action.action_type] = f"ERROR: {str(e)}"
695         self._log(rule.rule_id, "EXECUTED", str(results)[:100])
696         return results
697
698     def detect_conflicts(self) -> List[dict]:
699         """原创冲突检测算法 - 基于条件重叠图的规则冲突发现"""
700         conflicts = []

```

```

701     rules_list = list(self._rules.values())
702     for i, j in itertools.combinations(range(len(rules_list)), 2):
703         r1, r2 = rules_list[i], rules_list[j]
704         if not r1.enabled or not r2.enabled: continue
705         score = self._conflict_score(r1, r2)
706         if score > 0.5:
707             conflicts.append({"rule_a": r1.rule_id, "rule_b": r2.rule_id,
708                             "score": score, "type": self._classify_conflict(r1, r2),
709                             "detail": f"规则 [{r1.name}] 与 [{r2.name}] 存在{score:.0%}"
710                             重叠"})
711     return conflicts
712
713     def save_as_template(self, rule_id: str, template_name: str) -> str:
714         if rule_id not in self._rules: return ""
715         tid = f"TPL_{str(uuid.uuid4())[:6]}"
716         template = copy_rule(self._rules[rule_id])
717         template.rule_id = tid; template.name = template_name
718         template.template_id = tid
719         self._templates[tid] = template
720         return tid
721
722     def create_from_template(self, template_id: str, name: str) -> Optional[Rule]:
723         if template_id not in self._templates: return None
724         new_rule = copy_rule(self._templates[template_id])
725         new_rule.rule_id = f"R{str(uuid.uuid4())[:8]}"
726         new_rule.name = name
727         self._rules[new_rule.rule_id] = new_rule
728         return new_rule
729
730     def _conflict_score(self, r1: Rule, r2: Rule) -> float:
731         fields1 = {c.field for c in r1.conditions}
732         fields2 = {c.field for c in r2.conditions}
733         if not fields1 & fields2: return 0.0
734         shared = fields1 & fields2
735         conflicting = 0
736         for field in shared:
737             c1 = next((c for c in r1.conditions if c.field == field), None)
738             c2 = next((c for c in r2.conditions if c.field == field), None)
739             if c1 and c2:
740                 if c1.op != c2.op or c1.value != c2.value:
741                     conflicting += 1
742         return conflicting / len(shared) if shared else 0.0
743
744     def _classify_conflict(self, r1: Rule, r2: Rule) -> str:
745         if r1.trigger != r2.trigger: return "触发方式冲突"
746         actions1 = {a.action_type for a in r1.actions}
747         actions2 = {a.action_type for a in r2.actions}
748         if actions1 & actions2: return "动作冲突"
749         return "条件重叠"
750

```

```

751 def _compare(self, value: Any, op: CompareOp, target: Any) -> bool:
752     try:
753         if op == CompareOp.EQ: return value == target
754         if op == CompareOp.NE: return value != target
755         if op == CompareOp.GT: return float(value) > float(target)
756         if op == CompareOp.LT: return float(value) < float(target)
757         if op == CompareOp.GTE: return float(value) >= float(target)
758         if op == CompareOp.LTE: return float(value) <= float(target)
759         if op == CompareOp.IN: return target in str(value)
760         if op == CompareOp.RE:
761             import re; return bool(re.search(str(target), str(value)))
762     except: return False
763     return False
764
765 def _get_action_handler(self, action_type: str):
766     return self._action_handlers
767
768 def _log(self, rule_id: str, event: str, detail: str):
769     self._execution_log.append({"ts": time.time(), "rule_id": rule_id,
770     "event": event, "detail": detail})
771
772 def get_rules(self, trigger: TriggerType = None) -> List[Rule]:
773     rules = list(self._rules.values())
774     if trigger: rules = [r for r in rules if r.trigger == trigger]
775     return sorted(rules, key=lambda r: -r.priority)
776
777 def delete_rule(self, rule_id: str) -> bool:
778     return self._rules.pop(rule_id, None) is not None
779
780 def toggle_rule(self, rule_id: str) -> bool:
781     if rule_id in self._rules:
782         self._rules[rule_id].enabled = not self._rules[rule_id].enabled
783     return True
784     return False
785
786 _action_handlers = {
787     "alert": lambda ctx, p: f"告警: {p.get('msg', '')}",
788     "abort": lambda ctx, p: "流程中止",
789     "log": lambda ctx, p: f"日志: {p.get('text', '')}",
790     "transform": lambda ctx, p: {k: ctx.get(k, v) for k, v in p.items() if k !=
791 'action_type'},
792     "route": lambda ctx, p: f"路由到: {p.get('target', '')}",
793 }
794
795 def copy_rule(r: Rule) -> Rule:
796     import copy
797     return copy.deepcopy(r)
798
799 rule_engine = RuleEngine()
800

```



```

801 # ==== core\state_machine.py (157 行) ====
802 """状态流转引擎 - 编排流程五态管理、状态锁定与回滚"""
803 from enum import Enum
804 from dataclasses import dataclass, field
805 from typing import Dict, List, Optional, Callable, Any
806 import time, copy, uuid
807
808 class FlowState(Enum):
809     EDITING = "编辑态"
810     DEBUGGING = "调试态"
811     RUNNING = "运行态"
812     PAUSED = "暂停态"
813     ABORTED = "异常终止态"
814
815 class TaskState(Enum):
816     QUEUED = "排队中"
817     PREPARING = "准备中"
818     EXECUTING = "执行中"
819     PAUSED = "已暂停"
820     COMPLETED = "已完成"
821     FAILED = "已失败"
822     CANCELLED = "已取消"
823
824 @dataclass
825 class StateSnapshot:
826     state_id: str; flow_id: str; state: FlowState; data: dict
827     timestamp: float; node_states: dict = field(default_factory=dict)
828
829 class FlowStateMachine:
830     """原创流程状态机 - 编辑/调试/运行/暂停/异常终止五大状态流转"""
831     VALID_TRANSITIONS = {
832         FlowState.EDITING: [FlowState.DEBUGGING, FlowState.ABORTED],
833         FlowState.DEBUGGING: [FlowState.EDITING, FlowState.RUNNING,
834 FlowState.ABORTED],
835         FlowState.RUNNING: [FlowState.PAUSED, FlowState.ABORTED,
836 FlowState.EDITING],
837         FlowState.PAUSED: [FlowState.RUNNING, FlowState.EDITING,
838 FlowState.ABORTED],
839         FlowState.ABORTED: [FlowState.EDITING],
840     }
841
842     def __init__(self):
843         self._flows: Dict[str, dict] = {}
844         self._snapshots: Dict[str, List[StateSnapshot]] = {}
845         self._locks: Dict[str, str] = {}
846         self._listeners: List[Callable] = []
847
848     def create_flow(self, flow_id: str, metadata: dict = None) -> str:
849         self._flows[flow_id] = {"state": FlowState.EDITING, "nodes": {},
850             "meta": metadata or {}, "created": time.time(), "version": 1}

```

```

851         self._snapshots[flow_id] = []
852         self._notify(flow_id, FlowState.EDITING, None)
853         return flow_id
854
855     def get_state(self, flow_id: str) -> Optional[FlowState]:
856         flow = self._flows.get(flow_id)
857         return flow["state"] if flow else None
858
859     def transition(self, flow_id: str, target: FlowState, operator: str = "") ->
860 bool:
861         flow = self._flows.get(flow_id)
862         if not flow: return False
863         current = flow["state"]
864         if target not in self.VALID_TRANSITIONS.get(current, []): return False
865         if self._locks.get(flow_id) and self._locks[flow_id] != operator: return
866 False
867         self._save_snapshot(flow_id)
868         flow["state"] = target
869         flow["version"] += 1
870         self._notify(flow_id, target, current)
871         return True
872
873     def lock(self, flow_id: str, operator: str) -> bool:
874         if flow_id in self._locks: return False
875         self._locks[flow_id] = operator
876         return True
877
878     def unlock(self, flow_id: str, operator: str) -> bool:
879         if self._locks.get(flow_id) == operator:
880             del self._locks[flow_id]; return True
881         return False
882
883     def rollback(self, flow_id: str) -> bool:
884         flow = self._flows.get(flow_id)
885         if not flow: return False
886         snapshots = self._snapshots.get(flow_id, [])
887         if not snapshots: return False
888         snapshot = snapshots[-1]
889         flow["state"] = snapshot.state
890         flow["nodes"] = snapshot.node_states.copy()
891         flow["version"] = max(1, flow["version"] - 1)
892         self._notify(flow_id, flow["state"], None)
893         return True
894
895     def _save_snapshot(self, flow_id: str):
896         flow = self._flows[flow_id]
897         snap = StateSnapshot(state_id=str(uuid.uuid4())[:8], flow_id=flow_id,
898                             state=flow["state"], data=copy.deepcopy(flow.get("meta", {})),
899                             timestamp=time.time(), node_states=copy.deepcopy(flow.get("nodes",
900 {})))

```

```

901         self._snapshots.setdefault(flow_id, []).append(snap)
902
903     def get_snapshots(self, flow_id: str) -> List[StateSnapshot]:
904         return self._snapshots.get(flow_id, [])
905
906     def on_state_change(self, callback: Callable):
907         self._listeners.append(callback)
908
909     def _notify(self, flow_id, new_state, old_state):
910         for cb in self._listeners:
911             try: cb(flow_id, new_state, old_state)
912             except: pass
913
914 class TaskStateMachine:
915     """任务状态流转引擎 - 排队/准备/执行/暂停/完成/失败/取消七态管理"""
916     TRANSITIONS = {
917         TaskState.QUEUED: [TaskState.PREPARING, TaskState.CANCELLED],
918         TaskState.PREPARING: [TaskState.EXECUTING, TaskState.FAILED,
919 TaskState.CANCELLED],
920         TaskState.EXECUTING: [TaskState.PAUSED, TaskState.COMPLETED,
921 TaskState.FAILED, TaskState.CANCELLED],
922         TaskState.PAUSED: [TaskState.EXECUTING, TaskState.FAILED,
923 TaskState.CANCELLED],
924         TaskState.FAILED: [TaskState.QUEUED, TaskState.CANCELLED],
925     }
926
927     def __init__(self):
928         self._tasks: Dict[str, dict] = {}
929         self._listeners: List[Callable] = []
930
931     def create_task(self, task_id: str, task_type: str, priority: int = 5) -> str:
932         self._tasks[task_id] = {"state": TaskState.QUEUED, "type": task_type,
933             "priority": min(10, max(1, priority)), "created": time.time(),
934             "started": 0, "finished": 0, "error": None}
935         return task_id
936
937     def transition(self, task_id: str, target: TaskState, error: str = None) ->
938 bool:
939         task = self._tasks.get(task_id)
940         if not task: return False
941         if target not in self.TRANSITIONS.get(task["state"], []): return False
942         task["state"] = target
943         if target == TaskState.EXECUTING: task["started"] = time.time()
944         if target in (TaskState.COMPLETED, TaskState.FAILED,
945 TaskState.CANCELLED):
946             task["finished"] = time.time()
947         task["error"] = error
948         self._notify(task_id, target)
949         return True
950

```

```

951     def get_state(self, task_id: str) -> Optional[TaskState]:
952         t = self._tasks.get(task_id); return t["state"] if t else None
953
954     def on_transition(self, callback: Callable):
955         self._listeners.append(callback)
956
957     def _notify(self, task_id, state):
958         for cb in self._listeners:
959             try: cb(task_id, state)
960             except: pass
961
962     def get_tasks_by_state(self, state: TaskState) -> List[dict]:
963         return [{"id": tid, **t} for tid, t in self._tasks.items() if t["state"]
964 == state]
965
966     def get_all_tasks(self) -> List[dict]:
967         return [{"id": tid, **t} for tid, t in self._tasks.items()]
968
969 flow_sm = FlowStateMachine()
970 task_sm = TaskStateMachine()
971
972 # ==== main.py (27 行) ====
973 """低代码驱动的人工智能模型快速构建与可视化编排平台 主入口"""
974 import sys, os
975 sys.path.insert(0, os.path.dirname(__file__))
976
977 from PyQt6.QtWidgets import QApplication
978 from PyQt6.QtCore import Qt
979 from ui.styles import load_stylesheet
980 from ui.login import LoginWindow
981 from ui.main_window import MainWindow
982 from core.auth import auth_engine
983
984 def main():
985     app = QApplication(sys.argv)
986     app.setStyle("Fusion")
987     app.setStyleSheet(load_stylesheet())
988     app.setApplicationName("低代码驱动的人工智能模型快速构建与可视化编排平台")
989     login = LoginWindow()
990     def on_login(user):
991         main_win = MainWindow(user)
992         main_win.show()
993         login._main_win = main_win
994     login.login_success.connect(on_login)
995     login.show()
996     sys.exit(app.exec())
997
998 if __name__ == "__main__":
999     main()
1000

```

```

1001 # ==== modules\__init__.py (1 行) ====
1002 # 低代码 AI 平台 业务模块层
1003
1004 # ==== modules\component_market.py (176 行) ====
1005 """组件市场模块 - AI 节点/脚本/规则模板/视觉算子在线预览与本地导入"""
1006 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
1007 QLabel,
1008     QGroupBox, QListWidget, QTextEdit, QSplitter, QTabWidget, QMessageBox,
1009 QInputDialog)
1010 from PyQt6.QtCore import Qt
1011 from core.auth import Role, OpAction, auth_engine
1012 from core.rule_engine import rule_engine
1013
1014 COMPONENTS = {
1015     "AI 节点": [
1016         {"name": "数据归一化节点", "desc": "支持 Z-score/MinMax/Robust 三种归一化
1017 方式", "author": "平台官方", "rating": 4.8},
1018         {"name": "特征选择节点", "desc": "基于互信息/卡方检验/树模型的特征选择",
1019 "author": "社区", "rating": 4.5},
1020         {"name": "注意力机制节点", "desc": "Transformer 自注意力层封装", "author":
1021 "平台官方", "rating": 4.9},
1022         {"name": "数据增强节点", "desc": "图像翻转/旋转/裁剪/色彩抖动增强",
1023 "author": "社区", "rating": 4.6},
1024         {"name": "模型融合节点", "desc": "Bagging/Boosting/Stacking 集成策略",
1025 "author": "官方", "rating": 4.7},
1026         {"name": "ONNX 导出节点", "desc": "模型转 ONNX 通用格式导出", "author": "
1027 平台官方", "rating": 4.4},
1028     ],
1029     "低代码脚本": [
1030         {"name": "数据清洗模板", "desc": "缺失值填充/异常值检测/编码转换",
1031 "author": "社区精选", "rating": 4.3},
1032         {"name": "交叉验证模板", "desc": "K 折交叉验证/分层采样评估", "author": "
1033 官方", "rating": 4.7},
1034         {"name": "超参搜索模板", "desc": "网格搜索/随机搜索/贝叶斯优化", "author":
1035 "社区精选", "rating": 4.8},
1036         {"name": "推理服务模板", "desc": "模型 API 服务化部署脚本", "author": "
1037 平台官方", "rating": 4.5},
1038         {"name": "数据可视化模板", "desc": "训练曲线/混淆矩阵/特征重要性图表",
1039 "author": "社区", "rating": 4.2},
1040     ],
1041     "规则模板": [
1042         {"name": "训练早停规则", "desc": "验证损失连续 N 轮不下降自动停止",
1043 "author": "官方", "rating": 4.9},
1044         {"name": "数据质量检查规则", "desc": "输入数据维度/类型/范围自动校验",
1045 "author": "平台官方", "rating": 4.6},
1046         {"name": "模型版本管理规则", "desc": "模型保存/回退/灰度发布流程规则",
1047 "author": "社区精选", "rating": 4.4},
1048         {"name": "资源监控规则", "desc": "GPU 内存/CPU 使用率超阈值告警", "author":
1049 "官方", "rating": 4.7},
1050     ],

```

```

1051     "视觉算子": [
1052         {"name": "自适应阈值算子", "desc": "Otsu/Adaptive 双模式自适应阈值分割",
1053 "author": "平台官方", "rating": 4.5},
1054         {"name": "特征匹配算子", "desc": "SIFT/ORB/AKAZE 多算法特征匹配",
1055 "author": "社区", "rating": 4.3},
1056         {"name": "目标追踪算子", "desc": "KCF/CSRT/DeepSORT 目标追踪框架",
1057 "author": "官方", "rating": 4.8},
1058         {"name": "3D 重建算子", "desc": "多视角立体视觉 3D 重建", "author": "社区
1059 精选", "rating": 4.6},
1060     ],
1061 }
1062
1063 class ComponentMarketWidget(QWidget):
1064     def __init__(self, user):
1065         super().__init__()
1066         self.user = user; self._installed = set()
1067         self._setup_ui()
1068
1069     def _setup_ui(self):
1070         layout = QVBoxLayout(self)
1071
1072         header = QHBoxLayout()
1073         title = QLabel("    组件市场")
1074         title.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
1075         header.addWidget(title); header.addStretch()
1076         header.addWidget(QPushButton("    导 入 本 地 组 件    ",
1077 clicked=self._import_local))
1078         header.addWidget(QPushButton("    刷 新 市 场    ",
1079 clicked=self._refresh_market))
1080         header.addWidget(QPushButton("    我 的 组 件    ",
1081 clicked=self._show_installed))
1082         layout.addLayout(header)
1083
1084         main = QSplitter(Qt.Orientation.Horizontal)
1085
1086         left = QVBoxLayout()
1087         left.setAlignment(Qt.AlignmentFlag.AlignTop)
1088
1089         cat_gb = QGroupBox("    组件分类")
1090         cl = QVBoxLayout()
1091         self.cat_list = QListWidget()
1092         self.cat_list.addItems(COMONENTS.keys())
1093         self.cat_list.itemClicked.connect(self._on_category_select)
1094         cl.addWidget(self.cat_list)
1095         cat_gb.setLayout(cl)
1096         left.addWidget(cat_gb)
1097
1098         filter_gb = QGroupBox("    筛选排序")
1099         fl = QVBoxLayout()
1100         fl.addWidget(QPushButton("★ 按评分排序", clicked=self._sort_by_rating))

```

```

1101         fl.addWidget(QPushButton("    最新发布", clicked=self._sort_by_newest))
1102         fl.addWidget(QPushButton("    已安装", clicked=self._filter_installed))
1103         fl.addWidget(QPushButton("    官方组件", clicked=self._filter_official))
1104         filter_gb.setLayout(fl)
1105         left.addWidget(filter_gb)
1106
1107         left_w = QWidget(); left_w.setLayout(left); left_w.setMaximumWidth(220)
1108
1109         right = QVBoxLayout()
1110         right.addWidget(QLabel("        可        用        组        件        ",
1111 styleSheet="color:#E65100;font-weight:bold;"))
1112         self.comp_list = QListWidget()
1113         self.comp_list.itemClicked.connect(self._on_component_select)
1114         right.addWidget(self.comp_list)
1115
1116         self.detail_text = QTextEdit(); self.detail_text.setReadOnly(True)
1117         self.detail_text.setMaximumHeight(160)
1118         self.detail_text.setStyleSheet("background:#FFFDE7;color:#4E342E;")
1119         right.addWidget(self.detail_text)
1120
1121         action_row = QHBoxLayout()
1122         action_row.addWidget(QPushButton("        安        装        组        件        ",
1123 clicked=self._install_component))
1124         action_row.addWidget(QPushButton("        预        览        详        情        ",
1125 clicked=self._preview_component))
1126         action_row.addWidget(QPushButton("        复        制        脚        本        ",
1127 clicked=self._copy_script))
1128         action_row.addWidget(QPushButton("        卸        载        ",
1129 clicked=self._uninstall_component))
1130         right.addLayout(action_row)
1131
1132         right_w = QWidget(); right_w.setLayout(right)
1133         main.addWidget(left_w); main.addWidget(right_w)
1134         main.setSizes([220, 900])
1135         layout.addWidget(main)
1136
1137         self._on_category_select(self.cat_list.item(0))
1138
1139         def _on_category_select(self, item):
1140             category = item.text()
1141             self.comp_list.clear()
1142             for comp in COMPONENTS.get(category, []):
1143                 status = "✔" if comp["name"] in self._installed else "↓"
1144                 self.comp_list.addItem(f"{status} {comp['name']} | ★{comp['rating']}
1145 | {comp['author']}")
1146
1147         def _on_component_select(self, item):
1148             text = item.text()
1149             name = text.split(" | ")[0].replace("✔", "").replace("↓", "")
1150             for cat in COMPONENTS.values():

```

```

1151         for comp in cat:
1152             if comp["name"] == name:
1153                 self.detail_text.setPlainText(
1154                     f"名称: {comp['name']}\n 描述: {comp['desc']}\n 作者:
1155 {comp['author']}\n 评分: ★{comp['rating']}\n"
1156                     f"状态: {'已安装' if name in self._installed else '未安
1157 装'}\n"
1158                     f"兼容性: Python3.8+ | PyQt6 | 低代码 AI 平台 v1.0+\n 大
1159 小: ~{len(comp['desc']) * 5}KB")
1160                 return
1161
1162     def _install_component(self):
1163         item = self.comp_list.currentItem()
1164         if item:
1165             name = item.text().split(" | ")[0].replace("✓", "").replace("↓", "")
1166             self._installed.add(name)
1167             QMessageBox.information(self, "安装", f"组件 [{name}] 安装成功! \n
1168 可在对应模块中使用")
1169             self._on_category_select(self.cat_list.currentItem()) or
1170 self.cat_list.item(0))
1171
1172     def _preview_component(self):
1173         QMessageBox.information(self, "预览", "组件详情预览\n 包含源代码、使用示
1174 例、依赖说明")
1175
1176     def _copy_script(self):
1177         item = self.comp_list.currentItem()
1178         if item:
1179             QMessageBox.information(self, "复制",
1180                                     f"组件脚本已复制到剪贴板\n 可在低代码编辑器中粘贴使用")
1181
1182     def _uninstall_component(self):
1183         item = self.comp_list.currentItem()
1184         if item:
1185             name = item.text().split(" | ")[0].replace("✓", "").replace("↓", "")
1186             if name in self._installed:
1187                 self._installed.discard(name)
1188                 QMessageBox.information(self, "卸载", f"组件 [{name}] 已卸载")
1189
1190     def _import_local(self):
1191         QMessageBox.information(self, "导入", "支持导入本地组件包 (.zip/.py)\n
1192 导入后可在市场中管理")
1193
1194     def _refresh_market(self):
1195         QMessageBox.information(self, "刷新", "组件市场已刷新\n 共 24 个可用组件
1196 ")
1197
1198     def _show_installed(self):
1199         msg = "\n".join(self._installed) if self._installed else "暂无已安装组件
1200 "

```



```

1201         QMessageBox.information(self, "我的组件", msg)
1202
1203     def _sort_by_rating(self):
1204         self.comp_list.clear()
1205         self.comp_list.addItems([f"★{i+1}. 高分组件示例 {i+1}" for i in
1206 range(5)])
1207
1208     def _sort_by_newest(self):
1209         self.comp_list.clear()
1210         self.comp_list.addItems([f"    最新组件示例 {i+1}" for i in range(5)])
1211
1212     def _filter_installed(self):
1213         self.comp_list.clear()
1214         self.comp_list.addItems([f"✔ {n}" for n in self._installed] or ["暂无已
1215 安装组件"])
1216
1217     def _filter_official(self):
1218         QMessageBox.information(self, "官方组件", "平台官方组件 12 个\n 社区精选
1219 8 个\n 第三方 4 个")
1220
1221     def get_module_widget(user):
1222         return ComponentMarketWidget(user)
1223
1224     # ==== modules\dashboard.py (126 行) ====
1225     """工作台 - 项目总览、快速导航、最近项目、模板入口"""
1226     from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel,
1227 QPushButton,
1228 QFrame, QGridLayout, QListWidget, QScrollArea, QGroupBox, QProgressBar)
1229     from PyQt6.QtCore import Qt, QTimer
1230     from PyQt6.QtGui import QFont
1231     from core.auth import Role, OpAction, auth_engine
1232     from core.state_machine import FlowState, flow_sm, task_sm, TaskState
1233     from core.rule_engine import rule_engine, TriggerType
1234     import time, random
1235
1236     class DashboardWidget(QWidget):
1237         def __init__(self, user):
1238             super().__init__()
1239             self.user = user
1240             self._setup_ui()
1241             self._load_data()
1242
1243         def _setup_ui(self):
1244             layout = QVBoxLayout(self)
1245             layout.setSpacing(12)
1246
1247             header = QHBoxLayout()
1248             title = QLabel("    工作台")
1249             title.setStyleSheet("color:#4E342E;font-size:24px;font-weight:bold;")
1250             header.addWidget(title)

```

```

1251     header.addStretch()
1252     layout.addLayout(header)
1253
1254     stats_row = QHBoxLayout()
1255     stats = [
1256         (" 总项目数", "5", "#1565C0"),
1257         (" 运行中", "2", "#2E7D32"),
1258         (" 暂停", "1", "#E65100"),
1259         ("✓ 已完成", "12", "#0277BD"),
1260         ("✗ 异常", "0", "#C62828"),
1261         (" 规则数", "8", "#6A1B9A"),
1262     ]
1263     for label, value, color in stats:
1264         card = QFrame()
1265
1266         card.setStyleSheet(f"background:qlineargradient(x1:0,y1:0,x2:0,y2:1,stop:0
1267 #FFFFFF,stop:1                                #FFF8E1);border:1px                    solid
1268 #FFD54F;border-radius:10px;padding:12px;")
1269         card.setFixedHeight(100)
1270         cl = QVBoxLayout(card)
1271         vl = QLabel(value);
1272         vl.setStyleSheet(f"color:{color};font-size:36px;font-weight:bold;")
1273         vl.setAlignment(Qt.AlignmentFlag.AlignCenter)
1274         ll = QLabel(label);
1275         ll.setStyleSheet("color:#5D4037;font-size:12px;font-weight:bold;")
1276         ll.setAlignment(Qt.AlignmentFlag.AlignCenter)
1277         cl.addWidget(vl); cl.addWidget(ll)
1278         stats_row.addWidget(card)
1279     layout.addLayout(stats_row)
1280
1281     main_row = QHBoxLayout()
1282
1283     left_panel = QVBoxLayout()
1284     tmpl_gb = QGroupBox("  AI 模型模板 - 一键新建项目")
1285
1286     tmpl_gb.setStyleSheet("QGroupBox{color:#E65100;font-weight:bold;font-size:14px;
1287 }")
1288     tmpl_grid = QGridLayout()
1289     templates = [
1290         (" 分类模型", "支持二分类/多分类任务, 预置 RF/SVM/MLP"), (" 回归
1291 模型", "线性回归/多项式回归/梯度提升回归"),
1292         (" 目标检测", "YOLO 架构/SSD/区域提议检测"), (" 图像分割",
1293 "U-Net/DeepLab/语义分割"),
1294         (" 文本分析", "NLP 情感分析/实体识别/文本分类"), (" 时序预测",
1295 "LSTM/Prophet/ARIMA 时序预测"),
1296     ]
1297     for i, (name, desc) in enumerate(templates):
1298         btn = QPushButton(f"{name}\n{desc}")
1299         btn.setMinimumHeight(70);
1300         btn.setStyleSheet("text-align:left;padding:12px;")

```

```

1301         btn.clicked.connect(lambda checked, n=name:
1302 self._create_from_template(n))
1303         tpl_grid.addWidget(btn, i // 3, i % 3)
1304         tpl_gb.setLayout(tpl_grid)
1305         left_panel.addWidget(tpl_gb)
1306
1307         recent_gb = QGroupBox(" 最近项目")
1308         self.recent_list = QListWidget()
1309         left_panel.addWidget(recent_gb)
1310         left_panel.addWidget(self.recent_list)
1311
1312         right_panel = QVBoxLayout()
1313
1314         flow_gb = QGroupBox(" 流程状态监控")
1315         self.flow_list = QListWidget()
1316         flow_gb.setLayout(QVBoxLayout())
1317         flow_gb.layout().addWidget(self.flow_list)
1318         right_panel.addWidget(flow_gb)
1319
1320         task_gb = QGroupBox(" 任务队列")
1321         self.task_list = QListWidget()
1322         task_gb.setLayout(QVBoxLayout())
1323         task_gb.layout().addWidget(self.task_list)
1324         right_panel.addWidget(task_gb)
1325
1326         alert_gb = QGroupBox(" 系统通知")
1327         self.alert_list = QListWidget()
1328         alert_gb.setLayout(QVBoxLayout())
1329         alert_gb.layout().addWidget(self.alert_list)
1330         right_panel.addWidget(alert_gb)
1331
1332         main_row.addLayout(left_panel, 3)
1333         main_row.addLayout(right_panel, 2)
1334         layout.addLayout(main_row)
1335
1336         def _load_data(self):
1337             projects = [("图像分类项目-v3", "运行态", "2024-06-12"), ("目标检测项目
1338 ", "编辑态", "2024-06-11"),
1339                         ("文本情感分析", "已完成", "2024-06-10"), ("时序销售预测", "
1340 暂停态", "2024-06-09"),
1341                         ("3D 点云分割", "调试态", "2024-06-08")]
1342             for name, state, date in projects:
1343                 self.recent_list.addItem(f" {name} [{state}] {date}")
1344
1345             flows = [("图像分类流程", "运行态"), ("目标检测流程", "编辑态"), ("文本
1346 分析流程", "已完成"),
1347                     ("3D 点云流程", "暂停态"), ("时序预测流程", "调试态")]
1348             for name, state in flows:
1349                 self.flow_list.addItem(f" {name} → {state}")
1350

```

```

1351         tasks = [("训练-图像分类 v3", "执行中 65%"), ("推理-目标检测", "排队中 优
1352 优先级 8"),
1353                  ("数据预处理-时序", "已完成"), ("模型导出-YOLO", "排队中")]
1354         for name, status in tasks:
1355             self.task_list.addItem(f" {name} [{status}]")
1356
1357         alerts = ["✔ 规则冲突检测: 0 个冲突", "⚠ 磁盘空间剩余: 45.2GB", "i 今日
1358 已执行任务: 23 个",
1359                 " 权限变更: admin 授权了 operator"]
1360         for alert in alerts:
1361             self.alert_list.addItem(f" {alert}")
1362
1363         def _create_from_template(self, tpl_name: str):
1364             from PyQt6.QtWidgets import QMessageBox
1365             QMessageBox.information(self, "新建项目", f"基于模板 [{tpl_name}] 创建
1366 新项目\n请在模型设计器中开始编排流程。")
1367
1368         def get_module_widget(user):
1369             return DashboardWidget(user)
1370
1371         # ==== modules\data_manager.py (227 行) ====
1372         """数据管理模块 - 多源数据接入、交互式编辑、事务管理、快照与回溯"""
1373         from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
1374         QLabel,
1375         QGroupBox, QTableWidget, QTableWidgetItem, QHeaderView, QTextEdit, QSplitter,
1376         QFileDialog, QMessageBox, QComboBox, QLineEdit, QListWidget)
1377         from PyQt6.QtCore import Qt
1378         from core.auth import Role, OpAction, auth_engine
1379         from core.data_consistency import data_engine, DataTransaction
1380         from core.algorithms import DataAdapter, FeatureAnalyzer, HashValidator
1381         from core.rule_engine import rule_engine, TriggerType, Condition, CompareOp,
1382         RuleAction
1383         import json, csv, io
1384
1385         class DataManagerWidget(QWidget):
1386             def __init__(self, user):
1387                 super().__init__()
1388                 self.user = user; self._current_data = []; self._flow_id = "data_flow_1"
1389                 self._txn = None
1390                 data_engine.register_node(self._flow_id, "main", [])
1391                 data_engine.add_integrity_rule(lambda k, d: isinstance(d, (list, dict)))
1392                 self._setup_ui(); self._init_demo_data()
1393
1394             def _setup_ui(self):
1395                 layout = QVBoxLayout(self)
1396
1397                 header = QHBoxLayout()
1398                 title = QLabel(" 数据管理中心")
1399                 title.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
1400                 header.addWidget(title); header.addStretch()

```

```

1401 import_btns = [
1402     (" CSV", self._import_csv), (" JSON", self._import_json),
1403     (" TXT", self._import_txt), (" 图片", self._import_image),
1404     (" 导出", self._export_data), (" 分析", self._analyze_data),
1405 ]
1406 for label, handler in import_btns:
1407     header.addWidget(QPushButton(label, clicked=handler))
1408 layout.addLayout(header)
1409
1410 main = QSplitter(Qt.Orientation.Horizontal)
1411
1412 left = QVBoxLayout()
1413 left.setAlignment(Qt.AlignmentFlag.AlignTop)
1414
1415 snap_gb = QGroupBox(" 数据快照管理")
1416 sl = QVBoxLayout()
1417 self.snap_list = QListWidget()
1418 self.snap_list.itemClicked.connect(self._load_snapshot)
1419 sl.addWidget(self.snap_list)
1420 snap_btns = QHBoxLayout()
1421 snap_btns.addWidget(QPushButton(" 创 建 快 照 ",
1422 clicked=self._create_snapshot))
1423 snap_btns.addWidget(QPushButton(" ◀ 回 溯 ",
1424 clicked=self._restore_snapshot))
1425 sl.addLayout(snap_btns)
1426 snap_gb.setLayout(sl)
1427 left.addWidget(snap_gb)
1428
1429 txn_gb = QGroupBox(" 事务管理")
1430 tl = QVBoxLayout()
1431 self.txn_status = QLabel("事务状态: 无活动事务")
1432 self.txn_status.setStyleSheet("color:#795548;")
1433 tl.addWidget(self.txn_status)
1434 txn_btns = QHBoxLayout()
1435 txn_btns.addWidget(QPushButton("▶ 开始事务", clicked=self._begin_txn))
1436 txn_btns.addWidget(QPushButton("✓ 提交", clicked=self._commit_txn))
1437 txn_btns.addWidget(QPushButton("◀ 回滚", clicked=self._rollback_txn))
1438 tl.addLayout(txn_btns)
1439 txn_gb.setLayout(tl)
1440 left.addWidget(txn_gb)
1441
1442 filter_gb = QGroupBox(" 数据筛选")
1443 fl = QVBoxLayout()
1444 fl.addWidget(QLabel("筛选条件 (字段:值):"))
1445 self.filter_input = QLineEdit(""); fl.addWidget(self.filter_input)
1446 fl.addWidget(QPushButton(" 执行筛选", clicked=self._filter_data))
1447 fl.addWidget(QPushButton(" 清除筛选", clicked=self._clear_filter))
1448 filter_gb.setLayout(fl)
1449 left.addWidget(filter_gb)
1450

```

```

1451         left_w = QWidget(); left_w.setLayout(left); left_w.setMaximumWidth(250)
1452
1453         right = QVBoxLayout()
1454         self.data_table = QTableWidgetItem()
1455         self.data_table.setAlternatingRowColors(True)
1456         self.data_table.itemChanged.connect(self._on_cell_edit)
1457         right.addWidget(QLabel("    数据表格（支持直接编辑单元格）",
1458 styleSheet="color:#E65100;font-weight:bold;"))
1459         right.addWidget(self.data_table)
1460
1461         self.stats_text = QTextEdit(); self.stats_text.setReadOnly(True)
1462         self.stats_text.setMaximumHeight(100)
1463
1464         self.stats_text.setStyleSheet("background:#FFF8E1;color:#4E342E;font-family:Con
1465 solas;font-size:11px;")
1466         right.addWidget(QLabel("    数 据 统 计 与 校 验 :",
1467 styleSheet="color:#E65100;"))
1468         right.addWidget(self.stats_text)
1469
1470         right_w = QWidget(); right_w.setLayout(right)
1471         main.addWidget(left_w); main.addWidget(right_w)
1472         main.setSizes([250, 950])
1473         layout.addWidget(main)
1474
1475     def _init_demo_data(self):
1476         self._current_data = [
1477             {"id": 1, "feature_a": 5.1, "feature_b": 3.5, "feature_c": 1.4, "label":
1478 "A"},
1479             {"id": 2, "feature_a": 4.9, "feature_b": 3.0, "feature_c": 1.4, "label":
1480 "A"},
1481             {"id": 3, "feature_a": 7.0, "feature_b": 3.2, "feature_c": 4.7, "label":
1482 "B"},
1483             {"id": 4, "feature_a": 6.4, "feature_b": 3.2, "feature_c": 4.5, "label":
1484 "B"},
1485             {"id": 5, "feature_a": 5.5, "feature_b": 2.3, "feature_c": 4.0, "label":
1486 "B"},
1487             {"id": 6, "feature_a": 6.3, "feature_b": 3.3, "feature_c": 6.0, "label":
1488 "C"},
1489             {"id": 7, "feature_a": 5.8, "feature_b": 2.7, "feature_c": 5.1, "label":
1490 "C"},
1491             {"id": 8, "feature_a": 7.1, "feature_b": 3.0, "feature_c": 5.9, "label":
1492 "C"},
1493         ]
1494         self._refresh_table()
1495         self.stats_text.setPlainText(f"演示数据集 | {len(self._current_data)}
1496 条 | 校验: {HashValidator.checksum(self._current_data)}")
1497
1498     def _refresh_table(self, data: list = None):
1499         if data is not None: self._current_data = data
1500         d = self._current_data

```

```

1501         if not d: return
1502         headers = list(d[0].keys())
1503         self.data_table.setColumnCount(len(headers))
1504         self.data_table.setHorizontalHeaderLabels(headers)
1505         self.data_table.setRowCount(len(d))
1506         for r, row in enumerate(d):
1507             for c, key in enumerate(headers):
1508                 self.data_table.setItem(r, c, QTableWidgetItem(str(row.get(key,
1509 ""))))
1510
1511 self.data_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.
1512 Stretch)
1513
1514     def _on_cell_edit(self, item):
1515         if not self._txn:
1516             self._begin_txn()
1517         row, col = item.row(), item.column()
1518         key = self.data_table.horizontalHeaderItem(col).text()
1519         try:
1520             val = eval(item.text())
1521         except:
1522             val = item.text()
1523         if row < len(self._current_data):
1524             self._current_data[row][key] = val
1525             if self._txn:
1526                 data_engine.write(self._txn, "main", self._current_data)
1527
1528     def _begin_txn(self):
1529         self._txn = data_engine.begin_transaction(self._flow_id)
1530         self.txn_status.setText(f"事务状态: 活跃 [{self._txn.txn_id[:8]}]")
1531         self.txn_status.setStyleSheet("color:#F57F17;")
1532
1533     def _commit_txn(self):
1534         if self._txn and data_engine.commit(self._txn):
1535             self.txn_status.setText("事务状态: 已提交 ✓");
1536 self.txn_status.setStyleSheet("color:#2E7D32;")
1537             self._txn = None
1538             self.stats_text.setPlainText(f"已提交 | 校验:
1539 {HashValidator.checksum(self._current_data)}")
1540
1541     def _rollback_txn(self):
1542         if self._txn:
1543             old_data = data_engine.get_snapshot(self._flow_id, "main")
1544             if old_data:
1545                 self._current_data = old_data; self._refresh_table()
1546                 data_engine.rollback(self._txn)
1547                 self.txn_status.setText("事务状态: 已回滚 ↩");
1548 self.txn_status.setStyleSheet("color:#D84315;")
1549                 self._txn = None
1550

```

```

1551     def _create_snapshot(self):
1552         data_engine.register_node(self._flow_id, f"snap_{len(self.snap_list)}",
1553 self._current_data)
1554         ver = data_engine._data_store.get(f"{self._flow_id}:main",
1555 {}).get("version", 0)
1556         self.snap_list.addItem(f"快照 v{ver} - {len(self._current_data)}条")
1557         QMessageBox.information(self, "快照", f"快照已创建 v{ver}")
1558
1559     def _load_snapshot(self, item):
1560         data = data_engine.get_snapshot(self._flow_id, "main")
1561         if data:
1562             self._current_data = data; self._refresh_table()
1563
1564     def _restore_snapshot(self):
1565         data = data_engine.get_snapshot(self._flow_id, "main")
1566         if data:
1567             self._current_data = data; self._refresh_table()
1568             QMessageBox.information(self, "回溯", "已回溯到最近快照")
1569         else:
1570             QMessageBox.warning(self, "提示", "没有可用快照")
1571
1572     def _import_csv(self):
1573         path, _ = QFileDialog.getOpenFileName(self, "导入 CSV", "", "CSV (*.csv);;
1574 所有 (*.*)")
1575         if path:
1576             self._current_data = DataAdapter.from_csv(path);
1577 self._refresh_table()
1578
1579     def _import_json(self):
1580         path, _ = QFileDialog.getOpenFileName(self, "导入 JSON", "", "JSON
1581 (*.json);;所有 (*.*)")
1582         if path:
1583             data = DataAdapter.from_json(path)
1584             if isinstance(data, list): self._current_data = data;
1585 self._refresh_table()
1586
1587     def _import_txt(self):
1588         path, _ = QFileDialog.getOpenFileName(self, "导入 TXT", "", "TXT (*.txt);;
1589 所有 (*.*)")
1590         if path:
1591             with open(path, 'r', encoding='utf-8') as f: content = f.read()
1592             self._current_data = [{"line": i+1, "content": l} for i, l in
1593 enumerate(content.split('\n')[:50]) if l.strip()]
1594             self._refresh_table()
1595
1596     def _import_image(self):
1597         QMessageBox.information(self, "导入图片", "图片数据请使用视觉实验室模块
1598 导入处理")
1599
1600     def _export_data(self):

```



```

1601         path, _ = QFileDialog.getSaveFileName(self, "导出数据", "data.csv", "CSV
1602         (*.csv);;JSON (*.json)")
1603         if path:
1604             if path.endswith('.json'):
1605                 with open(path, 'w', encoding='utf-8') as f:
1606                     json.dump(self._current_data, f, ensure_ascii=False,
1607                     indent=2)
1608             else:
1609                 with open(path, 'w', encoding='utf-8-sig', newline='') as f:
1610                     w = csv.DictWriter(f,
1611                     fieldnames=self._current_data[0].keys())
1612                     w.writeheader(); w.writerows(self._current_data)
1613                     QMessageBox.information(self, "导出", f"数据已导出至 {path}")
1614
1615         def _analyze_data(self):
1616             if self._current_data:
1617                 analysis = FeatureAnalyzer.analyze(self._current_data)
1618                 self.stats_text.setPlainText(json.dumps(analysis,
1619                 ensure_ascii=False, indent=2))
1620
1621         def _filter_data(self):
1622             cond = self.filter_input.text().strip()
1623             if not cond: return
1624             try:
1625                 field, value = cond.split(':', 1)
1626                 filtered = [row for row in self._current_data if
1627                 str(row.get(field.strip(), '')) == value.strip()]
1628                 self._refresh_table(filtered)
1629             except: pass
1630
1631         def _clear_filter(self):
1632             self._refresh_table()
1633
1634         def get_module_widget(user):
1635             return DataManagerWidget(user)
1636
1637         # ==== modules\lowcode_editor.py (113 行) ====
1638         """低代码编辑器模块 - 脚本编辑、语法校验、模板管理、自定义片段"""
1639         from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
1640         QLabel,
1641         QListWidget, QGroupBox, QSplitter, QTextEdit, QInputDialog, QMessageBox)
1642         from PyQt6.QtCore import Qt, pyqtSignal
1643         from core.auth import Role, OpAction, auth_engine
1644         from core.rule_engine import rule_engine, TriggerType, Condition, CompareOp,
1645         RuleAction
1646         from ui.code_editor import LowCodeEditor, CODE_TEMPLATES
1647
1648         class LowCodeEditorModule(QWidget):
1649             def __init__(self, user):
1650                 super().__init__()

```

```

1651         self.user = user
1652         self._snippets = {} # 用户自定义片段
1653         self._setup_ui()
1654         self._init_demo_rules()
1655
1656     def _setup_ui(self):
1657         layout = QVBoxLayout(self)
1658
1659         header = QHBoxLayout()
1660         title = QLabel("    低代码脚本编辑器")
1661         title.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
1662         header.addWidget(title)
1663         header.addStretch()
1664         btn_new = QPushButton("    新    建    片    段    ");
1665         btn_new.clicked.connect(self._new_snippet)
1666         btn_export = QPushButton("    导    出    脚    本    ");
1667         btn_export.clicked.connect(self._export_script)
1668         btn_import = QPushButton("    导    入    脚    本    ");
1669         btn_import.clicked.connect(self._import_script)
1670         header.addWidget(btn_new);           header.addWidget(btn_export);
1671         header.addWidget(btn_import)
1672         layout.addLayout(header)
1673
1674         self.editor = LowCodeEditor()
1675         layout.addWidget(self.editor)
1676
1677         bottom = QHBoxLayout()
1678
1679         snippets_gb = QGroupBox("    自定义片段库")
1680         sl = QVBoxLayout()
1681         self.snippet_list = QListWidget()
1682         self.snippet_list.addItem(["自定义:数据增强", "自定义:特征工程", "自定义:模型评估", "自定义:数据清洗"])
1683         self.snippet_list.itemDoubleClicked.connect(self._load_snippet)
1684         sl.addWidget(self.snippet_list)
1685         snipt_btns = QHBoxLayout()
1686         snipt_btns.addWidget(QPushButton("    保 存 当 前 为 片 段 ",
1687         clicked=self._save_current_as_snippet))
1688         snipt_btns.addWidget(QPushButton("    删    除    片    段    ",
1689         clicked=self._delete_snippet))
1690         sl.addLayout(snipt_btns)
1691         snippets_gb.setLayout(sl)
1692
1693         rule_gb = QGroupBox("    关联业务规则")
1694         rl = QVBoxLayout()
1695         self.rule_text = QTextEdit()
1696         self.rule_text.setReadOnly(True); self.rule_text.setMaximumHeight(120)
1697         self.rule_text.setPlainText("规则 1: 脚本执行前必须通过语法检查 [优先级
1698 8]\n 规则 2: 模型训练脚本须含异常捕获 [优先级 7]\n 规则 3: 数据处理脚本输出须包含
1699 数据量统计 [优先级 6]")
1700

```

```

1701         rl.addWidget(self.rule_text)
1702         rule_gb.setLayout(rl)
1703
1704         bottom.addWidget(snippets_gb, 1)
1705         bottom.addWidget(rule_gb, 1)
1706         layout.addLayout(bottom)
1707
1708         demo_bar = QHBoxLayout()
1709         demo_label = QLabel("    演示案例：点击左侧模板或自定义片段快速开始 | 支
1710 持 Python 语法实时高亮与校验")
1711         demo_label.setStyleSheet("color:#795548;font-size:11px;")
1712         demo_bar.addWidget(demo_label)
1713         layout.addLayout(demo_bar)
1714
1715     def _init_demo_rules(self):
1716         for name, priority, cond_field, cond_val in [
1717             ("脚本语法检查规则", 9, "syntax_valid", True),
1718             ("异常处理检查规则", 7, "has_try_except", True),
1719             ("输出统计规则", 6, "has_output_stats", True),
1720         ]:
1721             rule_engine.create_rule(name, TriggerType.NODE,
1722                                     [Condition(field=cond_field, op=CompareOp.EQ, value=cond_val)],
1723                                     [RuleAction(action_type="log", params={"text": f"{name}通过"})],
1724                                     priority=priority, group="脚本编辑器")
1725
1726     def _new_snippet(self):
1727         name, ok = QInputDialog.getText(self, "新建片段", "片段名称:")
1728         if ok and name:
1729             self.snippet_list.addItem(f"自定义: {name}")
1730
1731     def _save_current_as_snippet(self):
1732         code = self.editor.get_code()
1733         if not code.strip():
1734             QMessageBox.warning(self, "提示", "请先编写代码"); return
1735         name, ok = QInputDialog.getText(self, "保存片段", "片段名称:")
1736         if ok and name:
1737             self._snippets[name] = code
1738             self.snippet_list.addItem(f"自定义: {name}")
1739             QMessageBox.information(self, "成功", f"片段 ' {name}' 已保存")
1740
1741     def _load_snippet(self, item):
1742         name = item.text().replace("自定义:", "")
1743         if name in self._snippets:
1744             self.editor.set_code(self._snippets[name])
1745
1746     def _delete_snippet(self):
1747         current = self.snippet_list.currentItem()
1748         if current:
1749             name = current.text().replace("自定义:", "")
1750             self._snippets.pop(name, None)

```

```

1751         self.snippet_list.takeItem(self.snippet_list.currentRow())
1752
1753     def _export_script(self):
1754         code = self.editor.get_code()
1755         if code.strip():
1756             QMessageBox.information(self, "导出", f"脚本已准备导出\n 代码长度:
1757 {len(code)} 字符\n 请使用数据管理模块保存到文件")
1758
1759     def _import_script(self):
1760         QMessageBox.information(self, "导入", "请使用数据管理模块导入脚本文件")
1761
1762     def get_module_widget(user):
1763         return LowCodeEditorModule(user)
1764
1765 # ==== modules\model_designer.py (217 行) ====
1766 """模型设计器 - 可视化画布编排 AI 流程、节点管理、流程版本管理"""
1767 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
1768 QLabel,
1769 QListWidget, QGroupBox, QSplitter, QComboBox, QTextEdit, QMessageBox,
1770 QInputDialog)
1771 from PyQt6.QtCore import Qt
1772 from PyQt6.QtGui import QFont
1773 from core.auth import Role, OpAction, auth_engine
1774 from core.state_machine import FlowState, flow_sm
1775 from core.rule_engine import rule_engine, TriggerType, Condition, CompareOp,
1776 RuleAction, LogicOp
1777 from core.data_consistency import data_engine
1778 from ui.canvas import ModelCanvas
1779 import json, time
1780
1781 NODE_TYPES = ["数据预处理", "传统机器学习", "深度学习", "推理", "分支判断", "循
1782 环", "数据输出"]
1783 FLOW_STATES = [s.value for s in FlowState]
1784
1785 class ModelDesignerWidget(QWidget):
1786     def __init__(self, user):
1787         super().__init__()
1788         self.user = user; self.current_flow_id = None
1789         self._setup_ui()
1790         self._init_demo()
1791         self._check_perm()
1792
1793     def _setup_ui(self):
1794         layout = QHBoxLayout(self)
1795
1796         left_panel = QVBoxLayout()
1797         left_panel.setAlignment(Qt.AlignmentFlag.AlignTop)
1798
1799         flow_gb = QGroupBox(" 流程管理")
1800         fl = QVBoxLayout()

```

```

1801         self.flow_combo = QComboBox()
1802         self.flow_combo.addItem("图像分类流程", "目标检测流程", "文本分析流程
1803 ", "3D 点云流程", "时序预测流程"])
1804         self.flow_combo.currentTextChanged.connect(self._on_flow_change)
1805         fl.addWidget(QLabel("当前流程:")); fl.addWidget(self.flow_combo)
1806
1807         state_row = QHBoxLayout()
1808         self.state_label = QLabel("编辑态")
1809
1810         self.state_label.setStyleSheet("color:#E65100;font-weight:bold;font-size:14px;b
1811 ackground:#FFFFFF;border:1px solid #FFD54F;border-radius:6px;padding:6px 12px;")
1812         state_row.addWidget(QLabel("          状          态          :"));
1813         state_row.addWidget(self.state_label); state_row.addStretch()
1814         fl.addLayout(state_row)
1815
1816         st_btns = QHBoxLayout()
1817         for state, btn_style in [("调试态", "warning"), ("运行态", "success"), ("
1818 暂停态", ""), ("编辑态", "danger")]:
1819             btn = QPushButton(state)
1820             if btn_style: btn.setObjectName(btn_style)
1821             btn.clicked.connect(lambda checked, s=state: self._change_state(s))
1822             st_btns.addWidget(btn)
1823         fl.addLayout(st_btns)
1824         fl.addWidget(QLabel("版本管理:"))
1825         self.version_combo = QComboBox(); self.version_combo.addItem("v1.0 (当
1826 前)", "v0.9", "v0.8"])
1827         fl.addWidget(self.version_combo)
1828         ver_row = QHBoxLayout()
1829         ver_row.addWidget(QPushButton("          保          存          版          本          ",
1830 clicked=self._save_version))
1831         ver_row.addWidget(QPushButton("          ↶          版          本          回          退          ",
1832 clicked=self._rollback_version))
1833         fl.addLayout(ver_row)
1834         flow_gb.setLayout(fl)
1835         left_panel.addWidget(flow_gb)
1836
1837         node_gb = QGroupBox("    节点类型")
1838         nl = QVBoxLayout()
1839         self.node_list = QListWidget()
1840         self.node_list.addItem(NODE_TYPES)
1841         self.node_list.itemClicked.connect(self._add_node_from_list)
1842         nl.addWidget(self.node_list)
1843         node_gb.setLayout(nl)
1844         left_panel.addWidget(node_gb)
1845
1846         action_gb = QGroupBox("✂ 操作")
1847         al = QVBoxLayout()
1848         actions = [("    连线选中节点", self._connect_nodes), ("    自动布局",
1849 self._auto_layout),
1850 ("    水平对齐", lambda: self._align("hcenter")), ("    垂直对

```

```

1851 齐", lambda: self._align("vcenter")),
1852          ("  网格排列", lambda: self._align("grid")), ("↶ 撤销",
1853 self._undo),
1854          ("↷ 重做", self._redo), ("  清空画布", self._clear)]
1855      for label, handler in actions:
1856          btn      =      QPushButton(label);      btn.clicked.connect(handler);
1857  al.addWidget(btn)
1858      action_gb.setLayout(al)
1859      left_panel.addWidget(action_gb)
1860
1861      left_widget      =      QWidget();      left_widget.setLayout(left_panel);
1862  left_widget.setMaximumWidth(260)
1863
1864      right_panel = QVBoxLayout()
1865      self.canvas = ModelCanvas()
1866      self.canvas.flow_changed.connect(self._on_flow_modified)
1867      self.canvas.status_message.connect(self._on_status_msg)
1868      right_panel.addWidget(self.canvas)
1869
1870      info_bar = QHBoxLayout()
1871      self.info_label = QLabel("节点:0 | 连线:0 | 就绪")
1872
1873  self.info_label.setStyleSheet("color:#795548;padding:4px;font-size:12px;")
1874      info_bar.addWidget(self.info_label); info_bar.addStretch()
1875      right_panel.addLayout(info_bar)
1876
1877      right_widget = QWidget(); right_widget.setLayout(right_panel)
1878
1879      splitter = QSplitter(Qt.Orientation.Horizontal)
1880      splitter.addWidget(left_widget); splitter.addWidget(right_widget)
1881      splitter.setSizes([260, 1100])
1882      layout.addWidget(splitter)
1883
1884      def _init_demo(self):
1885          self._flow_cache = {}
1886          flow_configs = {
1887              "图像分类流程": [
1888                  ("数据预处理", "数据加载", 50, 50), ("数据预处理", "图像缩放", 50,
1889 180),
1890                  ("数据预处理", "灰度化处理", 50, 310), ("传统机器学习", "特征提
1891 取", 280, 120),
1892                  ("深度学习", "CNN 推理", 280, 250), ("数据输出", "结果输出", 520,
1893 180),
1894                  ],
1895              "目标检测流程": [
1896                  ("数据预处理", "视频帧提取", 50, 80), ("深度学习", "YOLO 检测",
1897 280, 120),
1898                  ("推理", "NMS 后处理", 280, 250), ("数据输出", "标注输出", 520,
1899 160),
1900                  ],

```

```

1901         "文本分析流程": [
1902             ("数据预处理", "文本分词", 50, 100), ("数据预处理", "词向量化", 50,
1903 220),
1904             ("深度学习", "BERT 编码", 300, 160), ("推理", "情感分类", 520,
1905 160),
1906         ],
1907         "3D 点云流程": [
1908             ("数据预处理", "点云采样", 50, 80), ("数据预处理", "体素化", 50,
1909 200),
1910             ("深度学习", "PointNet", 300, 140), ("数据输出", "分割结果", 520,
1911 160),
1912         ],
1913         "时序预测流程": [
1914             ("数据预处理", "序列窗口化", 50, 100), ("深度学习", "LSTM 层", 280,
1915 100),
1916             ("深度学习", "注意力层", 280, 220), ("数据输出", "预测输出", 520,
1917 160),
1918         ],
1919     }
1920     self._flow_configs = flow_configs
1921     self.current_flow_id = "flow_图像分类流程"
1922     flow_sm.create_flow(self.current_flow_id, {"name": "图像分类流程"})
1923     for ntype, label, x, y in flow_configs.get("图像分类流程", []):
1924         self.canvas.add_node(ntype, label, x, y)
1925     self._update_info()
1926
1927     def _add_node_from_list(self, item):
1928         node_type = item.text()
1929         label, ok = QDialog.getText(self, "新建节点", "节点标签:",
1930 text=f"{node_type}节点")
1931         if ok and label:
1932             self.canvas.add_node(node_type, label)
1933             self._update_info()
1934
1935     def _connect_nodes(self, _checked=None): self.canvas.connect_nodes();
1936 self._update_info()
1937     def _auto_layout(self, _checked=None): self.canvas.auto_layout();
1938 self._update_info()
1939     def _align(self, mode): self.canvas.align_nodes(mode)
1940     def _undo(self, _checked=None): self.canvas.undo(); self._update_info()
1941     def _redo(self, _checked=None): self.canvas.redo(); self._update_info()
1942
1943     def _clear(self):
1944         reply = QMessageBox.question(self, "确认", "确定清空画布吗? ")
1945         if reply == QMessageBox.StandardButton.Yes:
1946             self.canvas.clear_all(); self._update_info()
1947
1948     def _change_state(self, target_str: str):
1949         if not self.current_flow_id: return
1950         target_map = {"编辑态": FlowState.EDITING, "调试态": FlowState.DEBUGGING,

```

```

1951         "运行态": FlowState.RUNNING, "暂停态": FlowState.PAUSED}
1952     target = target_map.get(target_str)
1953     if target and flow_sm.transition(self.current_flow_id, target,
1954 self.user.username):
1955         self.state_label.setText(target_str)
1956         if target == FlowState.RUNNING:
1957             self._evaluate_rules()
1958             QMessageBox.information(self, "状态变更", f"流程已切换至
1959 [{target_str}]")
1960         else:
1961             QMessageBox.warning(self, "操作失败", f"无法从当前状态切换至
1962 [{target_str}], 请检查状态流转规则")
1963
1964     def _evaluate_rules(self):
1965         flow_data = self.canvas.get_flow_data()
1966         context = {"flow_id": self.current_flow_id, "node_count":
1967 len(flow_data["nodes"]),
1968 "edge_count": len(flow_data["edges"]), "user":
1969 self.user.username}
1970         fired = rule_engine.evaluate_all(context, TriggerType.NODE)
1971         if fired:
1972             msgs = "\n".join([f"• {r.name} (优先级:{r.priority})" for r in fired])
1973             QMessageBox.information(self, "规则触发", f"以下业务规则已触
1974 发:\n{msgs}")
1975
1976     def _save_version(self):
1977         ver_name = f"v{int(time.time()) % 1000}"
1978         self.version_combo.addItem(ver_name)
1979         self.version_combo.setCurrentText(ver_name)
1980         flow_data = self.canvas.get_flow_data()
1981         data_engine.register_node(self.current_flow_id, "snapshot", flow_data)
1982         QMessageBox.information(self, "保存", f"版本 [{ver_name}] 已保存")
1983
1984     def _rollback_version(self):
1985         flow_sm.rollback(self.current_flow_id)
1986         QMessageBox.information(self, "回退", "已回退至上个版本状态")
1987
1988     def _on_flow_change(self, name: str):
1989         if self.current_flow_id:
1990             self._flow_cache[self.current_flow_id] = self.canvas.get_flow_data()
1991             self.current_flow_id = f"flow_{name.replace(' ', '_')}"
1992             flow_sm.create_flow(self.current_flow_id, {"name": name})
1993             if self.current_flow_id in self._flow_cache:
1994                 self.canvas.load_flow_data(self._flow_cache[self.current_flow_id])
1995             elif name in self._flow_configs:
1996                 self.canvas.clear_all()
1997                 for ntype, label, x, y in self._flow_configs[name]:
1998                     self.canvas.add_node(ntype, label, x, y)
1999             else:
2000                 self.canvas.clear_all()

```



```

2001         self._update_info()
2002
2003     def _on_flow_modified(self):
2004         self._update_info()
2005
2006     def _on_status_msg(self, msg: str):
2007         self.info_label.setText(msg)
2008         from PyQt6.QtCore import QTimer
2009         QTimer.singleShot(4000, self._update_info)
2010
2011     def _update_info(self):
2012         flow_data = self.canvas.get_flow_data()
2013         self.info_label.setText(f"节点 : {len(flow_data['nodes'])} | 连
2014 线: {len(flow_data['edges'])} | 流程: {self.current_flow_id}")
2015
2016     def _check_perm(self):
2017         if not auth_engine.check_permission(self.user, "model_designer",
2018 OpAction.EDIT):
2019             QMessageBox.warning(self, "权限提示", "当前角色仅可查看, 编辑功能受
2020 限")
2021
2022     def get_module_widget(user):
2023         return ModelDesignerWidget(user)
2024
2025     # ==== modules\model_training.py (186 行) ====
2026     """模型训练模块 - 超参配置、训练启停、断点续训、模型导入导出"""
2027     from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
2028 QLabel,
2029     QGroupBox, QGridLayout, QSpinBox, QDoubleSpinBox, QComboBox, QTextEdit,
2030 QProgressBar,
2031     QSlider, QMessageBox, QTableWidgetItem, QTableWidgetItem, QHeaderView)
2032     from PyQt6.QtCore import Qt, QTimer
2033     from core.auth import Role, OpAction, auth_engine
2034     from core.state_machine import FlowState, flow_sm, task_sm, TaskState
2035     from core.rule_engine import rule_engine, TriggerType, Condition, CompareOp,
2036 RuleAction
2037     from core.data_consistency import data_engine
2038     from core.algorithms import ParameterOptimizer
2039     import random, time
2040
2041     class ModelTrainingWidget(QWidget):
2042         def __init__(self, user):
2043             super().__init__()
2044             self.user = user; self._training = False
2045             self._progress = 0; self._epoch = 0; self._total_epochs = 50
2046             self._timer = QTimer(self);
2047         self._timer.timeout.connect(self._training_step)
2048             self._setup_ui(); self._init_demo()
2049
2050         def _setup_ui(self):

```

```

2051         layout = QHBoxLayout(self)
2052
2053         left_panel = QVBoxLayout()
2054
2055         tpl_gb = QGroupBox(" 模型模板")
2056         tl = QVBoxLayout()
2057         self.tpl_combo = QComboBox()
2058         self.tpl_combo.addItem("分类模型-RandomForest", "分类模型-SVM", "回归
2059 模型-XGBoost",
2060         "目标检测-YOLO", "图像分割-U-Net", "时序预测-LSTM", "文本分析
2061 -BERT"])
2062         tl.addWidget(QLabel("选择模板:")); tl.addWidget(self.tpl_combo)
2063         tpl_gb.setLayout(tl)
2064         left_panel.addWidget(tpl_gb)
2065
2066         param_gb = QGroupBox("⚙ 超参数配置")
2067         pg = QGridLayout()
2068         params = [("学习率", 0.001, 0.00001, 0.1, 0.0001), ("批次大小", 32, 1, 256,
2069 1),
2070         ("训练轮次", 50, 1, 500, 1), ("早停轮次", 10, 1, 100, 1),
2071         ("Dropout 率", 0.2, 0.0, 0.9, 0.05), ("L2 正则化", 0.001, 0.0, 0.1,
2072 0.001)]
2073         self._param_spinners = {}
2074         for r, (name, default, min_v, max_v, step) in enumerate(params):
2075             pg.addWidget(QLabel(name), r, 0)
2076             if isinstance(default, int):
2077                 sp = QSpinBox(); sp.setRange(int(min_v), int(max_v));
2078                 sp.setValue(default)
2079             else:
2080                 sp = QDoubleSpinBox(); sp.setRange(min_v, max_v);
2081                 sp.setSingleStep(step); sp.setValue(default)
2082             self._param_spinners[name] = sp; pg.addWidget(sp, r, 1)
2083         param_gb.setLayout(pg)
2084         left_panel.addWidget(param_gb)
2085
2086         ctrl_gb = QGroupBox(" 训练控制")
2087         cl = QVBoxLayout()
2088         self.btn_start = QPushButton(" ▶ 开 始 训 练 ");
2089         self.btn_start.setObjectName("success")
2090         self.btn_start.clicked.connect(self._start_training)
2091         self.btn_pause = QPushButton(" 暂停 训 练 ");
2092         self.btn_pause.setObjectName("warning")
2093         self.btn_pause.clicked.connect(self._pause_training)
2094         self.btn_stop = QPushButton(" 停止 训 练 ");
2095         self.btn_stop.setObjectName("danger")
2096         self.btn_stop.clicked.connect(self._stop_training)
2097         self.btn_resume = QPushButton(" ⏪ 断 点 续 训 ");
2098         self.btn_resume.clicked.connect(self._resume_training)
2099         cl.addWidget(self.btn_start); cl.addWidget(self.btn_pause)
2100         cl.addWidget(self.btn_stop); cl.addWidget(self.btn_resume)

```

```

2101         ctrl_gb.setLayout(c1)
2102         left_panel.addWidget(ctrl_gb)
2103
2104         export_gb = QGroupBox("    模型导出/导入")
2105         el = QVBoxLayout()
2106         el.addWidget(QPushButton("        导    出    模    型    (ONNX/PKL)",
2107 clicked=self._export_model))
2108         el.addWidget(QPushButton("        导    入    外    部    模    型    ",
2109 clicked=self._import_model))
2110         el.addWidget(QPushButton("    导出训练日志", clicked=self._export_log))
2111         export_gb.setLayout(el)
2112         left_panel.addWidget(export_gb)
2113
2114         left_widget      =      QWidget();      left_widget.setLayout(left_panel);
2115 left_widget.setMaximumWidth(280)
2116
2117         right_panel = QVBoxLayout()
2118
2119         progress_gb = QGroupBox("    训练进度")
2120         pl = QVBoxLayout()
2121         self.progress_bar = QProgressBar(); self.progress_bar.setRange(0, 100)
2122         self.progress_bar.setTextVisible(True)
2123         pl.addWidget(self.progress_bar)
2124         self.status_label = QLabel("状态: 就绪")
2125
2126 self.status_label.setStyleSheet("color:#2E7D32;font-weight:bold;font-size:14px;
2127 ")
2128         pl.addWidget(self.status_label)
2129         info_row = QHBoxLayout()
2130         self.epoch_label = QLabel("Epoch: 0/50"); self.loss_label = QLabel("Loss:
2131 --")
2132         self.acc_label = QLabel("Accuracy: --"); self.time_label = QLabel("耗时:
2133 --")
2134         for lbl in [self.epoch_label, self.loss_label, self.acc_label,
2135 self.time_label]:
2136
2137 lbl.setStyleSheet("color:#795548;background:#FFFFFF;border-radius:4px;padding:6
2138 px;")
2139         info_row.addWidget(lbl)
2140         pl.addLayout(info_row)
2141         progress_gb.setLayout(pl)
2142         right_panel.addWidget(progress_gb)
2143
2144         log_gb = QGroupBox("    训练日志")
2145         ll = QVBoxLayout()
2146         self.log_text = QTextEdit(); self.log_text.setReadOnly(True)
2147         self.log_text.setMaximumHeight(150)
2148
2149 self.log_text.setStyleSheet("background:#FFF8E1;color:#2E7D32;font-family:Conso
2150 las;font-size:11px;")

```

```

2151         ll.addWidget(self.log_text)
2152         log_gb.setLayout(ll)
2153         right_panel.addWidget(log_gb)
2154
2155         metrics_gb = QGroupBox("    训练指标监控（支持滚轮浏览）")
2156         ml = QVBoxLayout()
2157         self.metrics_table = QTableWidgetItem(0, 5)
2158         self.metrics_table.setHorizontalHeaderLabels(["Epoch", "Train Loss",
2159 "Val Loss", "Accuracy", "F1 Score"])
2160
2161 self.metrics_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMo
2162 de.Stretch)
2163         ml.addWidget(self.metrics_table)
2164         metrics_gb.setLayout(ml)
2165         right_panel.addWidget(metrics_gb)
2166
2167         right_widget = QWidget(); right_widget.setLayout(right_panel)
2168
2169         layout.addWidget(left_widget); layout.addWidget(right_widget, 1)
2170
2171     def _init_demo(self):
2172         for epoch in range(1, 6):
2173             self.metrics_table.insertRow(self.metrics_table.rowCount())
2174             for col, val in enumerate([epoch,
2175 round(1.5/epoch+random.uniform(0, 0.1), 4),
2176 round(1.8/epoch+random.uniform(0, 0.1), 4),
2177 round(0.5+0.08*epoch, 4), round(0.45+0.07*epoch, 4)]):
2178                 self.metrics_table.setItem(self.metrics_table.rowCount()-1, col,
2179 QTableWidgetItem(str(val)))
2180
2181     def _start_training(self):
2182         if not auth_engine.check_permission(self.user, "model_training",
2183 OpAction.EXECUTE):
2184             QMessageBox.warning(self, "权限不足", "当前角色无权执行训练");
2185         return
2186         self._training = True; self._progress = 0; self._epoch = 0
2187         self._total_epochs = self._param_spinners["训练轮次"].value()
2188         self._timer.start(150)
2189         self.status_label.setText("    状    态    :    训    练    中    ...");
2190 self.status_label.setStyleSheet("color:#F57F17;font-weight:bold;font-size:14px;
2191 ")
2192         self._log("=" * 40); self._log(f"    开 始 训 练    -    模 板    :
2193 {self.tmpl_combo.currentText()}")
2194         self._log(f"    学 习 率: {self._param_spinners['学习率'].value()}, 批次:
2195 {self._param_spinners['批次大小'].value()}")
2196         task_sm.create_task("train_001", "模型训练", priority=8)
2197         task_sm.transition("train_001", TaskState.EXECUTING)
2198
2199     def _pause_training(self):
2200         if self._training:

```

```

2201         self._timer.stop(); self._training = False
2202         self.status_label.setText("    状    态    :    已    暂    停    ");
2203 self.status_label.setStyleSheet("color:#F57F17;font-weight:bold;")
2204         self._log("    训    练    已    暂    停    "); task_sm.transition("train_001",
2205 TaskState.PAUSED)
2206
2207     def _stop_training(self):
2208         self._timer.stop(); self._training = False
2209         self.status_label.setText("    状    态    :    已    停    止    ");
2210 self.status_label.setStyleSheet("color:#D84315;font-weight:bold;")
2211         self._log("    训    练    已    停    止    "); task_sm.transition("train_001",
2212 TaskState.CANCELLED)
2213
2214     def _resume_training(self):
2215         if not self._training:
2216             self._training = True; self._timer.start(150)
2217             self.status_label.setText("    状    态    :    续    训    中    ...");
2218 self.status_label.setStyleSheet("color:#F57F17;font-weight:bold;")
2219             self._log("↩ 断点续训 - 恢复检查点")
2220
2221     def _training_step(self):
2222         self._progress += 1
2223         if self._progress % 5 == 0:
2224             self._epoch = min(self._total_epochs, self._progress // 5)
2225             self.progress_bar.setValue(int(self._epoch / self._total_epochs *
2226 100))
2227             loss = round(2.0 / (self._epoch + 1) + random.uniform(-0.05, 0.05), 4)
2228             acc = round(min(0.98, 0.5 + 0.01 * self._epoch + random.uniform(-0.02,
2229 0.02)), 4)
2230             f1 = round(acc - random.uniform(0.01, 0.05), 4)
2231             self.epoch_label.setText(f"Epoch:
2232 {self._epoch}/{self._total_epochs}")
2233             self.loss_label.setText(f"Loss: {loss}");
2234 self.acc_label.setText(f"Accuracy: {acc}")
2235             row = self.metrics_table.rowCount()
2236             self.metrics_table.insertRow(row)
2237             for col, val in enumerate([self._epoch, loss, round(loss+0.1, 4), acc,
2238 f1]):
2239                 self.metrics_table.setItem(row, col, QTableWidgetItem(str(val)))
2240             self._log(f"Epoch {self._epoch}/{self._total_epochs} | Loss: {loss} |
2241 Acc: {acc} | F1: {f1}")
2242             if self._epoch >= self._total_epochs:
2243                 self._timer.stop(); self._training = False
2244                 self.status_label.setText("    状    态    :    训    练    完    成    ✓    ");
2245 self.status_label.setStyleSheet("color:#2E7D32;font-weight:bold;font-size:14px;
2246 ")
2247                 self._log("✔ 训    练    完    成    !"); task_sm.transition("train_001",
2248 TaskState.COMPLETED)
2249
2250     def _log(self, msg: str):

```

```

2251         self.log_text.append(f"[{time.strftime('%H:%M:%S')}] {msg}")
2252
2253     def _export_model(self):
2254         QMessageBox.information(self, "导出", f"模型已导出为 ONNX/PKL 格式\n 保
2255 存路径: ./exports/{self.tmpl_combo.currentText()}_{int(time.time())}.onnx")
2256
2257     def _import_model(self):
2258         QMessageBox.information(self, "导入", "请选择模型文件 (.onnx/.pkl/.h5)\n
2259 导入后可在推理节点中使用")
2260
2261     def _export_log(self):
2262         QMessageBox.information(self, "导出日志", f"训练日志已导出\n 共
2263 {self.metrics_table.rowCount()} 条记录")
2264
2265     def get_module_widget(user):
2266         return ModelTrainingWidget(user)
2267
2268 # ==== modules\project_manager.py (173 行) ====
2269 """项目管理模块 - 项目文件夹分类、资源管理、文件检索、冗余清理"""
2270 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
2271 QLabel,
2272     QGroupBox, QTreeWidget, QTreeWidgetItem, QListWidget, QLineEdit, QTextEdit,
2273     QSplitter, QMessageBox, QInputDialog)
2274 from PyQt6.QtCore import Qt
2275 from core.auth import Role, OpAction, auth_engine
2276 from core.state_machine import flow_sm, FlowState
2277 from core.data_consistency import data_engine
2278 import time, os, json
2279
2280 class ProjectManagerWidget(QWidget):
2281     def __init__(self, user):
2282         super().__init__()
2283         self.user = user; self._projects = {}
2284         self._setup_ui(); self._init_demo_projects()
2285
2286     def _setup_ui(self):
2287         layout = QVBoxLayout(self)
2288
2289         header = QHBoxLayout()
2290         title = QLabel(" 项目资源管理")
2291         title.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
2292         header.addWidget(title); header.addStretch()
2293         header.addWidget(QPushButton(" 新建项目", clicked=self._new_project))
2294         header.addWidget(QPushButton("      新      建      文      件      夹      ",
2295 clicked=self._new_folder))
2296         header.addWidget(QPushButton("          检          索          文          件          ",
2297 clicked=self._search_files))
2298         header.addWidget(QPushButton("          冗          余          清          理          ",
2299 clicked=self._clean_redundant))
2300         header.addWidget(QPushButton("          导          入          资          源          ",

```

```

2301 clicked=self._import_resource))
2302         layout.addLayout(header)
2303
2304         main = QSplitter(Qt.Orientation.Horizontal)
2305
2306         left = QVBoxLayout()
2307         left.setAlignment(Qt.AlignmentFlag.AlignTop)
2308
2309         tree_gb = QGroupBox("    项目目录")
2310         tl = QVBoxLayout()
2311         self.tree = QTreeWidget()
2312         self.tree.setHeaderLabels(["名称", "类型", "修改时间"])
2313         self.tree.itemClicked.connect(self._on_tree_click)
2314         tl.addWidget(self.tree)
2315         tree_gb.setLayout(tl)
2316         left.addWidget(tree_gb)
2317
2318         search_gb = QGroupBox("    快速检索")
2319         sl = QVBoxLayout()
2320         self.search_input = QLineEdit(); self.search_input.setPlaceholderText("
2321 输入文件名或关键字...")
2322         self.search_input.textChanged.connect(self._on_search)
2323         sl.addWidget(self.search_input)
2324         self.search_result = QListWidget(); sl.addWidget(self.search_result)
2325         search_gb.setLayout(sl)
2326         left.addWidget(search_gb)
2327
2328         left_w = QWidget(); left_w.setLayout(left); left_w.setMaximumWidth(320)
2329
2330         right = QVBoxLayout()
2331         right.addWidget(QLabel("        资        源        详        情        ",
2332 styleSheet="color:#E65100;font-weight:bold;"))
2333         self.detail_text = QTextEdit(); self.detail_text.setReadOnly(True)
2334         self.detail_text.setStyleSheet("background:#FFFDE7;color:#4E342E;")
2335         right.addWidget(self.detail_text)
2336
2337         op_row = QHBoxLayout()
2338         op_row.addWidget(QPushButton("📄 重命名", clicked=self._rename_item))
2339         op_row.addWidget(QPushButton("    复制路径", clicked=self._copy_path))
2340         op_row.addWidget(QPushButton("    删除", clicked=self._delete_item))
2341         op_row.addWidget(QPushButton("        压        缩        下        载        ",
2342 clicked=self._download_zip))
2343         right.addLayout(op_row)
2344
2345         right_w = QWidget(); right_w.setLayout(right)
2346         main.addWidget(left_w); main.addWidget(right_w)
2347         main.setSizes([320, 800])
2348         layout.addWidget(main)
2349
2350         def _init_demo_projects(self):

```

```

2351     projects = {
2352         "图像分类项目": {"datasets": ["train.csv", "val.csv", "test_images/"],
2353         "models": ["model_v3.onnx", "checkpoint.pkl"],
2354         "scripts": ["preprocess.py", "train.py", "infer.py"], "logs":
2355         ["train_log.txt"], "exports": ["result_v3.onnx"]},
2356         "目标检测项目": {"datasets": ["coco_subset/", "annotations.json"],
2357         "models": ["yolo_v5.onnx", "detector.pkl"],
2358         "scripts": ["detect.py"], "exports": ["detect_result.onnx"]},
2359         "文本分析项目": {"datasets": ["corpus.csv", "embeddings.npy"],
2360         "models": ["bert_finetuned.h5"],
2361         "scripts": ["tokenize.py", "classify.py"], "exports": []},
2362         "3D 点云项目": {"datasets": ["pointcloud.ply", "labels.txt"],
2363         "models": ["pointnet.pkl"],
2364         "scripts": ["voxelize.py"], "logs": ["train_3d.log"], "exports":
2365         []},
2366         "时序预测项目": {"datasets": ["sales_2023.csv"], "models":
2367         ["lstm_model.h5"],
2368         "scripts": ["forecast.py"], "exports": ["forecast.onnx"]},
2369     }
2370     self._projects = projects
2371     for proj_name, folders in projects.items():
2372         proj_item = QTreeWidgetItem([proj_name, "项目",
2373         time.strftime("%Y-%m-%d")])
2374         for folder_name, files in folders.items():
2375             folder_item = QTreeWidgetItem([folder_name, "文件夹", ""])
2376             for f in files:
2377                 ftype = "模型" if any(f.endswith(e) for e in
2378                 ['.onnx', '.pkl', '.h5']) else \
2379                 "数据" if any(f.endswith(e) for e in
2380                 ['.csv', '.json', '.ply', '.npy', '.txt']) else \
2381                 "脚本" if f.endswith('.py') else "日志" if
2382                 f.endswith('.log') else "文件夹" if f.endswith('/') else "文件"
2383                 QTreeWidgetItem(folder_item, [f, ftype,
2384                 time.strftime("%H:%M")])
2385                 proj_item.addChild(folder_item)
2386                 self.tree.addTopLevelItem(proj_item)
2387
2388     def _on_tree_click(self, item):
2389         details = f"名称: {item.text(0)}\n类型: {item.text(1)}\n"
2390         if item.text(2): details += f"修改时间: {item.text(2)}\n"
2391         if item.parent():
2392             details += f"路径: {item.parent().text(0)}/{item.text(0)}\n"
2393         self.detail_text.setPlainText(details)
2394
2395     def _new_project(self):
2396         name, ok = QDialog.getText(self, "新建项目", "项目名称:")
2397         if ok and name:
2398             item = QTreeWidgetItem([name, "项目", time.strftime("%Y-%m-%d")])
2399             for f in ["datasets", "models", "scripts", "logs", "exports"]:
2400                 item.addChild(QTreeWidgetItem([f, "文件夹", ""]))

```



```

2401         self.tree.addTopLevelItem(item)
2402         self._projects[name] = {f: [] for f in ["datasets", "models", "scripts",
2403 "logs", "exports"]}
2404
2405     def _new_folder(self):
2406         current = self.tree.currentItem()
2407         if current:
2408             name, ok = QInputDialog.getText(self, "新建文件夹", "文件夹名:")
2409             if ok and name:
2410                 current.addChild(QTreeWidgetItem([name, "文件夹", ""]))
2411
2412     def _search_files(self):
2413         query = self.search_input.text().lower()
2414         self.search_result.clear()
2415         def search_item(item):
2416             if query in item.text(0).lower():
2417                 self.search_result.addItem(f"                                {item.text(0)}
2418 ({item.text(1)})")
2419             for i in range(item.childCount()):
2420                 search_item(item.child(i))
2421         for i in range(self.tree.topLevelItemCount()):
2422             search_item(self.tree.topLevelItem(i))
2423
2424     def _on_search(self, text):
2425         if len(text) >= 1: self._search_files()
2426
2427     def _clean_redundant(self):
2428         QMessageBox.information(self, "冗余清理", "扫描完成\n 发现 3 个冗余日志
2429 文件, 可释放 2.5MB\n 建议清理后刷新项目目录")
2430
2431     def _import_resource(self):
2432         QMessageBox.information(self, "导入资源", "请将外部资源文件拖入对应项目
2433 文件夹\n 支持: .onnx/.pkl/.h5/.csv/.json/.py/.log")
2434
2435     def _rename_item(self):
2436         current = self.tree.currentItem()
2437         if current:
2438             name, ok = QInputDialog.getText(self, "重命名", "新名称:",
2439 text=current.text(0))
2440             if ok and name: current.setText(0, name)
2441
2442     def _copy_path(self):
2443         current = self.tree.currentItem()
2444         if current:
2445             path = current.text(0)
2446             parent = current.parent()
2447             while parent:
2448                 path = parent.text(0) + "/" + path
2449                 parent = parent.parent()
2450             from PyQt6.QtWidgets import QApplication

```

```

2451         QApplication.clipboard().setText(path)
2452         QMessageBox.information(self, "复制", f"路径已复制: {path}")
2453
2454     def _delete_item(self):
2455         current = self.tree.currentItem()
2456         if current:
2457             reply = QMessageBox.question(self, "确认删除", f"确定删除
2458 [{current.text(0)}] 吗? ")
2459             if reply == QMessageBox.StandardButton.Yes:
2460                 parent = current.parent() or self.tree.invisibleRootItem()
2461                 parent.removeChild(current)
2462
2463     def _download_zip(self):
2464         QMessageBox.information(self, "压缩下载", "项目资源压缩包已准备\n 包含所
2465 有相关脚本、模型、数据文件")
2466
2467     def get_module_widget(user):
2468         return ProjectManagerWidget(user)
2469
2470     # ==== modules\report_center.py (170 行) ====
2471     """报表中心 - 模型指标/训练耗时/推理耗时等图表展示, 支持滚轮浏览"""
2472     from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
2473     QLabel,
2474     QGroupBox, QScrollArea, QGridLayout, QComboBox, QTableWidget,
2475     QTableWidgetItem,
2476     QHeaderView, QSplitter, QMessageBox)
2477     from PyQt6.QtCore import Qt
2478     from PyQt6.QtGui import QPainter, QPen, QColor, QFont, QLinearGradient, QBrush
2479     from core.auth import Role, OpAction, auth_engine
2480     from core.algorithms import FeatureAnalyzer
2481     import math
2482
2483     class MiniChart(QWidget):
2484         """微型图表组件"""
2485         def __init__(self, title: str, data: list, color: str = "#4fc3f7", chart_type:
2486 str = "line", parent=None):
2487             super().__init__(parent)
2488             self._title = title; self._data = data; self._color = QColor(color)
2489             self._chart_type = chart_type
2490             self.setMinimumSize(280, 200); self.setMaximumHeight(220)
2491
2492         def paintEvent(self, event):
2493             p = QPainter(self); p.setRenderHint(QPainter.RenderHint.Antialiasing)
2494             p.fillRect(self.rect(), QColor(255, 248, 225))
2495             p.setPen(QPen(QColor("#FFD54F"), 1))
2496             p.drawRoundedRect(self.rect().adjusted(1, 1, -1, -1), 8, 8)
2497
2498             p.setPen(QPen(QColor("#4E342E"))); p.setFont(QFont("Microsoft YaHei", 11,
2499 QFont.Weight.Bold))
2500             p.drawText(12, 24, self._title)

```

```

2501
2502     if not self._data: return
2503     margin_l, margin_r, margin_t, margin_b = 40, 16, 40, 36
2504     w = self.width() - margin_l - margin_r
2505     h = self.height() - margin_t - margin_b
2506     if w <= 0 or h <= 0: return
2507
2508     vals = [float(v) for v in self._data if isinstance(v, (int, float))]
2509     if not vals: return
2510     min_v, max_v = min(vals), max(vals)
2511     if max_v == min_v: max_v = min_v + 1
2512     range_v = max_v - min_v
2513
2514     # 网格线
2515     for i in range(5):
2516         y = margin_t + h * i / 4
2517         p.setPen(QPen(QColor(255, 213, 79, 120), 1, Qt.PenStyle.DotLine))
2518         p.drawLine(int(margin_l), int(y), int(margin_l + w), int(y))
2519         p.setPen(QPen(QColor("#795548")))
2520         val = max_v - range_v * i / 4
2521         p.drawText(2, int(y) + 4, f"{val:.1f}")
2522
2523     # 数据线/柱
2524     p.setPen(QPen(self._color, 2))
2525     step = w / max(1, len(vals) - 1) if len(vals) > 1 else w
2526     pts = []
2527     for i, val in enumerate(vals):
2528         x = margin_l + i * step; y = margin_t + h * (1 - (val - min_v) / range_v)
2529         pts.append((x, y))
2530     if self._chart_type == "bar":
2531         bar_w = max(6, step * 0.6)
2532         for i, (x, y) in enumerate(pts):
2533             gradient = QLinearGradient(x, y, x, margin_t + h)
2534             gradient.setColorAt(0, self._color.lighter(120))
2535             gradient.setColorAt(1, QColor(self._color.red()//3,
2536 self._color.green()//3, self._color.blue()//3))
2537             p.setBrush(QBrush(gradient))
2538             p.setPen(QPen(self._color, 1))
2539             p.drawRect(int(x - bar_w/2), int(y), int(bar_w), int(margin_t + h
2540 - y))
2541     else:
2542         path = None
2543         from PyQt6.QtGui import QPainterPath
2544         path = QPainterPath()
2545         path.moveTo(pts[0][0], pts[0][1])
2546         for x, y in pts[1:]: path.lineTo(x, y)
2547         p.drawPath(path)
2548         for x, y in pts:
2549             p.setBrush(self._color); p.setPen(QPen(QColor("#FFF8E1"), 1))
2550             p.drawEllipse(int(x)-3, int(y)-3, 6, 6)

```

```

2551
2552     def wheelEvent(self, event):
2553         """滚轮缩放图表"""
2554         self.setMaximumHeight(max(120, min(400, self.maximumHeight() + (20 if
2555 event.angleDelta().y() > 0 else -20))))
2556         self.update()
2557
2558 class ReportCenterWidget(QWidget):
2559     def __init__(self, user):
2560         super().__init__()
2561         self.user = user; self._setup_ui()
2562
2563     def _setup_ui(self):
2564         layout = QVBoxLayout(self)
2565
2566         header = QHBoxLayout()
2567         title = QLabel("  报表中心")
2568         title.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
2569         header.addWidget(title); header.addStretch()
2570         header.addWidget(QLabel("项目:"))
2571         self.proj_combo = QComboBox()
2572         self.proj_combo.addItem("图像分类项目", "目标检测项目", "文本分析项目
2573 ", "3D 点云项目", "时序预测项目"])
2574         header.addWidget(self.proj_combo)
2575         header.addWidget(QPushButton("  刷新", clicked=self._refresh))
2576         header.addWidget(QPushButton("      导      出      报      表      ",
2577 clicked=self._export_report))
2578         layout.addLayout(header)
2579
2580         scroll = QScrollArea()
2581         scroll.setWidgetResizable(True)
2582         scroll_w = QWidget()
2583         scroll_layout = QVBoxLayout(scroll_w)
2584
2585         # 图表网格
2586         chart_gb = QGroupBox("  模型性能指标（滚轮缩放图表）")
2587         cg = QGridLayout()
2588         charts = [
2589             ("  训    练    /    验    证    准    确    率    曲    线    ",
2590 [0.45, 0.58, 0.65, 0.72, 0.78, 0.82, 0.85, 0.88, 0.90, 0.91, 0.92, 0.93],
2591 "#4fc3f7",
2592 "line"),
2593             ("  训    练    Loss    下    降    曲    线    ",
2594 [2.1, 1.5, 1.1, 0.8, 0.6, 0.45, 0.35, 0.28, 0.22, 0.18, 0.15, 0.12], "#ef5350", "line"),
2595             ("各类别F1分数", [0.88, 0.92, 0.85, 0.79, 0.91, 0.87], "#66bb6a", "bar"),
2596             ("推理耗时分布 (ms)", [12, 15, 11, 18, 13, 10, 22, 14, 16, 13, 11, 17],
2597 "#ffa726", "bar"),
2598             ("模型参数量对比 (M)", [25.3, 48.1, 12.7, 8.4, 33.2, 19.6], "#ab47bc",
2599 "bar"),
2600             ("训练耗时 (Epoch)", [45, 42, 38, 35, 32, 30, 28, 27, 26, 25, 24, 23], "#00bcd4",

```

```

2601     ]
2602     for i, (title, data, color, ctype) in enumerate(charts):
2603         cg.addWidget(MiniChart(title, data, color, ctype), i // 3, i % 3)
2604     chart_gb.setLayout(cg)
2605     scroll_layout.addWidget(chart_gb)
2606
2607     # 数据表格
2608     table_gb = QGroupBox("    详细指标数据")
2609     tl = QVBoxLayout()
2610     self.metrics_table = QTableWidgetItem()
2611     self.metrics_table.setColumnCount(7)
2612     self.metrics_table.setHorizontalHeaderLabels(["Epoch", "Train Loss", "Val
2613 Loss", "Accuracy", "Precision", "Recall", "F1"])
2614
2615     self.metrics_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMo
2616 de.Stretch)
2617     demo_data = [
2618
2619 (1, 2.10, 1.95, 0.45, 0.42, 0.38, 0.40), (2, 1.50, 1.42, 0.58, 0.55, 0.52, 0.53),
2620
2621 (3, 1.10, 1.05, 0.65, 0.63, 0.60, 0.61), (4, 0.80, 0.78, 0.72, 0.70, 0.69, 0.69),
2622
2623 (5, 0.60, 0.58, 0.78, 0.76, 0.74, 0.75), (6, 0.45, 0.44, 0.82, 0.80, 0.79, 0.79),
2624
2625 (7, 0.35, 0.34, 0.85, 0.84, 0.82, 0.83), (8, 0.28, 0.27, 0.88, 0.87, 0.85, 0.86),
2626
2627 (9, 0.22, 0.21, 0.90, 0.89, 0.88, 0.88), (10, 0.18, 0.18, 0.91, 0.90, 0.89, 0.89),
2628
2629 (11, 0.15, 0.15, 0.92, 0.91, 0.90, 0.91), (12, 0.12, 0.12, 0.93, 0.92, 0.91, 0.92),
2630     ]
2631     self.metrics_table.setRowCount(len(demo_data))
2632     for r, row_data in enumerate(demo_data):
2633         for c, val in enumerate(row_data):
2634             self.metrics_table.setItem(r, c, QTableWidgetItem(str(val)))
2635     tl.addWidget(self.metrics_table)
2636     table_gb.setLayout(tl)
2637     scroll_layout.addWidget(table_gb)
2638
2639     # 汇总统计
2640     summary_gb = QGroupBox("    汇总统计")
2641     sl = QHBoxLayout()
2642     summaries = [("总训练轮次", "12"), ("最佳准确率", "0.93"), ("最佳 F1 分数
2643 ", "0.92"),
2644                  ("总训练耗时", "4h32m"), ("平均推理耗时", "14.7ms"), ("模型大
2645 小", "25.3MB")]
2646     for label, val in summaries:
2647         card = QGroupBox(label);
2648         card.setStyleSheet("QGroupBox{color:#E65100;font-size:11px;}")
2649         vl = QLabel(val); vl.setAlignment(Qt.AlignmentFlag.AlignCenter)
2650         vl.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")

```

```

2651         card.setLayout(QVBoxLayout()); card.layout().addWidget(v1)
2652         sl.addWidget(card)
2653         summary_gb.setLayout(sl)
2654         scroll_layout.addWidget(summary_gb)
2655         scroll_layout.addStretch()
2656
2657         scroll.setWidget(scroll_w)
2658         layout.addWidget(scroll)
2659
2660         def _refresh(self):
2661             QMessageBox.information(self, "刷新", f"已刷新
2662 [{self.proj_combo.currentText()}] 报表数据")
2663
2664         def _export_report(self):
2665             QMessageBox.information(self, "导出", "报表已导出为 PDF/Excel 格式\n 包含
2666 全部图表与数据表格")
2667
2668         def get_module_widget(user):
2669             return ReportCenterWidget(user)
2670
2671         # ==== modules\rule_config.py (202 行) ====
2672         """业务规则配置器 - 图形化条件/动作配置、冲突检测、规则模板"""
2673         from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
2674         QLabel,
2675         QGroupBox, QGridLayout, QComboBox, QLineEdit, QSpinBox, QTextEdit,
2676         QListWidget,
2677         QTableWidget, QTableWidgetItem, QHeaderView, QMessageBox, QSplitter)
2678         from PyQt6.QtCore import Qt
2679         from core.auth import Role, OpAction, auth_engine
2680         from core.rule_engine import (rule_engine, TriggerType, LogicOp, CompareOp,
2681         Condition, RuleAction, Rule)
2682
2683         class RuleConfigWidget(QWidget):
2684             def __init__(self, user):
2685                 super().__init__()
2686                 self.user = user
2687                 self._setup_ui(); self._load_rules()
2688
2689             def _setup_ui(self):
2690                 layout = QHBoxLayout(self)
2691
2692                 left_panel = QVBoxLayout()
2693                 left_panel.setAlignment(Qt.AlignmentFlag.AlignTop)
2694
2695                 create_gb = QGroupBox("创建/编辑规则")
2696                 cl = QVBoxLayout()
2697                 cl.addWidget(QLabel("规则名称:"))
2698                 self.rule_name = QLineEdit("新规则"); cl.addWidget(self.rule_name)
2699                 cl.addWidget(QLabel("触发方式:"))
2700                 self.trigger_combo = QComboBox()

```

```

2701         self.trigger_combo.addItem([t.value for t in TriggerType]);
2702     cl.addWidget(self.trigger_combo)
2703         cl.addWidget(QLabel("优先级:"))
2704         self.priority_spin = QSpinBox(); self.priority_spin.setRange(1, 10);
2705     self.priority_spin.setValue(5)
2706         cl.addWidget(self.priority_spin)
2707
2708         cl.addWidget(QLabel("条件字段+值:"))
2709         cond_row = QHBoxLayout()
2710         self.cond_field = QLineEdit("accuracy"); self.cond_value =
2711     QLineEdit("0.8")
2712         self.cond_op = QComboBox()
2713         self.cond_op.addItem([op.value for op in CompareOp])
2714         cond_row.addWidget(self.cond_field); cond_row.addWidget(self.cond_op);
2715     cond_row.addWidget(self.cond_value)
2716         cl.addLayout(cond_row)
2717
2718         cl.addWidget(QLabel("动作类型:"))
2719         action_row = QHBoxLayout()
2720         self.action_type = QComboBox()
2721         self.action_type.addItem(["alert", "log", "abort", "transform",
2722     "route"])
2723         self.action_param = QLineEdit({'msg':"条件满足"})
2724         action_row.addWidget(self.action_type);
2725     action_row.addWidget(self.action_param)
2726         cl.addLayout(action_row)
2727
2728         btn_row = QHBoxLayout()
2729         btn_row.addWidget(QPushButton("✚ 添加规则", clicked=self._add_rule))
2730         btn_row.addWidget(QPushButton("存 为 模 板 ",
2731     clicked=self._save_template))
2732         btn_row.addWidget(QPushButton("从 模 板 创 建 ",
2733     clicked=self._from_template))
2734         cl.addLayout(btn_row)
2735         create_gb.setLayout(cl)
2736         left_panel.addWidget(create_gb)
2737
2738         conflict_gb = QGroupBox("⚠ 冲突检测")
2739         cfl = QVBoxLayout()
2740         self.conflict_text = QTextEdit(); self.conflict_text.setReadOnly(True);
2741     self.conflict_text.setMaximumHeight(100)
2742         cfl.addWidget(self.conflict_text)
2743         cfl.addWidget(QPushButton("检 测 冲 突 ",
2744     clicked=self._detect_conflicts))
2745         cfl.addWidget(QPushButton("自动修复", clicked=self._auto_resolve))
2746         conflict_gb.setLayout(cfl)
2747         left_panel.addWidget(conflict_gb)
2748
2749         left_widget = QWidget(); left_widget.setLayout(left_panel);
2750     left_widget.setMaximumWidth(320)

```

```

2751
2752     right_panel = QVBoxLayout()
2753     self.rule_table = QTableWidgetItem(0, 6)
2754     self.rule_table.setHorizontalHeaderLabels(["规则 ID", "名称", "触发方式",
2755 " ", "优先级", "启用", "分组"])
2756
2757 self.rule_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.
2758 Stretch)
2759     self.rule_table.itemSelectionChanged.connect(self._on_rule_select)
2760     right_panel.addWidget(QLabel(" 规则列表（可点击选择编辑）",
2761 styleSheet="color:#E65100;font-weight:bold;"))
2762     right_panel.addWidget(self.rule_table)
2763
2764     exec_gb = QGroupBox(" 规则测试")
2765     el = QVBoxLayout()
2766     test_row = QHBoxLayout()
2767     self.test_context = QLineEdit('{"accuracy":0.92,"node_count":5}')
2768     test_row.addWidget(QLabel("      上      下      文      JSON:")));
2769 test_row.addWidget(self.test_context)
2770     el.addLayout(test_row)
2771     test_btn_row = QHBoxLayout()
2772     test_btn_row.addWidget(QPushButton(" ▶      测 试 选 中 规 则 ",
2773 clicked=self._test_rule))
2774     test_btn_row.addWidget(QPushButton(" ▶      测 试 全 部 规 则 ",
2775 clicked=self._test_all))
2776     test_btn_row.addWidget(QPushButton("      切 换 启 用      ",
2777 clicked=self._toggle_rule))
2778     test_btn_row.addWidget(QPushButton("      删 除 规 则      ",
2779 clicked=self._delete_rule))
2780     el.addLayout(test_btn_row)
2781     self.test_result = QTextEdit(); self.test_result.setReadOnly(True);
2782 self.test_result.setMaximumHeight(80)
2783     el.addWidget(self.test_result)
2784     exec_gb.setLayout(el)
2785     right_panel.addWidget(exec_gb)
2786
2787     right_widget = QWidget(); right_widget.setLayout(right_panel)
2788     layout.addWidget(left_widget); layout.addWidget(right_widget, 1)
2789
2790     def _add_rule(self):
2791         if not auth_engine.check_permission(self.user, "rule_config",
2792 OpAction.CREATE):
2793             QMessageBox.warning(self, "权限不足", "无创建规则权限"); return
2794         try:
2795             conditions = []
2796             if self.cond_field.text():
2797                 op_map = {op.value: op for op in CompareOp}
2798                 conditions = [Condition(field=self.cond_field.text(),
2799 op=op_map.get(self.cond_op.currentText(), CompareOp.EQ),
2800 value=self._parse_value(self.cond_value.text()))]

```



```

2801         actions = [RuleAction(action_type=self.action_type.currentText(),
2802                                params=self._parse_json(self.action_param.text()))]
2803         trigger_map = {t.value: t for t in TriggerType}
2804         rule = rule_engine.create_rule(self.rule_name.text(),
2805                                       trigger_map[self.trigger_combo.currentText()],
2806                                       conditions, actions, self.priority_spin.value())
2807         QMessageBox.information(self, "成功", f"规则 [{rule.rule_id}] 已创建
2808 ")
2809         self._load_rules()
2810     except Exception as e:
2811         QMessageBox.critical(self, "错误", str(e))
2812
2813     def _load_rules(self):
2814         rules = rule_engine.get_rules()
2815         self.rule_table.setRowCount(0)
2816         for rule in rules:
2817             r = self.rule_table.rowCount()
2818             self.rule_table.insertRow(r)
2819             for c, val in enumerate([rule.rule_id, rule.name, rule.trigger.value,
2820                                     str(rule.priority), "✓ if rule.enabled else "⊖, rule.group]):
2821                 self.rule_table.setItem(r, c, QTableWidgetItem(str(val)))
2822
2823     def _detect_conflicts(self):
2824         conflicts = rule_engine.detect_conflicts()
2825         if conflicts:
2826             msgs = "\n".join([f"⚠ {c['detail']} [{c['type']}]" for c in
2827 conflicts])
2828             self.conflict_text.setHtml(f'<span
2829 style="color:#D84315;">{msgs}</span>')
2830         else:
2831             self.conflict_text.setHtml(f'<span style="color:#2E7D32;">✓ 未检测到
2832 规则冲突</span>')
2833
2834     def _auto_resolve(self):
2835         conflicts = rule_engine.detect_conflicts()
2836         resolved = 0
2837         for c in conflicts:
2838             if rule_engine.toggle_rule(c["rule_b"]): resolved += 1
2839             self.conflict_text.setHtml(f'<span style="color:#2E7D32;">已禁用
2840 {resolved} 个冲突规则</span>')
2841             self._load_rules()
2842
2843     def _test_rule(self):
2844         selected = self.rule_table.selectedItems()
2845         if not selected: return
2846         rid = self.rule_table.item(selected[0].row(), 0).text()
2847         try:
2848             context = eval(self.test_context.text())
2849             result = rule_engine.evaluate(rid, context)
2850             self.test_result.setHtml(f'<span style="color:{"#66bb6a" if result

```

```

2851 else "#ef5350";">规则 {rid}: {"触发" if result else "未触发"}</span>')
2852     except Exception as e:
2853         self.test_result.setHtml(f'<span style="color:#D84315;">测试失败:
2854 {e}</span>')
2855
2856     def _test_all(self):
2857         try:
2858             context = eval(self.test_context.text())
2859             fired = rule_engine.evaluate_all(context)
2860             names = [r.name for r in fired]
2861             self.test_result.setHtml(f'<span style="color:#E65100;">触 发
2862 {len(fired)} 条规则: {names}</span>')
2863         except Exception as e:
2864             self.test_result.setHtml(f'<span style="color:#D84315;">失 败 :
2865 {e}</span>')
2866
2867     def _save_template(self):
2868         selected = self.rule_table.selectedItems()
2869         if not selected:
2870             QMessageBox.warning(self, "提示", "请先选择规则"); return
2871         rid = self.rule_table.item(selected[0].row(), 0).text()
2872         tid = rule_engine.save_as_template(rid, f"模板_{rid}")
2873         if tid: QMessageBox.information(self, "保存模板", f"模板 [{tid}] 已保存
2874 ")
2875
2876     def _from_template(self):
2877         QMessageBox.information(self, "模板", "请在模板列表中选择模板创建\n当前
2878 模板库: 2 个预置模板")
2879
2880     def _toggle_rule(self):
2881         selected = self.rule_table.selectedItems()
2882         if selected:
2883             rid = self.rule_table.item(selected[0].row(), 0).text()
2884             rule_engine.toggle_rule(rid); self._load_rules()
2885
2886     def _delete_rule(self):
2887         selected = self.rule_table.selectedItems()
2888         if selected:
2889             rid = self.rule_table.item(selected[0].row(), 0).text()
2890             if rule_engine.delete_rule(rid): self._load_rules()
2891
2892     def _on_rule_select(self):
2893         selected = self.rule_table.selectedItems()
2894         if selected:
2895             row = selected[0].row()
2896             self.rule_name.setText(self.rule_table.item(row, 1).text())
2897
2898     def _parse_value(self, s: str):
2899         try: return eval(s)
2900         except: return s

```

```

2901
2902     def _parse_json(self, s: str):
2903         try:
2904             import json; return json.loads(s)
2905         except: return {"msg": s}
2906
2907     def get_module_widget(user):
2908         return RuleConfigWidget(user)
2909
2910     # ==== modules\system_admin.py (220 行) ====
2911     """系统管理模块 - 用户管理、权限分配、审计日志、系统配置"""
2912     from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
2913     QLabel,
2914     QGroupBox, QTableWidget, QTableWidgetItem, QHeaderView, QTextEdit, QComboBox,
2915     QSplitter, QMessageBox, QInputDialog, QListWidget, QCheckBox)
2916     from PyQt6.QtCore import Qt
2917     from core.auth import Role, OpAction, auth_engine
2918     from core.rule_engine import rule_engine
2919     from core.state_machine import flow_sm, task_sm
2920     import time
2921
2922     class SystemAdminWidget(QWidget):
2923         def __init__(self, user):
2924             super().__init__()
2925             self.user = user
2926             self._setup_ui()
2927             self._load_data()
2928
2929         def _setup_ui(self):
2930             layout = QVBoxLayout(self)
2931
2932             header = QHBoxLayout()
2933             title = QLabel("    系统管理")
2934             title.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
2935             header.addWidget(title); header.addStretch()
2936             if self.user.role == Role.ADMIN:
2937                 header.addWidget(QPushButton("    +        新    建    用    户    ",
2938 clicked=self._create_user))
2939                 header.addWidget(QPushButton("        批    量    授    权    ",
2940 clicked=self._batch_grant))
2941                 header.addWidget(QPushButton("    ←        回    收    权    限    ",
2942 clicked=self._revoke_perm))
2943                 header.addWidget(QPushButton("    审计日志", clicked=self._show_audit))
2944                 header.addWidget(QPushButton("⊗ 系统配置", clicked=self._sys_config))
2945             layout.addLayout(header)
2946
2947             main = QSplitter(Qt.Orientation.Horizontal)
2948
2949             left = QVBoxLayout()
2950             left.setAlignment(Qt.AlignmentFlag.AlignTop)

```

```

2951
2952     user_gb = QGroupBox("    用户管理")
2953     ul = QVBoxLayout()
2954     self.user_table = QTableWidgetItem(0, 5)
2955     self.user_table.setHorizontalHeaderLabels(["UID", "用户名", "角色", "最
2956 后登录", "状态"])
2957
2958     self.user_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.
2959 Stretch)
2960     self.user_table.itemSelectionChanged.connect(self._on_user_select)
2961     ul.addWidget(self.user_table)
2962     if self.user.role == Role.ADMIN:
2963         ur = QHBoxLayout()
2964         ur.addWidget(QPushButton("✎ 修改角色", clicked=self._change_role))
2965         ur.addWidget(QPushButton("🔒 锁定账号", clicked=self._lock_user))
2966         ur.addWidget(QPushButton("🔓 解锁账号", clicked=self._unlock_user))
2967         ul.addLayout(ur)
2968     user_gb.setLayout(ul)
2969     left.addWidget(user_gb)
2970
2971     perm_gb = QGroupBox("    权限配置")
2972     pl = QVBoxLayout()
2973     pl.addWidget(QLabel("资源权限:"))
2974     self.perm_list = QListWidget()
2975     resources =
2976 ["dashboard", "model_designer", "lowcode_editor", "model_training",
2977
2978 "vision_lab", "vision_3d", "rule_config", "data_manager", "task_manager",
2979
2980 "project_manager", "component_market", "report_center", "system_admin"]
2981     for res in resources:
2982         self.perm_list.addItem(res)
2983
2984     self.perm_list.setSelectionMode(QListWidget.SelectionMode.MultiSelection)
2985     pl.addWidget(self.perm_list)
2986     pl.addWidget(QLabel("操作权限:"))
2987     op_row = QHBoxLayout()
2988     self._op_checks = {}
2989     for op in OpAction:
2990         cb = QCheckBox(op.value); self._op_checks[op] = cb;
2991     op_row.addWidget(cb)
2992     pl.addLayout(op_row)
2993     if self.user.role == Role.ADMIN:
2994         pl.addWidget(QPushButton("✔ 授予权限", clicked=self._grant_perm))
2995         pl.addWidget(QPushButton("✖ 回收权限", clicked=self._revoke_perm))
2996     perm_gb.setLayout(pl)
2997     left.addWidget(perm_gb)
2998
2999     left_w = QWidget(); left_w.setLayout(left); left_w.setMaximumWidth(420)
3000

```

```

3001         right = QVBoxLayout()
3002
3003         log_gb = QGroupBox("    操作审计日志")
3004         ll = QVBoxLayout()
3005         self.log_text = QTextEdit(); self.log_text.setReadOnly(True)
3006
3007         self.log_text.setStyleSheet("background:#FFF8E1;color:#2E7D32;font-family:Conso
3008         las;font-size:11px;")
3009         ll.addWidget(self.log_text)
3010         log_filter = QHBoxLayout()
3011         log_filter.addWidget(QLabel("筛选:"))
3012         self.log_filter = QComboBox()
3013         self.log_filter.addItem("全      部
3014         ", "LOGIN_SUCCESS", "LOGIN_FAIL", "PERM_GRANT", "PERM_REVOKE", "SESSION", "REGISTER"]
3015         )
3016         self.log_filter.currentTextChanged.connect(self._filter_logs)
3017         log_filter.addWidget(self.log_filter)
3018         log_filter.addStretch()
3019         log_filter.addWidget(QPushButton("      导      出      日      志      ",
3020         clicked=self._export_logs))
3021         ll.addLayout(log_filter)
3022         log_gb.setLayout(ll)
3023         right.addWidget(log_gb)
3024
3025         stats_gb = QGroupBox("    系统统计")
3026         sl = QHBoxLayout()
3027         self.stats_labels = {}
3028         for label in ["总用户", "活跃会话", "规则总数", "流程总数", "任务总数"]:
3029             card = QGroupBox(label);
3030             card.setStyleSheet("QGroupBox{color:#E65100;font-size:11px;}")
3031             vl = QLabel("--"); vl.setAlignment(Qt.AlignmentFlag.AlignCenter)
3032             vl.setStyleSheet("color:#4E342E;font-size:18px;font-weight:bold;")
3033             card.setLayout(QVBoxLayout()); card.layout().addWidget(vl)
3034             self.stats_labels[label] = vl; sl.addWidget(card)
3035         stats_gb.setLayout(sl)
3036         right.addWidget(stats_gb)
3037
3038         right_w = QWidget(); right_w.setLayout(right)
3039         main.addWidget(left_w); main.addWidget(right_w)
3040         main.setSizes([420, 700])
3041         layout.addWidget(main)
3042
3043         def _load_data(self):
3044             self.user_table.setRowCount(0)
3045             users = auth_engine.get_all_users()
3046             for user in users:
3047                 r = self.user_table.rowCount()
3048                 self.user_table.insertRow(r)
3049                 for c, val in enumerate([user.uid, user.username, user.role.value,
3050                 time.strftime("%Y-%m-%d %H:%M", time.localtime(user.last_login))

```

```

3051 if user.last_login else "从未登录",
3052     "  锁定" if time.time() < user.locked_until else "✔正常"]):
3053     self.user_table.setItem(r, c, QTableWidgetItem(str(val)))
3054     self.stats_labels["总用户"].setText(str(len(users)))
3055     self.stats_labels["      规      则      总      数
3056   "].setText(str(len(rule_engine.get_rules())))
3057     self._refresh_logs()
3058
3059     def _refresh_logs(self):
3060         logs = auth_engine.get_audit_log(limit=100)
3061         filter_val = self.log_filter.currentText()
3062         lines = []
3063         for log in reversed(logs):
3064             if filter_val != "全部" and log.get("action") != filter_val: continue
3065             ts = time.strftime("%m-%d %H:%M:%S", time.localtime(log["ts"]))
3066             lines.append(f"[{ts}]      [{log['action']}]      [{log['uid']}]
3067   [{log['detail']}]")
3068             self.log_text.setPlainText("\n".join(lines))
3069
3070     def _filter_logs(self):
3071         self._refresh_logs()
3072
3073     def _on_user_select(self):
3074         selected = self.user_table.selectedItems()
3075         if selected:
3076             uid = self.user_table.item(selected[0].row(), 0).text()
3077             user = auth_engine.get_user_by_uid(uid)
3078             if user:
3079                 for i in range(self.perm_list.count()):
3080                     item = self.perm_list.item(i)
3081                     item.setSelected(any(p.resource == item.text() for p in
3082   user.permissions))
3083
3084     def _create_user(self):
3085         name, ok = QDialog.getText(self, "新建用户", "用户名:")
3086         if not ok or not name: return
3087         pwd, ok = QDialog.getText(self, "设置密码", "密码:")
3088         if not ok: return
3089         roles = [r.value for r in Role]
3090         role_str, ok = QDialog.getItem(self, "选择角色", "角色:", roles, 0,
3091   False)
3092         if ok:
3093             role_map = {r.value: r for r in Role}
3094             user = auth_engine.register(name, pwd, role_map.get(role_str,
3095   Role.GUEST))
3096             if user:
3097                 self._load_data(); QMessageBox.information(self, "成功", f"用户
3098   [{name}] 创建成功")
3099             else:
3100                 QMessageBox.critical(self, "失败", "用户名已存在")

```

```

3101
3102     def _change_role(self):
3103         QMessageBox.information(self, "修改角色", "请在用户列表选择用户后, 通过
3104 权限配置修改角色权限")
3105
3106     def _lock_user(self):
3107         QMessageBox.information(self, "锁定", "账号已锁定\n 该用户将无法登录直到
3108 管理员解锁")
3109
3110     def _unlock_user(self):
3111         QMessageBox.information(self, "解锁", "账号已解锁\n 该用户可正常登录")
3112
3113     def _grant_perm(self):
3114         if self.user.role != Role.ADMIN:
3115             QMessageBox.warning(self, "权限不足", "仅管理员可授权"); return
3116         selected_user = self.user_table.selectedItems()
3117         if not selected_user:
3118             QMessageBox.warning(self, "提示", "请选择目标用户"); return
3119         uid = self.user_table.item(selected_user[0].row(), 0).text()
3120         resources = [item.text() for item in self.perm_list.selectedItems()]
3121         actions = {op for op, cb in self._op_checks.items() if cb.isChecked()}
3122         if not actions: actions = {OpAction.VIEW}
3123         if not resources: resources = ["dashboard"]
3124         for res in resources:
3125             auth_engine.grant_permission(self.user, uid, res, actions)
3126         self._load_data()
3127         QMessageBox.information(self, " 授 权 ", f" 已 为      [{uid}]  授 予
3128 {len(resources)} 个资源权限")
3129
3130     def _revoke_perm(self):
3131         if self.user.role != Role.ADMIN: return
3132         selected_user = self.user_table.selectedItems()
3133         if not selected_user: return
3134         uid = self.user_table.item(selected_user[0].row(), 0).text()
3135         resources = [item.text() for item in self.perm_list.selectedItems()]
3136         for res in resources:
3137             auth_engine.revoke_permission(self.user, uid, res)
3138         self._load_data()
3139         QMessageBox.information(self, "回收", f"已回收权限")
3140
3141     def _batch_grant(self):
3142         if self.user.role != Role.ADMIN: return
3143         QMessageBox.information(self, "批量授权",
3144             "批量授权功能:\n1. 选择多个目标用户\n2. 选择资源列表\n3. 选择操作权限
3145 \n4. 一键批量授予\n\n 请通过多选用户表行实现批量操作")
3146
3147     def _show_audit(self):
3148         self._refresh_logs()
3149         QMessageBox.information(self, " 审 计 日 志 ", f" 共
3150 {len(auth_engine.get_audit_log())} 条审计记录")

```

```

3151
3152     def _sys_config(self):
3153         QMessageBox.information(self, "系统配置",
3154             "⚙️ 系统配置项:\n• 会话超时: 3600 秒\n• 最大登录尝试: 5 次\n• 锁定时间: 600 秒\n• 审计日志保留: 10000 条\n• 自动刷新闻隔: 3 秒")
3155
3156     def _export_logs(self):
3157         QMessageBox.information(self, "导出", "审计日志已导出为 CSV 文件\n包含完整操作记录轨迹")
3158
3159
3160
3161     def get_module_widget(user):
3162         return SystemAdminWidget(user)
3163
3164 # ==== modules\task_manager.py (185 行) ====
3165 """任务管控模块 - 任务队列管理、运行日志、异常处理、优先级调度"""
3166 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
3167     QLabel,
3168     QGroupBox, QTableWidget, QTableWidgetItem, QHeaderView, QTextEdit, QSpinBox,
3169     QComboBox, QSplitter, QMessageBox)
3170 from PyQt6.QtCore import Qt, QTimer
3171 from core.auth import Role, OpAction, auth_engine
3172 from core.state_machine import task_sm, TaskState, flow_sm, FlowState
3173 from core.rule_engine import rule_engine, TriggerType
3174 import time, uuid
3175
3176 class TaskManagerWidget(QWidget):
3177     def __init__(self, user):
3178         super().__init__()
3179         self.user = user; self._tasks = []
3180         self._log_buffer = []; self._error_buffer = []
3181         self._setup_ui(); self._init_demo_tasks()
3182         self._timer = QTimer(self); self._timer.timeout.connect(self._refresh)
3183         self._timer.start(2000)
3184
3185     def _setup_ui(self):
3186         layout = QVBoxLayout(self)
3187
3188         header = QHBoxLayout()
3189         title = QLabel(" 任务管控中心")
3190         title.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
3191         header.addWidget(title); header.addStretch()
3192         header.addWidget(QPushButton("✚ 创建任务", clicked=self._create_task))
3193         header.addWidget(QPushButton("⏸ 暂停全部", clicked=self._pause_all))
3194         header.addWidget(QPushButton("▶ 恢复全部", clicked=self._resume_all))
3195         header.addWidget(QPushButton("          终    止    选    中    ",
3196 clicked=self._terminate_selected))
3197         header.addWidget(QPushButton("          清    理    已    完    成    ",
3198 clicked=self._clean_completed))
3199         layout.addLayout(header)
3200

```



```

3201         main = QSplitter(Qt.Orientation.Horizontal)
3202
3203         left = QVBoxLayout()
3204         left.setAlignment(Qt.AlignmentFlag.AlignTop)
3205
3206         ctrl_gb = QGroupBox("🔧 新建任务")
3207         cl = QVBoxLayout()
3208         cl.addWidget(QLabel("任务类型:"))
3209         self.task_type = QComboBox()
3210         self.task_type.addItem("模型训练", "数据预处理", "模型推理", "批量处理
3211 ", "模型导出", "报表生成", "数据校验"])
3212         cl.addWidget(self.task_type)
3213         cl.addWidget(QLabel("优先级 (1-10):"))
3214         self.task_priority = QSpinBox(); self.task_priority.setRange(1, 10);
3215 self.task_priority.setValue(5)
3216         cl.addWidget(self.task_priority)
3217         cl.addWidget(QPushButton("✔ 确认创建", clicked=self._create_task))
3218         ctrl_gb.setLayout(cl)
3219         left.addWidget(ctrl_gb)
3220
3221         stats_gb = QGroupBox(" 任务统计")
3222         sl = QVBoxLayout()
3223         self.stats_label = QLabel("排队:0 | 执行:0 | 完成:0 | 失败:0")
3224         self.stats_label.setStyleSheet("color:#795548;")
3225         sl.addWidget(self.stats_label)
3226         stats_gb.setLayout(sl)
3227         left.addWidget(stats_gb)
3228         left.addStretch()
3229
3230         left_w = QWidget(); left_w.setLayout(left); left_w.setMaximumWidth(240)
3231
3232         right = QVBoxLayout()
3233         right.addWidget(QLabel("          任          务          队          列          ",
3234 styleSheet="color:#E65100;font-weight:bold;"))
3235         self.task_table = QTableWidget(0, 6)
3236         self.task_table.setHorizontalHeaderLabels(["任务 ID", "类型", "状态", "
3237 优先级", "创建时间", "错误信息"])
3238
3239 self.task_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.
3240 Stretch)
3241         right.addWidget(self.task_table)
3242
3243         log_split = QSplitter(Qt.Orientation.Vertical)
3244         log_gb = QGroupBox(" 运行日志")
3245         ll = QVBoxLayout()
3246         self.log_text = QTextEdit(); self.log_text.setReadOnly(True)
3247
3248 self.log_text.setStyleSheet("background:#FFF8E1;color:#2E7D32;font-family:Conso
3249 las;font-size:11px;")
3250         ll.addWidget(self.log_text)

```

```

3251         log_gb.setLayout(l1)
3252
3253         err_gb = QGroupBox("✖ 异常日志 & 修复指引")
3254         el = QVBoxLayout()
3255         self.error_text = QTextEdit(); self.error_text.setReadOnly(True)
3256
3257         self.error_text.setStyleSheet("background:#FFF8E1;color:#D84315;font-family:Con
3258         solas;font-size:11px;")
3259         el.addWidget(self.error_text)
3260         el.addWidget(QPushButton("      异      常      修      复      指      引      ",
3261         clicked=self._show_repair_guide))
3262         err_gb.setLayout(el)
3263
3264         log_split.addWidget(log_gb); log_split.addWidget(err_gb)
3265         right.addWidget(log_split)
3266
3267         right_w = QWidget(); right_w.setLayout(right)
3268         main.addWidget(left_w); main.addWidget(right_w)
3269         main.setSizes([240, 900])
3270         layout.addWidget(main)
3271
3272         def _init_demo_tasks(self):
3273             demo_tasks = [
3274                 ("T001", "模型训练", [TaskState.PREPARING, TaskState.EXECUTING], 8),
3275                 ("T002", "数据预处理", [TaskState.PREPARING], 6),
3276                 ("T003", "模型推理", [], 9),
3277                 ("T004", "批量处理", [TaskState.PREPARING, TaskState.EXECUTING,
3278                 TaskState.COMPLETED], 4),
3279                 ("T005", "模型导出", [TaskState.PREPARING, TaskState.EXECUTING,
3280                 TaskState.FAILED], 7),
3281                 ("T006", "报表生成", [TaskState.PREPARING, TaskState.EXECUTING,
3282                 TaskState.PAUSED], 3),
3283             ]
3284             for tid, ttype, transitions, priority in demo_tasks:
3285                 task_sm.create_task(tid, ttype, priority)
3286                 for target_state in transitions:
3287                     task_sm.transition(tid, target_state)
3288                 t = task_sm._tasks.get(tid)
3289                 if t and t["state"] in (TaskState.FAILED,):
3290                     t["error"] = "模拟异常：数据格式校验失败"
3291             self._log("系统", "任务队列初始化完成（6个演示任务）", "INFO")
3292             self._refresh()
3293
3294         def _create_task(self):
3295             if not auth_engine.check_permission(self.user, "task_manager",
3296             OpAction.CREATE):
3297                 QMessageBox.warning(self, "权限不足", "无创建任务权限"); return
3298             tid = f"T{uuid.uuid4().hex[:6]}"
3299             task_sm.create_task(tid, self.task_type.currentText(),
3300             self.task_priority.value())

```

```

3301         self._log(tid, f"任务创建：{self.task_type.currentText()} 优先级: {self.task_priority.value()}", "CREATE")
3302         self._refresh()
3303
3304     def _pause_all(self):
3305         self._log("SYSTEM", "暂停所有执行中任务", "PAUSE")
3306
3307     def _resume_all(self):
3308         self._log("SYSTEM", "恢复所有暂停任务", "RESUME")
3309
3310     def _terminate_selected(self):
3311         selected = self.task_table.selectedItems()
3312         if selected:
3313             tid = self.task_table.item(selected[0].row(), 0).text()
3314             task_sm.transition(tid, TaskState.CANCELLED)
3315             self._log(tid, "任务强制终止", "TERMINATE")
3316             self._refresh()
3317
3318     def _clean_completed(self):
3319         self._log("SYSTEM", "清理已完成/已取消任务", "CLEAN")
3320
3321     def _refresh(self):
3322         tasks = task_sm.get_all_tasks()
3323         self.task_table.setRowCount(0)
3324         counts = {s: 0 for s in TaskState}
3325         for t in tasks:
3326             state = t["state"]
3327             if isinstance(state, TaskState):
3328                 state_str = state.value
3329                 counts[state] = counts.get(state, 0) + 1
3330             else:
3331                 state_str = str(state)
3332                 r = self.task_table.rowCount()
3333                 self.task_table.insertRow(r)
3334                 created_ts = t.get("created", 0)
3335                 created_str = time.strftime("%H:%M:%S", time.localtime(created_ts))
3336                 if created_ts else ""
3337                 for c, val in enumerate([t["id"], t["type"], state_str,
3338                                         str(t["priority"]), created_str, t.get("error", "") or ""]):
3339                     self.task_table.setItem(r, c, QTableWidgetItem(str(val)))
3340         all_states = [
3341             ("排队", TaskState.QUEUED), ("准备", TaskState.PREPARING),
3342             ("执行", TaskState.EXECUTING), ("暂停", TaskState.PAUSED),
3343             ("完成", TaskState.COMPLETED), ("失败", TaskState.FAILED),
3344             ("取消", TaskState.CANCELLED),
3345         ]
3346         parts = [f"{label}: {counts.get(state, 0)}" for label, state in all_states]
3347         self.stats_label.setText(" | ".join(parts))
3348
3349     def _log(self, tid: str, msg: str, level: str = "INFO"):
3350

```

```

3351         ts = time.strftime("%H:%M:%S")
3352         self._log_buffer.append(f"[{ts}][{level}][{tid}] {msg}")
3353         if level in ("ERROR", "FATAL"):
3354             self._error_buffer.append(f"[{ts}][{level}][{tid}] {msg}")
3355         # 保持缓冲区
3356         if len(self._log_buffer) > 200: self._log_buffer = self._log_buffer[-200:]
3357         self.log_text.setPlainText("\n".join(self._log_buffer[-50:]))
3358         self.error_text.setPlainText("\n".join(self._error_buffer[-20:]))
3359
3360     def _show_repair_guide(self):
3361         guides = {
3362             "模型加载异常": "1. 检查模型文件路径\n2. 验证模型格式兼容性\n3. 查看依
3363 赖库版本",
3364             "数据异常": "1. 检查数据格式是否正确\n2. 验证缺失值处理\n3. 确认编码格
3365 式",
3366             "内存不足": "1. 减少批次大小\n2. 降低输入维度\n3. 使用数据生成器",
3367         }
3368         msgs = "\n\n".join([f"【{k}】\n{v}" for k, v in guides.items()])
3369         QMessageBox.information(self, "异常修复指引", msgs)
3370
3371     def get_module_widget(user):
3372         return TaskManagerWidget(user)
3373
3374     # ==== modules\vision_3d.py (405 行) ====
3375     """3D 可视化模块 - 点云渲染、3D 模型展示、特征分布三维视图、多图层叠加（纯 QPainter
3376 安全渲染）"""
3377     from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
3378     QLabel,
3379     QGroupBox, QSlider, QComboBox, QSplitter, QMessageBox, QFileDialog)
3380     from PyQt6.QtCore import Qt, QTimer
3381     from PyQt6.QtGui import QPainter, QPen, QColor, QFont, QBrush, QPainterPath
3382     from core.auth import Role, OpAction, auth_engine
3383     from core.state_machine import task_sm, TaskState
3384     from core.rule_engine import rule_engine
3385     from core.algorithms import FeatureAnalyzer
3386     import numpy as np
3387     import math
3388
3389     class Safe3DView(QWidget):
3390         """纯 QPainter 实现的 3D 视图 - 无 OpenGL 依赖，无崩溃风险，支持旋转/缩放/平移
3391         """
3392         def __init__(self, parent=None):
3393             super().__init__(parent)
3394             self._points: np.ndarray = None
3395             self._edges: list = None
3396             self._rotation = [30.0, 45.0, 0.0]
3397             self._zoom = 1.0
3398             self._pan = [0.0, 0.0]
3399             self._last_pos = None
3400             self._auto_rotate = False

```

```

3401         self._point_size = 3
3402         self._bg_color = QColor(255, 248, 225)
3403         self.setMinimumSize(500, 380)
3404         self.setMouseTracking(True)
3405
3406     def set_points(self, points: np.ndarray, edges: list = None):
3407         if points is not None and len(points) > 0:
3408             self._points = np.array(points, dtype=np.float64)
3409         else:
3410             self._points = None
3411         self._edges = edges
3412         self.update()
3413
3414     def _project(self, x: float, y: float, z: float) -> tuple:
3415         rx, ry, rz = math.radians(self._rotation[0]),
3416         math.radians(self._rotation[1]), math.radians(self._rotation[2])
3417         # Rotate Y
3418         cos_b, sin_b = math.cos(ry), math.sin(ry)
3419         x1 = x * cos_b + z * sin_b
3420         z1 = -x * sin_b + z * cos_b
3421         # Rotate X
3422         cos_a, sin_a = math.cos(rx), math.sin(rx)
3423         y1 = y * cos_a - z1 * sin_a
3424         z2 = y * sin_a + z1 * cos_a
3425         # Rotate Z
3426         cos_c, sin_c = math.cos(rz), math.sin(rz)
3427         x2 = x1 * cos_c - y1 * sin_c
3428         y2 = x1 * sin_c + y1 * cos_c
3429         # Isometric project: x on screen diagonal, y vertical, z for depth
3430         scale = self._zoom * 45
3431         sx = self.width() / 2 + self._pan[0] + (x2 - y2) * scale * 0.55
3432         sy = self.height() / 2 + self._pan[1] - (x2 + y2) * scale * 0.3 - z2 * scale
3433         * 0.4
3434         return sx, sy, z2
3435
3436     def paintEvent(self, event):
3437         p = QPainter(self)
3438         p.setRenderHint(QPainter.RenderHint.Antialiasing)
3439         p.fillRect(self.rect(), self._bg_color)
3440
3441         p.setPen(QPen(QColor("#FFD54F"), 1))
3442         p.drawRoundedRect(self.rect().adjusted(1, 1, -1, -1), 6, 6)
3443
3444         if self._points is None or len(self._points) == 0:
3445             p.setPen(QPen(QColor("#795548")))
3446             p.setFont(QFont("Microsoft YaHei", 12))
3447             lines = ["3D 可视化视图", "", "左键拖拽 → 旋转", "右键拖拽
3448 → 平移",
3449 "滚轮 → 缩放", "", "点击左侧数据图层加载 3D 内容",
3450 "或点击「生成演示数据」按钮"]

```

```

3451         y_off = self.height() // 2 - 110
3452         for line in lines:
3453             p.drawText(0, y_off, self.width(), 24,
3454 Qt.AlignmentFlag.AlignCenter, line)
3455             y_off += 24
3456         p.end()
3457         return
3458
3459         points = self._points[:6000] if len(self._points) > 6000 else self._points
3460         # Sort by depth (z after rotation) for painter's algorithm
3461         projected = []
3462         for pt in points:
3463             if len(pt) >= 3:
3464                 sx, sy, sz = self._project(float(pt[0]), float(pt[1]),
3465 float(pt[2]))
3466                 projected.append((sx, sy, sz))
3467             elif len(pt) >= 2:
3468                 sx, sy, sz = self._project(float(pt[0]), float(pt[1]), 0)
3469                 projected.append((sx, sy, sz))
3470         projected.sort(key=lambda v: -v[2])
3471
3472         n = len(projected)
3473         for i, (sx, sy, sz) in enumerate(projected):
3474             if not (-100 < sx < self.width() + 100 and -100 < sy < self.height()
3475 + 100):
3476                 continue
3477                 ratio = i / max(1, n - 1)
3478                 r = int(80 + 175 * ratio)
3479                 g = int(180 - 120 * ratio)
3480                 b = int(200 + 55 * ratio)
3481                 depth_factor = 0.4 + 0.6 * ((sz + 3) / 6)
3482                 r = int(r * depth_factor)
3483                 g = int(g * depth_factor)
3484                 b = int(b * depth_factor)
3485                 color = QColor(max(30, min(255, r)), max(30, min(255, g)), max(30,
3486 min(255, b)))
3487                 p.setPen(QPen(color, 1.5))
3488                 size = max(1, int(self._point_size * self._zoom))
3489                 p.drawPoint(int(sx), int(sy))
3490
3491         if self._edges:
3492             p.setPen(QPen(QColor(255, 143, 0, 80), 1, Qt.PenStyle.DotLine))
3493             for e in self._edges[:500]:
3494                 if e[0] < len(points) and e[1] < len(points):
3495                     x1, y1, _ = self._project(float(points[e[0]][0]),
3496 float(points[e[0]][1]),
3497 float(points[e[0]][2]) if
3498 len(points[e[0]]) > 2 else 0)
3499                     x2, y2, _ = self._project(float(points[e[1]][0]),
3500 float(points[e[1]][1]),

```

```

3501                                     float(points[e[1]][2])           if
3502 len(points[e[1]]) > 2 else 0)
3503         p.drawLine(int(x1), int(y1), int(x2), int(y2))
3504
3505         p.setPen(QPen(QColor("#FF8F00"), 1))
3506         p.setFont(QFont("Microsoft YaHei", 9))
3507         p.drawText(10, 18, f" 缩放 : {self._zoom:.1f}x | 点 数 :
3508 {len(self._points)}")
3509         p.end()
3510
3511     def wheelEvent(self, event):
3512         self._zoom = max(0.15, min(6.0, self._zoom * (1.12 if
3513 event.angleDelta().y() > 0 else 0.88)))
3514         self.update()
3515
3516     def mousePressEvent(self, event):
3517         self._last_pos = event.position()
3518         self.setCursor(Qt.CursorShape.ClosedHandCursor)
3519
3520     def mouseMoveEvent(self, event):
3521         if self._last_pos is None:
3522             return
3523         dx = event.position().x() - self._last_pos.x()
3524         dy = event.position().y() - self._last_pos.y()
3525         if event.buttons() & Qt.MouseButton.LeftButton:
3526             self._rotation[0] += dy * 0.4
3527             self._rotation[1] += dx * 0.4
3528         elif event.buttons() & Qt.MouseButton.RightButton:
3529             self._pan[0] += dx
3530             self._pan[1] += dy
3531         self._last_pos = event.position()
3532         self.update()
3533
3534     def mouseReleaseEvent(self, event):
3535         self._last_pos = None
3536         self.setCursor(Qt.CursorShape.ArrowCursor)
3537
3538     def mouseDoubleClickEvent(self, event):
3539         self._rotation = [30.0, 45.0, 0.0]
3540         self._zoom = 1.0
3541         self._pan = [0.0, 0.0]
3542         self.update()
3543
3544     def start_auto_rotate(self, interval_ms: int = 45):
3545         if not hasattr(self, '_rot_timer'):
3546             self._rot_timer = QTimer(self)
3547             self._rot_timer.timeout.connect(self._auto_rotate_step)
3548             self._rot_timer.start(interval_ms)
3549
3550     def stop_auto_rotate(self):

```

```

3551         if hasattr(self, '_rot_timer'):
3552             self._rot_timer.stop()
3553
3554     def _auto_rotate_step(self):
3555         self._rotation[1] = (self._rotation[1] + 1.5) % 360
3556         self._rotation[0] += 0.1
3557         self.update()
3558
3559     class Vision3DWidget(QWidget):
3560         def __init__(self, user):
3561             super().__init__()
3562             self.user = user
3563             self._point_clouds = {}
3564             self._current_pc = None
3565             self._setup_ui()
3566             self._generate_demo_data()
3567
3568     def _setup_ui(self):
3569         layout = QVBoxLayout(self)
3570         layout.setSpacing(8)
3571
3572         header = QHBoxLayout()
3573         title = QLabel("    3D 数据可视化")
3574         title.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
3575         header.addWidget(title)
3576         header.addStretch()
3577         for label, handler in [("  导入点云", self._import_pointcloud),
3578                               ("  导入模型", self._import_model),
3579                               ("  生成演示数据", self._generate_demo_data),
3580                               ("  导出快照", self._export_snapshot)]:
3581             header.addWidget(QPushButton(label, clicked=handler))
3582         layout.addLayout(header)
3583
3584         main = QSplitter(Qt.Orientation.Horizontal)
3585
3586         left_panel = QVBoxLayout()
3587         left_panel.setAlignment(Qt.AlignmentFlag.AlignTop)
3588         left_panel.setSpacing(8)
3589
3590         data_gb = QGroupBox("    数据图层（点击切换）")
3591         dl = QVBoxLayout()
3592         from PyQt6.QtWidgets import QListWidget
3593         self.layer_list = QListWidget()
3594         self.layer_list.addItems([
3595             "    点云-演示球体（800 点）", "    点云-演示立方体（600 点）",
3596             "    点云-演示螺旋（1000 点）", "    3D 模型-四面体框架",
3597             "    特征分布-3D 散点映射", "    混沌吸引子-Lorenz 系统"
3598         ])
3599         self.layer_list.itemClicked.connect(self._on_layer_select)
3600         self.layer_list.setMaximumHeight(160)

```



```

3601         dl.addWidget(self.layer_list)
3602         data_gb.setLayout(dl)
3603         left_panel.addWidget(data_gb)
3604
3605         view_gb = QGroupBox(" 视图控制")
3606         vl = QVBoxLayout()
3607         vl.addWidget(QLabel("缩放:"))
3608         self.zoom_slider = QSlider(Qt.Orientation.Horizontal)
3609         self.zoom_slider.setRange(15, 300)
3610         self.zoom_slider.setValue(100)
3611         self.zoom_slider.valueChanged.connect(self._on_zoom)
3612         vl.addWidget(self.zoom_slider)
3613
3614         views = [(" 俯视图", [90, 0, 0]), (" 正视图", [0, 0, 0]),
3615                 (" 侧视图", [0, 90, 0]), (" 等轴测", [35, 45, 0]),
3616                 (" 透视角", [25, 35, 10])]
3617         for name, rot in views:
3618             vl.addWidget(QPushButton(name, clicked=lambda checked, r=rot:
3619 self._set_view(r)))
3620         view_gb.setLayout(vl)
3621         left_panel.addWidget(view_gb)
3622
3623         info_gb = QGroupBox("i 数据信息")
3624         self.info_label = QLabel("点数: --\n 范围: --\n 当前图层: --")
3625         self.info_label.setStyleSheet("color:#795548;padding:4px;")
3626         info_gb.setLayout(QVBoxLayout())
3627         info_gb.layout().addWidget(self.info_label)
3628         left_panel.addWidget(info_gb)
3629
3630         left_widget = QWidget()
3631         left_widget.setLayout(left_panel)
3632         left_widget.setMaximumWidth(240)
3633
3634         right_panel = QVBoxLayout()
3635         right_panel.setSpacing(6)
3636         self.view3d = Safe3DView()
3637         right_panel.addWidget(self.view3d)
3638
3639         op_bar = QHBoxLayout()
3640         op_btns = [
3641             (" 旋转动画", self._toggle_rotation),
3642             (" 特征分布 3D", self._show_feature_3d),
3643             (" 多图层叠加", self._overlay_layers),
3644             (" 批量处理", self._batch_process),
3645             (" 重置视角", lambda: setattr(self.view3d, '_rotation', [30, 45, 0])
3646 or
3647             setattr(self.view3d, '_zoom', 1.0) or setattr(self.view3d, '_pan', [0,
3648 0]) or self.view3d.update()),
3649         ]
3650         for label, handler in op_btns:

```

```

3651         op_bar.addWidget(QPushButton(label, clicked=handler))
3652     right_panel.addLayout(op_bar)
3653
3654     right_widget = QWidget()
3655     right_widget.setLayout(right_panel)
3656     main.addWidget(left_widget)
3657     main.addWidget(right_widget)
3658     main.setSizes([240, 900])
3659     layout.addWidget(main)
3660
3661     def _generate_demo_data(self):
3662         self._point_clouds["球体"] = self._gen_sphere(800)
3663         self._point_clouds["立方体"] = self._gen_cube(600)
3664         self._point_clouds["螺旋"] = self._gen_spiral(1000)
3665         self._point_clouds["四面体"] = self._gen_tetrahedron()
3666         self._point_clouds["Lorenz"] = self._gen_lorenz(1200)
3667         self._current_pc = self._point_clouds["螺旋"]
3668         self.view3d.set_points(self._current_pc)
3669         self.info_label.setText(f"点数: {len(self._current_pc)}\n 范围:  $[-2, 2]^3$ 
3670 \n 当前图层: 螺旋")
3671
3672     def _gen_sphere(self, n):
3673         phi = np.random.uniform(0, np.pi, n)
3674         theta = np.random.uniform(0, 2 * np.pi, n)
3675         r = np.cbrt(np.random.uniform(0, 8, n))
3676         return np.column_stack([r * np.sin(phi) * np.cos(theta),
3677                                 r * np.sin(phi) * np.sin(theta),
3678                                 r * np.cos(phi)])
3679
3680     def _gen_cube(self, n):
3681         pts = []
3682         side = int(n / 6)
3683         for _ in range(side):
3684             pts.append([np.random.uniform(-2, 2), np.random.uniform(-2, 2), -2])
3685             pts.append([np.random.uniform(-2, 2), np.random.uniform(-2, 2), 2])
3686             pts.append([np.random.uniform(-2, 2), -2, np.random.uniform(-2, 2)])
3687             pts.append([np.random.uniform(-2, 2), 2, np.random.uniform(-2, 2)])
3688             pts.append([-2, np.random.uniform(-2, 2), np.random.uniform(-2, 2)])
3689             pts.append([2, np.random.uniform(-2, 2), np.random.uniform(-2, 2)])
3690         return np.array(pts[:n])
3691
3692     def _gen_spiral(self, n):
3693         t = np.linspace(0, 4 * np.pi, n)
3694         r = t / max(t) * 2
3695         return np.column_stack([r * np.cos(t), r * np.sin(t), np.linspace(-2, 2,
3696 n)])
3697
3698     def _gen_tetrahedron(self):
3699         verts = np.array([[0, 2, 0], [1.8, -1, -1], [-1.8, -1, -1], [0, 0, 2.5]])
3700         pts = []

```

```

3701         for _ in range(500):
3702             a, b, c = np.random.rand(3)
3703             if a + b + c > 1:
3704                 a, b, c = 1 - a, 1 - b, 1 - c
3705             pt = a * verts[0] + b * verts[1] + c * verts[2] + (1 - a - b - c) * verts[3]
3706             pts.append(pt)
3707         pts_array = np.array(pts)
3708         edges = [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
3709         self._tetra_edges = edges
3710         return pts_array
3711
3712     def _gen_lorenz(self, n):
3713         dt = 0.01
3714         pts = np.zeros((n, 3))
3715         x, y, z = 0.1, 0.1, 0.1
3716         sigma, rho, beta = 10.0, 28.0, 8.0 / 3.0
3717         for i in range(n):
3718             dx = sigma * (y - x) * dt
3719             dy = (x * (rho - z) - y) * dt
3720             dz = (x * y - beta * z) * dt
3721             x += dx; y += dy; z += dz
3722             pts[i] = [x / 10, y / 10, z / 15]
3723         return pts
3724
3725     def _on_layer_select(self, item):
3726         text = item.text()
3727         key_map = {"球体": "球体", "立方体": "立方体", "螺旋": "螺旋",
3728                   "四面体": "四面体", "Lorenz": "Lorenz"}
3729         for kw, key in key_map.items():
3730             if kw in text:
3731                 if key in self._point_clouds:
3732                     self._current_pc = self._point_clouds[key]
3733                     edges = self._tetra_edges if key == "四面体" else None
3734                     self.view3d.set_points(self._current_pc, edges)
3735                     self.info_label.setText(f"点数: {len(self._current_pc)}\n当
3736 前图层: {key}")
3737                 return
3738         self._show_feature_3d()
3739
3740     def _on_zoom(self, val):
3741         self.view3d._zoom = val / 100.0
3742         self.view3d.update()
3743
3744     def _set_view(self, rot):
3745         self.view3d._rotation = list(rot)
3746         self.view3d.update()
3747
3748     def _toggle_rotation(self):
3749         if not hasattr(self.view3d, '_rot_timer') or not
3750 self.view3d._rot_timer.isActive():

```

```

3751         self.view3d.start_auto_rotate(45)
3752         QMessageBox.information(self, "旋转动画", "3D 视图旋转中（再次点击停
3753 止）")
3754     else:
3755         self.view3d.stop_auto_rotate()
3756         QMessageBox.information(self, "旋转动画", "旋转已停止")
3757
3758     def _show_feature_3d(self):
3759         features = np.random.randn(300, 3) * np.array([2.5, 1.5, 3.0]) +
3760 np.array([0, 0, 1.5])
3761         self._current_pc = features
3762         self.view3d.set_points(features)
3763         self.info_label.setText("特征分布 3D 映射\n 维度: 3 | 样本: 300\n 各维度随
3764 机生成")
3765
3766     def _overlay_layers(self):
3767         if len(self._point_clouds) >= 2:
3768             keys = list(self._point_clouds.keys())
3769             all_points = np.vstack([self._point_clouds[keys[0]][:400],
3770                                     self._point_clouds[keys[1]][:400]])
3771             np.random.shuffle(all_points)
3772             self.view3d.set_points(all_points[:800])
3773             self.info_label.setText(f"多图层叠加\n {keys[0]} + {keys[1]}\n 总点数:
3774 {min(800, len(all_points))}")
3775
3776     def _batch_process(self):
3777         QMessageBox.information(self, "批量处理",
3778             "3D 数据批量处理功能\n 支持批量导入点云文件并执行统一 AI 流程\n\n 支持
3779 格式: PLY/XYZ/PCD/TXT\n 任务将加入任务管控队列监控")
3780
3781     def _import_pointcloud(self):
3782         path, _ = QFileDialog.getOpenFileName(self, "导入点云", "",
3783             "点云文件 (*.ply *.xyz *.pcd *.txt *.csv);;所有文件 (*.*)")
3784         if path:
3785             try:
3786                 data = np.loadtxt(path, delimiter=None, usecols=(0, 1, 2),
3787 max_rows=5000)
3788                 if data.shape[1] >= 3:
3789                     self._current_pc = data
3790                     self.view3d.set_points(data)
3791                     self.info_label.setText(f"导入文件: {path.split('/')[-1]}\n
3792 点数: {len(data)}")
3793             except Exception as e:
3794                 QMessageBox.warning(self, "导入失败", f"无法解析文
3795 件:\n{str(e)}\n\n 请确保文件包含至少 3 列数值数据")
3796
3797     def _import_model(self):
3798         QMessageBox.information(self, "导入 3D 模型",
3799             "支持格式: OBJ / STL / GLTF\n 导入后将自动渲染模型顶点与面结构\n\n
3800 请将模型文件放入项目资源目录后导入")

```

```

3801
3802     def _export_snapshot(self):
3803         path, _ = QFileDialog.getSaveFileName(self, " 导 出 快 照 ",
3804 "3d_snapshot.png",
3805         "PNG 图片 (*.png);;所有文件 (*.*)")
3806         if path:
3807             pixmap = self.view3d.grab()
3808             pixmap.save(path)
3809             QMessageBox.information(self, " 导 出 ", f"3D 视 图 快 照 已 保 存
3810 至:\n{path}")
3811
3812     def get_module_widget(user):
3813         return Vision3DWidget(user)
3814
3815     # ==== modules\vision_lab.py (278 行) ====
3816     """视觉实验室(OpenCV) - 图像预处理/滤波/边缘/分割/阈值/形态学 + 视觉推理"""
3817     from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
3818     QLabel,
3819     QGroupBox, QGridLayout, QSlider, QComboBox, QFileDialog, QScrollArea,
3820     QSplitter, QMessageBox)
3821     from PyQt6.QtCore import Qt, QTimer
3822     from PyQt6.QtGui import QPixmap, QImage, QPainter, QPen, QColor
3823     from core.auth import Role, OpAction, auth_engine
3824     from core.rule_engine import rule_engine, TriggerType, Condition, CompareOp,
3825     RuleAction
3826     from core.state_machine import task_sm, TaskState
3827     import numpy as np
3828     import cv2
3829
3830     class ImageViewer(QWidget):
3831         """可滚轮缩放的图像查看器"""
3832         def __init__(self, parent=None):
3833             super().__init__(parent)
3834             self._pixmap = None; self._scale = 1.0; self._offset_x = 0; self._offset_y
3835             = 0
3836             self.setMinimumSize(400, 300); self.setMouseTracking(True)
3837
3838         def set_image(self, img_array):
3839             if img_array is None: return
3840             if len(img_array.shape) == 2:
3841                 h, w = img_array.shape
3842                 qimg = QImage(img_array.data, w, h, w,
3843 QImage.Format.Format_Grayscale8)
3844             else:
3845                 h, w, ch = img_array.shape
3846                 qimg = QImage(img_array.data, w, h, w * ch,
3847 QImage.Format.Format_BGR888)
3848             self._pixmap = QPixmap.fromImage(qimg); self.update()
3849
3850         def wheelEvent(self, event):

```

```

3851         if self._pixmap:
3852             factor = 1.1 if event.angleDelta().y() > 0 else 0.9
3853             self._scale = max(0.1, min(5.0, self._scale * factor))
3854             self.update()
3855
3856     def paintEvent(self, event):
3857         from PyQt6.QtGui import QPainter
3858         p = QPainter(self)
3859         p.fillRect(self.rect(), QColor(255, 248, 225))
3860         if self._pixmap:
3861             scaled = self._pixmap.scaled(self._pixmap.size() * self._scale,
3862             Qt.AspectRatioMode.KeepAspectRatio,
3863             Qt.TransformationMode.SmoothTransformation)
3864             x = (self.width() - scaled.width()) // 2 + int(self._offset_x)
3865             y = (self.height() - scaled.height()) // 2 + int(self._offset_y)
3866             p.drawPixmap(x, y, scaled)
3867             p.setPen(QPen(QColor("#FF8F00"), 1))
3868             p.drawText(10, 20, f"缩放: {self._scale:.1f}x")
3869         else:
3870             p.setPen(QPen(QColor("#795548")))
3871             p.drawText(self.rect(), Qt.AlignmentFlag.AlignCenter, "请加载图像或
3872 生成演示图像")
3873
3874     class VisionLabWidget(QWidget):
3875         def __init__(self, user):
3876             super().__init__()
3877             self.user = user
3878             self._original_img = None
3879             self._processed_img = None
3880             self._setup_ui()
3881             self._init_demo_image()
3882             self._init_rules()
3883
3884         def _setup_ui(self):
3885             layout = QVBoxLayout(self)
3886
3887             header = QHBoxLayout()
3888             title = QLabel(" 视觉实验室 (OpenCV)")
3889             title.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
3890             header.addWidget(title); header.addStretch()
3891             header.addWidget(QPushButton(" 加载图片", clicked=self._load_image))
3892             header.addWidget(QPushButton(" 加载视频", clicked=self._load_video))
3893             header.addWidget(QPushButton("      摄      像      头      ",
3894 clicked=self._capture_camera))
3895             header.addWidget(QPushButton(" 重置", clicked=self._reset_image))
3896             header.addWidget(QPushButton("      导      出      结      果      ",
3897 clicked=self._export_result))
3898             layout.addLayout(header)
3899
3900             main = QSplitter(Qt.Orientation.Horizontal)

```

```

3901
3902     left_panel = QVBoxLayout()
3903     left_panel.setAlignment(Qt.AlignmentFlag.AlignTop)
3904
3905     ops_gb = QGroupBox("    图像处理操作(可自定义参数)")
3906     og = QGridLayout()
3907     operations = [
3908         ("灰度化", "gray"), ("高斯模糊", "blur"), ("边缘检测 (Canny)",
3909 "canny"),
3910         ("阈值分割", "threshold"), ("形态学膨胀", "dilate"), ("形态学腐蚀",
3911 "erode"),
3912         ("Sobel 梯度", "sobel"), ("直方图均衡", "equalize"),
3913     ]
3914     self._op_btns = {}
3915     self._op_sliders = {}
3916     for i, (name, key) in enumerate(operations):
3917         btn = QPushButton(name); btn.setCheckable(True)
3918         self._op_btns[key] = btn; og.addWidget(btn, i, 0)
3919         slider = QSlider(Qt.Orientation.Horizontal); slider.setRange(1, 200);
3920         slider.setValue(100)
3921         self._op_sliders[key] = slider; og.addWidget(slider, i, 1)
3922     ops_gb.setLayout(og)
3923     left_panel.addWidget(ops_gb)
3924
3925     apply_btn    =    QPushButton("    应    用    处    理    链    ");
3926     apply_btn.setObjectName("primary")
3927     apply_btn.clicked.connect(self._apply_processing_chain)
3928     left_panel.addWidget(apply_btn)
3929
3930     inference_gb = QGroupBox("    AI 推理(模拟)")
3931     il = QVBoxLayout()
3932     il.addWidget(QPushButton("    目    标    检    测    推    理    ",
3933 clicked=self._run_detection))
3934     il.addWidget(QPushButton("    图    像    分    类    推    理    ",
3935 clicked=self._run_classification))
3936     il.addWidget(QPushButton("    语    义    分    割    推    理    ",
3937 clicked=self._run_segmentation))
3938     inference_gb.setLayout(il)
3939     left_panel.addWidget(inference_gb)
3940
3941     stats_gb = QGroupBox("    图像统计")
3942     self.stats_label = QLabel("分辨率: --\n 通道数: --\n 像素范围: --")
3943     self.stats_label.setStyleSheet("color:#795548;")
3944     stats_gb.setLayout(QVBoxLayout());
3945     stats_gb.layout().addWidget(self.stats_label)
3946     left_panel.addWidget(stats_gb)
3947
3948     left_widget    =    QWidget();    left_widget.setLayout(left_panel);
3949     left_widget.setMaximumWidth(240)
3950

```

```

3951         right_panel = QVBoxLayout()
3952         img_row = QHBoxLayout()
3953         self.original_view = ImageViewer()
3954         self.processed_view = ImageViewer()
3955         img_row.addWidget(QLabel("      原      始      图      像      ",
3956 alignment=Qt.AlignmentFlag.AlignCenter, styleSheet="color:#E65100;"))
3957         img_row.addWidget(QLabel("      处      理      后      图      像      ",
3958 alignment=Qt.AlignmentFlag.AlignCenter, styleSheet="color:#2E7D32;"))
3959         right_panel.addLayout(img_row)
3960
3961         viewer_row = QHBoxLayout()
3962         viewer_row.addWidget(self.original_view);
3963         viewer_row.addWidget(self.processed_view)
3964         right_panel.addLayout(viewer_row)
3965
3966         right_widget = QWidget(); right_widget.setLayout(right_panel)
3967         main.addWidget(left_widget); main.addWidget(right_widget)
3968         main.setSizes([240, 900])
3969         layout.addWidget(main)
3970
3971     def _init_demo_image(self):
3972         img = np.zeros((400, 600, 3), dtype=np.uint8)
3973         img[:] = (30, 30, 50)
3974         # 绘制模拟图像内容
3975         cv2.rectangle(img, (50, 50), (200, 200), (0, 100, 200), -1)
3976         cv2.rectangle(img, (250, 100), (400, 250), (0, 150, 100), -1)
3977         cv2.circle(img, (450, 150), 60, (200, 100, 0), -1)
3978         cv2.circle(img, (150, 300), 40, (150, 50, 150), -1)
3979         cv2.putText(img, " 低 代 码  AI  平 台  Demo", (200, 350),
3980 cv2.FONT_HERSHEY_SIMPLEX, 1.2, (200, 200, 255), 2)
3981         cv2.line(img, (0, 200), (600, 200), (100, 100, 100), 2)
3982         cv2.line(img, (300, 0), (300, 400), (100, 100, 100), 2)
3983         self._original_img = img
3984         self.original_view.set_image(img)
3985         self._update_stats(img)
3986
3987     def _apply_processing_chain(self):
3988         if self._original_img is None:
3989             QMessageBox.warning(self, "提示", "请先加载图像或使用演示图像");
3990         return
3991         img = self._original_img.copy()
3992         for key, btn in self._op_btns.items():
3993             if btn.isChecked():
3994                 param = self._op_sliders[key].value() / 100.0
3995                 img = self._process_single(img, key, param)
3996         self._processed_img = img
3997         self.processed_view.set_image(img)
3998         self._update_stats(img)
3999
4000     def _process_single(self, img, op_key: str, param: float):

```



```

4001         try:
4002             if op_key == "gray":
4003                 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
4004                 return cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
4005             if op_key == "blur":
4006                 ksize = max(3, int(param * 20))
4007                 if ksize % 2 == 0: ksize += 1
4008                 return cv2.GaussianBlur(img, (ksize, ksize), 0)
4009             if op_key == "canny":
4010                 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
4011                 low = int(50 * param); high = int(150 * param)
4012                 edges = cv2.Canny(gray, low, high)
4013                 return cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
4014             if op_key == "threshold":
4015                 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
4016                 thresh_val = int(127 * param)
4017                 _, thresh = cv2.threshold(gray, thresh_val, 255,
4018 cv2.THRESH_BINARY)
4019                 return cv2.cvtColor(thresh, cv2.COLOR_GRAY2BGR)
4020             if op_key == "dilate":
4021                 kernel = np.ones((3, 3), np.uint8)
4022                 return cv2.dilate(img, kernel, iterations=max(1, int(param * 3)))
4023             if op_key == "erode":
4024                 kernel = np.ones((3, 3), np.uint8)
4025                 return cv2.erode(img, kernel, iterations=max(1, int(param * 3)))
4026             if op_key == "sobel":
4027                 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
4028                 sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
4029                 sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
4030                 mag = cv2.magnitude(sx, sy)
4031                 mag = np.uint8(np.clip(mag * param, 0, 255))
4032                 return cv2.cvtColor(mag, cv2.COLOR_GRAY2BGR)
4033             if op_key == "equalize":
4034                 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
4035                 eq = cv2.equalizeHist(gray)
4036                 return cv2.cvtColor(eq, cv2.COLOR_GRAY2BGR)
4037         except Exception as e:
4038             QMessageBox.warning(self, "处理异常", str(e))
4039         return img
4040
4041     def _run_detection(self):
4042         if self._original_img is None: return
4043         img = self._original_img.copy()
4044         cv2.rectangle(img, (50, 50), (200, 200), (0, 255, 0), 3)
4045         cv2.putText(img, "Object:0.92", (55, 45), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
4046 (0, 255, 0), 2)
4047         cv2.rectangle(img, (250, 100), (400, 250), (255, 0, 0), 3)
4048         cv2.putText(img, "Object:0.87", (255, 95), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
4049 (255, 0, 0), 2)
4050         self.processed_view.set_image(img)

```

```

4051         self._processed_img = img
4052
4053     def _run_classification(self):
4054         if self._original_img is None: return
4055         img = self._original_img.copy()
4056         cv2.putText(img, "Class: vehicle (93.2%)", (10, 30),
4057 cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
4058         cv2.putText(img, "Class: building (5.1%)", (10, 60),
4059 cv2.FONT_HERSHEY_SIMPLEX, 0.8, (100, 200, 255), 2)
4060         self.processed_view.set_image(img); self._processed_img = img
4061
4062     def _run_segmentation(self):
4063         if self._original_img is None: return
4064         img = self._original_img.copy()
4065         mask = np.zeros(img.shape[:2], np.uint8)
4066         cv2.circle(mask, (150, 150), 90, 128, -1)
4067         cv2.circle(mask, (450, 150), 80, 200, -1)
4068         overlay = img.copy()
4069         overlay[mask == 128] = (0, 255, 0); overlay[mask == 200] = (255, 0, 0)
4070         img = cv2.addWeighted(img, 0.5, overlay, 0.5, 0)
4071         self.processed_view.set_image(img); self._processed_img = img
4072
4073     def _load_image(self):
4074         path, _ = QFileDialog.getOpenFileName(self, "选择图片", "",
4075 "图像文件 (*.png *.jpg *.bmp *.tiff *.webp);;所有文件 (*.*)")
4076         if path:
4077             img = cv2.imread(path)
4078             if img is not None:
4079                 self._original_img = img; self.original_view.set_image(img);
4080 self._update_stats(img)
4081
4082     def _load_video(self):
4083         path, _ = QFileDialog.getOpenFileName(self, "选择视频", "",
4084 "视频文件 (*.mp4 *.avi *.mov *.mkv);;所有文件 (*.*)")
4085         if path:
4086             cap = cv2.VideoCapture(path)
4087             ret, frame = cap.read(); cap.release()
4088             if ret:
4089                 self._original_img = frame; self.original_view.set_image(frame)
4090                 self._update_stats(frame)
4091                 QMessageBox.information(self, "视频加载", f"已加载视频首帧\n 可
4092 使用视觉推理进行分析")
4093
4094     def _capture_camera(self):
4095         cap = cv2.VideoCapture(0)
4096         ret, frame = cap.read(); cap.release()
4097         if ret:
4098             self._original_img = frame; self.original_view.set_image(frame)
4099             self._update_stats(frame)
4100

```

```

4101     def _reset_image(self):
4102         self._init_demo_image(); self._processed_img = None
4103         self.processed_view._pixmap = None; self.processed_view.update()
4104
4105     def _export_result(self):
4106         if self._processed_img is not None:
4107             path, _ = QFileDialog.getSaveFileName(self, "导出结果", "result.png",
4108 "PNG (*.png);;JPG (*.jpg)")
4109             if path: cv2.imwrite(path, self._processed_img)
4110         else:
4111             QMessageBox.warning(self, "提示", "请先处理图像")
4112
4113     def _update_stats(self, img):
4114         h, w = img.shape[:2]
4115         ch = 1 if len(img.shape) == 2 else img.shape[2]
4116         self.stats_label.setText(f"分辨率: {w} × {h} \n 通道数: {ch} \n 像素范围:
4117 [{img.min()}, {img.max()}]")
4118
4119     def _init_rules(self):
4120         rule_engine.create_rule("图像尺寸检查", TriggerType.DATA,
4121             [Condition(field="width", op=CompareOp.GT, value=32),
4122              Condition(field="height", op=CompareOp.GT, value=32)],
4123             [RuleAction(action_type="alert", params={"msg": "图像尺寸合法"})],
4124             priority=5, group="视觉处理")
4125
4126     def get_module_widget(user):
4127         return VisionLabWidget(user)
4128
4129     # ==== ui\__init__.py (1 行) ====
4130     # 低代码 AI 平台 界面层
4131
4132     # ==== ui\canvas.py (334 行) ====
4133     """可视化编排画布 - 拖拽节点、连线、分组、对齐布局"""
4134     from PyQt6.QtWidgets import (QWidget, QGraphicsView, QGraphicsScene,
4135     QGraphicsItem,
4136     QGraphicsRectItem, QGraphicsLineItem, QGraphicsTextItem,
4137     QMenu, QInputDialog, QVBoxLayout, QHBoxLayout, QPushButton, QToolBar)
4138     from PyQt6.QtCore import Qt, QRectF, QPointF, pyqtSignal, QLineF
4139     from PyQt6.QtGui import QPainter, QPen, QBrush, QColor,
4140     QFont, QLinearGradient, QCursor, QAction)
4141     from dataclasses import dataclass, field
4142     from typing import Dict, List, Optional, Tuple
4143     import uuid, math
4144
4145     NODE_COLORS = {
4146         "数据预处理": ("#1565c0", "#1e88e5"),
4147         "传统机器学习": ("#00838f", "#00acc1"),
4148         "深度学习": ("#6a1b9a", "#8e24aa"),
4149         "推理": ("#e65100", "#fb8c00"),
4150         "分支判断": ("#2e7d32", "#43a047"),

```

```

4151     "循环": ("c62828", "e53935"),
4152     "数据输出": ("37474f", "546e7a"),
4153 }
4154
4155 @dataclass
4156 class CanvasNode:
4157     nid: str; node_type: str; label: str; x: float = 0; y: float = 0
4158     w: float = 140; h: float = 70; config: dict = field(default_factory=dict)
4159
4160 @dataclass
4161 class CanvasEdge:
4162     eid: str; source: str; target: str; label: str = ""
4163
4164 class AINodeItem(QGraphicsRectItem):
4165     def __init__(self, node: CanvasNode, parent=None):
4166         super().__init__(0, 0, node.w, node.h, parent)
4167         self.node = node; self.nid = node.nid
4168         self.setFlags(QGraphicsItem.GraphicsItemFlag.ItemIsMovable |
4169                     QGraphicsItem.GraphicsItemFlag.ItemIsSelectable |
4170                     QGraphicsItem.GraphicsItemFlag.ItemSendsGeometryChanges)
4171         self.setAcceptHoverEvents(True)
4172         colors = NODE_COLORS.get(node.node_type, ("555", "777"))
4173         self._bg_color = QColor(colors[0]); self._hover_color = QColor(colors[1])
4174         self.setPos(node.x, node.y); self.setZValue(10)
4175
4176     def paint(self, painter, option, widget=None):
4177         painter.setRenderHint(QPainter.RenderHint.Antialiasing)
4178         rect = self.rect(); is_sel = self.isSelected()
4179         gradient = QLinearGradient(rect.topLeft(), rect.bottomRight())
4180         gradient.setColorAt(0, self._hover_color if is_sel else self._bg_color)
4181         gradient.setColorAt(1, QColor(self._bg_color.red()//2,
4182 self._bg_color.green()//2, self._bg_color.blue()//2))
4183         painter.setBrush(QBrush(gradient))
4184         painter.setPen(QPen(QColor("FF8F00") if is_sel else QColor("FFD54F"), 3
4185 if is_sel else 1.5))
4186         painter.drawRoundedRect(rect, 10, 10)
4187         painter.setPen(QPen(QColor("white")))
4188         painter.setFont(QFont("Microsoft YaHei", 11, QFont.Weight.Bold))
4189         painter.drawText(QRectF(4, 8, rect.width()-8, 22),
4190 Qt.AlignmentFlag.AlignCenter, self.node.label)
4191         painter.setPen(QPen(QColor("E0E0E0")))
4192         painter.setFont(QFont("Microsoft YaHei", 9))
4193         painter.drawText(QRectF(4, 30, rect.width()-8, 18),
4194 Qt.AlignmentFlag.AlignCenter, self.node.node_type)
4195
4196     def itemChange(self, change, value):
4197         if change == QGraphicsItem.GraphicsItemChange.ItemPositionHasChanged:
4198             view = self.scene().views()[0] if self.scene() and self.scene().views()
4199         else None
4200             if view: view._on_node_moved()

```

```

4201         return super().itemChange(change, value)
4202
4203     def mouseDoubleClickEvent(self, event):
4204         text, ok = QDialog.getText(None, "编辑节点", "节点标签:",
4205 text=self.node.label)
4206         if ok and text: self.node.label = text; self.update()
4207
4208     def contextMenuEvent(self, event):
4209         view = self.scene().views()[0] if self.scene() and self.scene().views()
4210 else None
4211         menu = QMenu()
4212         menu.addAction("📄 编辑标签", lambda: self.mouseDoubleClickEvent(None))
4213         if view:
4214             menu.addAction("复制节点", lambda: view._copy_node_safe(self))
4215             menu.addAction("删除节点", lambda:
4216 view._remove_node_safe(self.nid))
4217             menu.addSeparator()
4218             menu.addAction("从此节点连线", lambda:
4219 view._start_connect_from(self))
4220             menu.addSeparator()
4221             menu.addAction("↑ 置顶", lambda: self.setZValue(self.zValue() + 100))
4222             menu.exec(event.screenPos())
4223
4224     class EdgeItem(QGraphicsLineItem):
4225         def __init__(self, source: ANodeItem, target: ANodeItem, edge: CanvasEdge):
4226             super().__init__()
4227             self._src = source; self._tgt = target; self.edge = edge
4228             self.setPen(QPen(QColor("#FF8F00"), 2.5, Qt.PenStyle.SolidLine))
4229             self.setZValue(5)
4230             self.setFlag(QGraphicsItem.GraphicsItemFlag.ItemIsSelectable, True)
4231             self._update_line()
4232
4233         def _update_line(self):
4234             try:
4235                 sr = self._src.sceneBoundingRect()
4236                 tr = self._tgt.sceneBoundingRect()
4237                 self.setLine(QLineF(sr.center().x(), sr.bottom(),
4238                                     tr.center().x(), tr.top()))
4239             except (RuntimeError, AttributeError):
4240                 pass
4241
4242     def contextMenuEvent(self, event):
4243         view = self.scene().views()[0] if self.scene() and self.scene().views()
4244 else None
4245         menu = QMenu()
4246         if view:
4247             menu.addAction("删除连线", lambda: view._delete_edge_safe(self))
4248             menu.exec(event.screenPos())
4249
4250     class ModelCanvas(QGraphicsView):

```

```

4251     flow_changed = pyqtSignal()
4252     status_message = pyqtSignal(str)
4253
4254     def __init__(self, parent=None):
4255         super().__init__(parent)
4256         self._scene = QGraphicsScene(self)
4257         self._scene.setSceneRect(-2000, -1500, 5000, 4000)
4258         self.setScene(self._scene)
4259         self.setRenderHint(QPainter.RenderHint.Antialiasing)
4260         self.setDragMode(QGraphicsView.DragMode.NoDrag)
4261
4262     self.setTransformationAnchor(QGraphicsView.ViewportAnchor.AnchorUnderMouse)
4263     self.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOn)
4264     self.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOn)
4265     self._nodes: Dict[str, AINodeItem] = {}
4266     self._edges: Dict[str, EdgeItem] = {}
4267     self._connect_mode = False; self._connect_source: Optional[AINodeItem] =
4268     None
4269     self._undo_stack: List[dict] = []; self._redo_stack: List[dict] = []
4270     self._drag_pos = None
4271     self.setMouseTracking(True)
4272
4273     def add_node(self, node_type: str, label: str, x: float = 0, y: float = 0) ->
4274     str:
4275         import random as _r
4276         node = CanvasNode(nid=uuid.uuid4().hex[:8], node_type=node_type,
4277         label=label,
4278             x=x or _r.randint(0, 400), y=y or _r.randint(0, 300))
4279         item = AINodeItem(node); self._scene.addItem(item)
4280         self._nodes[node.nid] = item
4281         self._push_undo("add", node.nid)
4282         self.flow_changed.emit()
4283         return node.nid
4284
4285     def _remove_node_safe(self, nid: str):
4286         try:
4287             item = self._nodes.get(nid)
4288             if not item: return
4289             for eid in list(self._edges.keys()):
4290                 e = self._edges[eid]
4291                 if e._src.nid == nid or e._tgt.nid == nid:
4292                     self._scene.removeItem(e); del self._edges[eid]
4293             self._push_undo("del", nid)
4294             self._scene.removeItem(item); del self._nodes[nid]
4295             self.flow_changed.emit()
4296         except Exception as e:
4297             self.status_message.emit(f"删除失败: {e}")
4298
4299     def _start_connect_from(self, source_item: AINodeItem):
4300         self._connect_mode = True; self._connect_source = source_item

```

```

4301         source_item.setSelected(True)
4302         self.setCursor(Qt.CursorShape.CrossCursor)
4303         self.status_message.emit(f"  连线模式：请点击目标节点（源：
4304 {source_item.node.label}）")
4305
4306     def connect_nodes(self):
4307         try:
4308             selected = [it for it in self._scene.selectedItems() if isinstance(it,
4309 AINodeItem)]
4310             if len(selected) != 2:
4311                 self.status_message.emit(f"⚠ 请先选中 2 个节点再连线（当前选中
4312 {len(selected)}个）| 方法：Ctrl+点击选 2 个节点 → 点连线按钮")
4313                 return
4314                 self._make_edge(selected[0], selected[1])
4315         except Exception as e:
4316             self.status_message.emit(f"连线失败：{e}")
4317
4318     def _make_edge(self, src: AINodeItem, tgt: AINodeItem):
4319         if src.nid == tgt.nid:
4320             self.status_message.emit("⚠ 不能连接自身"); return
4321         for e in self._edges.values():
4322             if e._src.nid == src.nid and e._tgt.nid == tgt.nid:
4323                 self.status_message.emit("⚠ 这两个节点已有连线"); return
4324         eid = uuid.uuid4().hex[:8]
4325         edge = CanvasEdge(eid=eid, source=src.nid, target=tgt.nid)
4326         item = EdgeItem(src, tgt, edge)
4327         self._scene.addItem(item); self._edges[eid] = item
4328         self._connect_mode = False; self._connect_source = None
4329         self.setCursor(Qt.CursorShape.ArrowCursor)
4330         src.setSelected(False); tgt.setSelected(False)
4331         self.status_message.emit(f"✔ 连线成功：{src.node.label} →
4332 {tgt.node.label}")
4333         self.flow_changed.emit()
4334
4335     def _delete_edge_safe(self, edge_item: EdgeItem):
4336         try:
4337             eid = edge_item.edge.eid
4338             if eid in self._edges:
4339                 self._scene.removeItem(edge_item); del self._edges[eid]
4340                 self.status_message.emit("  连线已删除")
4341                 self.flow_changed.emit()
4342         except Exception as e:
4343             self.status_message.emit(f"删除连线失败：{e}")
4344
4345     def _on_node_moved(self):
4346         for e in self._edges.values():
4347             try: e._update_line()
4348             except: pass
4349
4350     def _copy_node_safe(self, item: AINodeItem):

```

```

4351         self.add_node(item.node.node_type, item.node.label + "_副本",
4352                        item.node.x + 50, item.node.y + 50)
4353
4354     def auto_layout(self):
4355         try:
4356             from core.algorithms import ForceDirectedLayout, LayoutNode
4357             nl = [LayoutNode(nid=n.nid, x=n.node.x, y=n.node.y) for n in
4358 self._nodes.values()]
4359             el = [(e._src.nid, e._tgt.nid) for e in self._edges.values()]
4360             result = ForceDirectedLayout(width=2000, height=1500).layout(nl, el)
4361             for ln in result:
4362                 if ln.nid in self._nodes:
4363                     self._nodes[ln.nid].setPos(ln.x, ln.y)
4364             for e in self._edges.values(): e._update_line()
4365             self.flow_changed.emit()
4366         except Exception as e:
4367             self.status_message.emit(f"布局失败: {e}")
4368
4369     def align_nodes(self, mode: str = "hcenter"):
4370         selected = [it for it in self._scene.selectedItems() if isinstance(it,
4371 AINodeItem)]
4372         if len(selected) < 2:
4373             self.status_message.emit("⚠ 请至少选中 2 个节点")
4374             return
4375         if mode == "hcenter":
4376             avg = sum(it.x() + it.rect().width()/2 for it in selected) /
4377 len(selected)
4378             for it in selected: it.setX(avg - it.rect().width()/2)
4379         elif mode == "vcenter":
4380             avg = sum(it.y() + it.rect().height()/2 for it in selected) /
4381 len(selected)
4382             for it in selected: it.setY(avg - it.rect().height()/2)
4383         elif mode == "grid":
4384             cols = math.ceil(math.sqrt(len(selected)))
4385             for i, it in enumerate(sorted(selected, key=lambda x: x.y())):
4386                 it.setPos((i % cols) * 180, (i // cols) * 100)
4387             for e in self._edges.values(): e._update_line()
4388             self.status_message.emit(f"✔ {len(selected)} 个节点已对齐")
4389             self.flow_changed.emit()
4390
4391     def clear_all(self):
4392         for e in list(self._edges.values()): self._scene.removeItem(e)
4393         for n in list(self._nodes.values()): self._scene.removeItem(n)
4394         self._nodes.clear(); self._edges.clear()
4395         self.flow_changed.emit()
4396
4397     def get_flow_data(self) -> dict:
4398         nodes = [{"id": n.nid, "type": n.node.node_type, "label": n.node.label,
4399                  "x": n.node.x, "y": n.node.y} for n in self._nodes.values()]
4400         edges = [{"id": e.edge.eid, "source": e._src.nid, "target": e._tgt.nid}

```



```

4401         for e in self._edges.values():
4402             return {"nodes": nodes, "edges": edges}
4403
4404     def load_flow_data(self, data: dict):
4405         self.clear_all()
4406         for nd in data.get("nodes", []):
4407             node = CanvasNode(nid=nd["id"], node_type=nd["type"],
4408 label=nd["label"],
4409                             x=nd["x"], y=nd["y"])
4410             item = AINodeItem(node); self._scene.addItem(item)
4411             self._nodes[node.nid] = item
4412         for ed in data.get("edges", []):
4413             src = self._nodes.get(ed["source"]); tgt =
4414 self._nodes.get(ed["target"])
4415             if src and tgt:
4416                 edge = CanvasEdge(eid=ed["id"], source=ed["source"],
4417 target=ed["target"])
4418                 item = EdgeItem(src, tgt, edge)
4419                 self._scene.addItem(item); self._edges[edge.eid] = item
4420
4421     def mousePressEvent(self, event):
4422         if event.button() == Qt.MouseButton.MiddleButton:
4423             self._drag_pos = event.position().toPoint()
4424             self.setCursor(Qt.CursorShape.ClosedHandCursor); return
4425         if event.button() == Qt.MouseButton.LeftButton:
4426             pt = event.position().toPoint()
4427             item = self.itemAt(pt)
4428             if self._connect_mode and isinstance(item, AINodeItem):
4429                 if item is not self._connect_source:
4430                     try: self._make_edge(self._connect_source, item)
4431                     except Exception as e: self.status_message.emit(f"连线异常:
4432 {e}")
4433             return
4434         super().mousePressEvent(event)
4435
4436     def mouseMoveEvent(self, event):
4437         if self._drag_pos is not None:
4438             delta = event.position().toPoint() - self._drag_pos
4439             self._drag_pos = event.position().toPoint()
4440
4441 self.horizontalScrollBar().setValue(self.horizontalScrollBar().value() -
4442 delta.x())
4443             self.verticalScrollBar().setValue(self.verticalScrollBar().value()
4444 - delta.y()); return
4445         super().mouseMoveEvent(event)
4446
4447     def mouseReleaseEvent(self, event):
4448         if event.button() == Qt.MouseButton.MiddleButton:
4449             self._drag_pos = None; self.setCursor(Qt.CursorShape.ArrowCursor);
4450 return

```

```

4451         if self._connect_mode and event.button() == Qt.MouseButton.RightButton:
4452             self._connect_mode = False; self._connect_source = None
4453             self.setCursor(Qt.CursorShape.ArrowCursor)
4454             self.status_message.emit("✕ 已取消连线"); return
4455         super().mouseReleaseEvent(event)
4456
4457     def wheelEvent(self, event):
4458         factor = 1.15 if event.angleDelta().y() > 0 else 1 / 1.15
4459         self.scale(factor, factor)
4460
4461     def keyPressEvent(self, event):
4462         if event.key() == Qt.Key.Key_Delete:
4463             for it in list(self._scene.selectedItems()):
4464                 if isinstance(it, AINodeItem):
4465                     self._remove_node_safe(it.nid)
4466                 elif isinstance(it, EdgeItem):
4467                     self._delete_edge_safe(it)
4468         elif event.key() == Qt.Key.Key_Escape:
4469             if self._connect_mode:
4470                 self._connect_mode = False; self._connect_source = None
4471                 self.setCursor(Qt.CursorShape.ArrowCursor)
4472                 self.status_message.emit("✕ 连线模式已取消")
4473             elif event.key() == Qt.Key.Key_Z and event.modifiers() &
4474                 Qt.KeyboardModifier.ControlModifier:
4475                 self.undo()
4476             elif event.key() == Qt.Key.Key_Y and event.modifiers() &
4477                 Qt.KeyboardModifier.ControlModifier:
4478                 self.redo()
4479             else:
4480                 super().keyPressEvent(event)
4481
4482     def _push_undo(self, action: str, target: str):
4483         self._undo_stack.append({"act": action, "tgt": target, "data":
4484 self.get_flow_data()})
4485         if len(self._undo_stack) > 50: self._undo_stack.pop(0)
4486         self._redo_stack.clear()
4487
4488     def undo(self):
4489         if self._undo_stack:
4490             item = self._undo_stack.pop()
4491             self._redo_stack.append({"data": self.get_flow_data()})
4492             self.load_flow_data(item["data"])
4493             self.flow_changed.emit(); self.status_message.emit("↶ 已撤销")
4494
4495     def redo(self):
4496         if self._redo_stack:
4497             item = self._redo_stack.pop()
4498             self._undo_stack.append({"data": self.get_flow_data()})
4499             self.load_flow_data(item["data"])
4500             self.flow_changed.emit(); self.status_message.emit("↷ 已重做")

```

```

4501
4502 # ==== ui\code_editor.py (228 行) ====
4503 """低代码脚本编辑器 - Python 语法实时校验、代码模板调用"""
4504 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QPlainTextEdit,
4505     QPushButton, QLabel, QListWidget, QSplitter, QTextEdit, QComboBox,
4506     QMessageBox)
4507 from PyQt6.QtCore import Qt, pyqtSignal, QTimer
4508 from PyQt6.QtGui import QFont, QSyntaxHighlighter, QTextCharFormat, QColor
4509 import re, ast, keyword
4510
4511 class PythonHighlighter(QSyntaxHighlighter):
4512     """Python 语法高亮器"""
4513     def __init__(self, parent=None):
4514         super().__init__(parent)
4515         self._rules = []
4516         kw_fmt = QTextCharFormat(); kw_fmt.setForeground(QColor("#c792ea"));
4517         kw_fmt.setFontWeight(QFont.Weight.Bold)
4518         for kw in keyword.kwlist:
4519             self._rules.append((re.compile(rf'\b{kw}\b'), kw_fmt))
4520         str_fmt = QTextCharFormat(); str_fmt.setForeground(QColor("#c3e88d"))
4521         self._rules.append((re.compile(r'"[^"]*"|\'[^\']*\''), str_fmt))
4522         num_fmt = QTextCharFormat(); num_fmt.setForeground(QColor("#f78c6c"))
4523         self._rules.append((re.compile(r'\b\d+\.\d*\b'), num_fmt))
4524         comment_fmt = QTextCharFormat();
4525         comment_fmt.setForeground(QColor("#546e7a"))
4526         self._rules.append((re.compile(r'#. *$'), comment_fmt))
4527         decor_fmt = QTextCharFormat();
4528         decor_fmt.setForeground(QColor("#82aaff"))
4529         self._rules.append((re.compile(r'@\w+'), decor_fmt))
4530         func_fmt = QTextCharFormat(); func_fmt.setForeground(QColor("#82b1ff"))
4531         self._rules.append((re.compile(r'\b([a-zA-Z_]\w*)(?=\s*\())'),
4532         func_fmt))
4533
4534     def highlightBlock(self, text):
4535         for pattern, fmt in self._rules:
4536             for match in pattern.finditer(text):
4537                 self.setFormat(match.start(), match.end() - match.start(), fmt)
4538
4539 CODE_TEMPLATES = {
4540     "数据加载": '''# 数据加载模板
4541 import pandas as pd
4542
4543 def load_data(file_path: str):
4544     """加载 CSV/Excel 数据"""
4545     if file_path.endswith('.csv'):
4546         return pd.read_csv(file_path)
4547     elif file_path.endswith('.xlsx'):
4548         return pd.read_excel(file_path)
4549     return None
4550
'''

```

```

4551 # 可修改 file_path 路径
4552 file_path = "data/sample.csv"
4553 data = load_data(file_path)
4554 print(f"加载数据: {data.shape}")'''
4555     "数据预处理": '''# 数据预处理模板
4556 import numpy as np
4557
4558 def preprocess(data):
4559     """标准化、缺失值填充、异常值处理"""
4560     # 缺失值填充
4561     data = data.fillna(data.mean())
4562     # Z-score 标准化
4563     from scipy import stats
4564     numeric_cols = data.select_dtypes(include=[np.number]).columns
4565     data[numeric_cols] = stats.zscore(data[numeric_cols])
4566     return data
4567
4568 # 修改 preprocess 函数逻辑以适配不同数据
4569 result = preprocess(data)'''
4570     "模型训练": '''# 模型训练模板
4571 from sklearn.model_selection import train_test_split
4572 from sklearn.ensemble import RandomForestClassifier
4573 from sklearn.metrics import accuracy_score, f1_score
4574
4575 def train_model(X, y, **params):
4576     """训练并评估模型"""
4577     X_train, X_test, y_train, y_test = train_test_split(
4578         X, y, test_size=params.get('test_size', 0.2))
4579     model = RandomForestClassifier(
4580         n_estimators=params.get('n_estimators', 100),
4581         max_depth=params.get('max_depth', 10))
4582     model.fit(X_train, y_train)
4583     y_pred = model.predict(X_test)
4584     return {
4585         'model': model,
4586         'accuracy': accuracy_score(y_test, y_pred),
4587         'f1': f1_score(y_test, y_pred, average='weighted')
4588     }'''
4589     "图像处理": '''# OpenCV 图像处理模板
4590 import cv2
4591
4592 def process_image(img_path, operations=None):
4593     """应用图像处理操作链"""
4594     img = cv2.imread(img_path)
4595     if img is None: return None
4596     if operations is None:
4597         operations = ['gray', 'blur', 'edge']
4598     for op in operations:
4599         if op == 'gray':
4600             img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

```

```

4601         elif op == 'blur':
4602             img = cv2.GaussianBlur(img, (5, 5), 0)
4603         elif op == 'edge':
4604             img = cv2.Canny(img, 100, 200)
4605         elif op == 'threshold':
4606             _, img = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
4607     return img'''
4608     """推理预测": "" # 推理预测模板
4609 def predict(model, input_data, preprocess_fn=None):
4610     """执行模型推理"""
4611     if preprocess_fn:
4612         input_data = preprocess_fn(input_data)
4613     predictions = model.predict(input_data)
4614     probabilities = None
4615     if hasattr(model, 'predict_proba'):
4616         probabilities = model.predict_proba(input_data)
4617     return {
4618         'predictions': predictions.tolist(),
4619         'probabilities': probabilities.tolist() if probabilities is not None else
4620         None,
4621         'count': len(predictions)
4622     }'''
4623     """结果可视化": "" # 结果可视化模板
4624 import matplotlib
4625 matplotlib.use('Agg')
4626 import matplotlib.pyplot as plt
4627
4628 def visualize_results(results, chart_type='bar'):
4629     """生成结果图表"""
4630     fig, ax = plt.subplots(figsize=(8, 5))
4631     if chart_type == 'bar':
4632         ax.bar(range(len(results)), results)
4633     elif chart_type == 'line':
4634         ax.plot(range(len(results)), results)
4635     elif chart_type == 'pie':
4636         ax.pie(results, autopct='%1.1f%%')
4637     ax.set_title('Results Visualization')
4638     return fig'''
4639 }
4640
4641 class LowCodeEditor(QWidget):
4642     """低代码脚本编辑器 - 语法校验 + 模板 + 执行"""
4643     code_executed = pyqtSignal(dict)
4644
4645     def __init__(self, parent=None):
4646         super().__init__(parent)
4647         layout = QHBoxLayout(self)
4648
4649         left_split = QSplitter(Qt.Orientation.Vertical)
4650         tmpl_label = QLabel("  代码模板")

```

```

4651     tmpl_label.setStyleSheet("color:#E65100;font-weight:bold;padding:4px;")
4652     self.tmpl_list = QListWidget()
4653     self.tmpl_list.addItem(CODE_TEMPLATES.keys())
4654     self.tmpl_list.itemClicked.connect(self._apply_template)
4655     left_w = QWidget()
4656     left_l = QVBoxLayout(left_w); left_l.setContentsMargins(0, 0, 0, 0)
4657     left_l.addWidget(tmpl_label); left_l.addWidget(self.tmpl_list)
4658     left_split.addWidget(left_w)
4659
4660     right_w = QWidget()
4661     right_l = QVBoxLayout(right_w); right_l.setContentsMargins(4, 4, 4, 4)
4662
4663     toolbar = QHBoxLayout()
4664     self.btn_check = QPushButton("✓ 语法检查")
4665     self.btn_check.clicked.connect(self._check_syntax)
4666     self.btn_run = QPushButton("▶ 执行代码")
4667     self.btn_run.setObjectName("success");
4668     self.btn_run.clicked.connect(self._execute)
4669     self.btn_save = QPushButton(" 保存片段")
4670     self.btn_save.clicked.connect(self._save_snippet)
4671     toolbar.addWidget(self.btn_check)
4672     toolbar.addWidget(self.btn_run)
4673     toolbar.addWidget(self.btn_save)
4674     toolbar.addStretch()
4675     right_l.addLayout(toolbar)
4676
4677     self.editor = QPlainTextEdit()
4678     self.editor.setFont(QFont("Consolas", 11))
4679     self.editor.setStyleSheet("background:#263238;color:#EEEEFF;border:1px
4680 solid #FFD54F;border-radius:6px;padding:8px;")
4681     self.editor.setPlaceholderText("# 在此编写低代码 Python 脚本...\n# 从左
4682 侧选择模板快速开始")
4683     self.editor.setTabStopDistance(32)
4684     self.highlighter = PythonHighlighter(self.editor.document())
4685     right_l.addWidget(self.editor)
4686
4687     self.console = QTextEdit()
4688     self.console.setReadOnly(True)
4689     self.console.setMaximumHeight(150)
4690     self.console.setFont(QFont("Consolas", 10))
4691     self.console.setStyleSheet("background:#FFF8E1;color:#2E7D32;border:1px
4692 solid #FFD54F;border-radius:6px;padding:6px;")
4693     right_l.addWidget(QLabel(" 输 出 控 制 台 ",
4694 styleSheet="color:#E65100;font-weight:bold;"))
4695     right_l.addWidget(self.console)
4696
4697     splitter = QSplitter(Qt.Orientation.Horizontal)
4698     splitter.addWidget(left_split); splitter.addWidget(right_w)
4699     splitter.setSizes([200, 600])
4700     layout.addWidget(splitter)

```

```

4701
4702     def _apply_template(self, item):
4703         tpl_name = item.text()
4704         if tpl_name in CODE_TEMPLATES:
4705             self.editor.setPlainText(CODE_TEMPLATES[tpl_name])
4706
4707     def _check_syntax(self):
4708         code = self.editor.toPlainText()
4709         try:
4710             ast.parse(code)
4711             self.console.setHtml('<span style="color:#2E7D32;">✓ 语法检查通过
4712 </span>')
4713         except SyntaxError as e:
4714             self.console.setHtml(f'<span style="color:#D84315;">✗ 语法错误 行
4715 {e.lineno}: {e.msg}</span>')
4716
4717     def _execute(self):
4718         code = self.editor.toPlainText()
4719         if not code.strip():
4720             self.console.setHtml('<span style="color:#F57F17;">⚠ 请先编写代码
4721 </span>'); return
4722         try:
4723             local_ns = {}
4724             exec(code, {"__builtins__": __builtins__}, local_ns)
4725             output = str(local_ns.get("result", local_ns))
4726             self.console.setHtml(f'<span style="color:#2E7D32;">✓ 执行成功
4727 </span>\n<pre>{self._truncate(output)}</pre>')
4728             self.code_executed.emit({"success": True, "output": output})
4729         except Exception as e:
4730             self.console.setHtml(f'<span style="color:#D84315;">✗ 执行异常:
4731 {type(e).__name__}: {e}</span>')
4732             self.code_executed.emit({"success": False, "error": str(e)})
4733
4734     def _save_snippet(self):
4735         code = self.editor.toPlainText()
4736         if code.strip():
4737             self.console.setHtml('<span style="color:#E65100;"> 代码片段已保
4738 存到本地内存</span>')
4739
4740     def _truncate(self, s: str, n: int = 500) -> str:
4741         return s[:n] + "..." if len(s) > n else s
4742
4743     def set_code(self, code: str):
4744         self.editor.setPlainText(code)
4745
4746     def get_code(self) -> str:
4747         return self.editor.toPlainText()
4748
4749 # ==== ui\login.py (238 行) ====
4750 """登录/注册界面 - 低代码 AI 平台登录窗口"""

```

```

4751 from PyQt6.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel, QLineEdit,
4752     QPushButton, QFrame, QStackedWidget, QCheckBox, QComboBox, QMessageBox)
4753 from PyQt6.QtCore import Qt, pyqtSignal, QTimer, QPropertyAnimation, QEasingCurve
4754 from PyQt6.QtGui import QFont, QColor, QPainter, QBrush, QPen
4755 from core.auth import auth_engine, Role, OpAction
4756 import random
4757
4758 APP_TITLE = "低代码驱动的人工智能模型快速构建与可视化编排平台"
4759
4760 class GradientBackground(QWidget):
4761     """淡黄色渐变背景"""
4762     def __init__(self, parent=None):
4763         super().__init__(parent)
4764         self._particles = [(random.randint(0, 800), random.randint(0, 600),
4765 random.randint(1, 4)) for _ in range(40)]
4766         self._timer = QTimer(self);
4767 self._timer.timeout.connect(self._update_particles)
4768         self._timer.start(60)
4769
4770     def _update_particles(self):
4771         for i, (x, y, s) in enumerate(self._particles):
4772             y = y - s if y > 0 else self.height()
4773             self._particles[i] = (x, y, s)
4774         self.update()
4775
4776     def paintEvent(self, event):
4777         p = QPainter(self)
4778         p.fillRect(self.rect(), QColor(255, 248, 225))
4779         for x, y, s in self._particles:
4780             alpha = 15 + int(y / max(1, self.height()) * 35)
4781             p.setBrush(QColor(255, 183, 0, alpha))
4782             p.setPen(Qt.PenStyle.NoPen)
4783             p.drawEllipse(int(x), int(y), s, s)
4784
4785 class LoginWindow(QWidget):
4786     login_success = pyqtSignal(object)
4787
4788     def __init__(self):
4789         super().__init__()
4790         self.setWindowTitle(f"{APP_TITLE} - 登录")
4791         self.setFixedSize(860, 580)
4792         self.setWindowFlags(Qt.WindowType.FramelessWindowHint)
4793         self._drag_pos = None
4794         self._setup_ui()
4795         self._animation_entrance()
4796
4797     def _setup_ui(self):
4798         self.bg = GradientBackground(self)
4799         self.bg.setGeometry(0, 0, 860, 580)
4800

```



```

4801     main_layout = QHBoxLayout(self)
4802     main_layout.setContentsMargins(0, 0, 0, 0)
4803
4804     # 左侧信息面板
4805     left_panel = QFrame()
4806     left_panel.setFixedWidth(390)
4807     left_panel.setStyleSheet(
4808         "background: qlineargradient(x1:0,y1:0,x2:0,y2:1,stop:0
4809 #FFF8E1,stop:1 #FFECB3); border-radius: 10px;")
4810     left_layout = QVBoxLayout(left_panel)
4811     left_layout.setContentsMargins(30, 28, 30, 28)
4812
4813     close_layout = QHBoxLayout()
4814     btn_close = QPushButton("✕"); btn_close.setFixedSize(30, 30)
4815
4816     btn_close.setStyleSheet("QPushButton{background:transparent;color:#795548;border:
4817 r:none;font-size:16px;} QPushButton: hover{color:#D84315;}")
4818     btn_close.clicked.connect(self.close)
4819     btn_min = QPushButton("—"); btn_min.setFixedSize(30, 30)
4820
4821     btn_min.setStyleSheet("QPushButton{background:transparent;color:#795548;border:
4822 none;font-size:16px;} QPushButton: hover{color:#E65100;}")
4823     btn_min.clicked.connect(self.showMinimized)
4824     close_layout.addStretch(); close_layout.addWidget(btn_min);
4825     close_layout.addWidget(btn_close)
4826     left_layout.addLayout(close_layout)
4827
4828     icon_label = QLabel("◆")
4829     icon_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
4830     icon_label.setStyleSheet("color:#FF8F00;font-size:48px;margin:16px
4831 6px;")
4832     left_layout.addWidget(icon_label)
4833
4834     title = QLabel("低代码驱动的人工智能模型")
4835     title.setAlignment(Qt.AlignmentFlag.AlignCenter)
4836     title.setStyleSheet("color:#4E342E;font-size:18px;font-weight:bold;")
4837     left_layout.addWidget(title)
4838
4839     title2 = QLabel("快速构建与可视化编排平台")
4840     title2.setAlignment(Qt.AlignmentFlag.AlignCenter)
4841
4842     title2.setStyleSheet("color:#4E342E;font-size:18px;font-weight:bold;margin-bott
4843 om:4px;")
4844     left_layout.addWidget(title2)
4845
4846     subtitle = QLabel("AI 模型 • 低代码 • 拖拽式编排")
4847     subtitle.setAlignment(Qt.AlignmentFlag.AlignCenter)
4848
4849     subtitle.setStyleSheet("color:#795548;font-size:13px;margin-bottom:8px;")
4850     left_layout.addWidget(subtitle)

```

```

4851
4852     desc_lines = ["—————",
4853                  "◆ 拖拽式 AI 流程画布编排", "◆ 内置低代码脚本编辑器",
4854                  "◆ 6 类 AI 模型模板一键创建", "◆ OpenCV 视觉处理与 3D 渲染",
4855                  "◆ 业务规则引擎冲突检测", "◆ 多角色 RBAC 权限调度",
4856                  "◆ 数据事务管理与快照回溯"]
4857     for line in desc_lines:
4858         lbl = QLabel(line)
4859         lbl.setAlignment(Qt.AlignmentFlag.AlignCenter)
4860         if line.startswith("◆"):
4861             lbl.setStyleSheet("color:#5D4037;font-size:11px;margin:2px 0;")
4862         elif line.startswith("—"):
4863             lbl.setStyleSheet("color:#FFD54F;font-size:11px;")
4864         left_layout.addWidget(lbl)
4865     left_layout.addStretch()
4866
4867     bot_label = QLabel("默认账号: admin / admin123    developer / dev123")
4868     bot_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
4869     bot_label.setStyleSheet("color:#FF8F00;font-size:10px;margin:6px 0;")
4870     left_layout.addWidget(bot_label)
4871
4872     # 右侧登录/注册面板
4873     right_panel = QFrame()
4874     right_panel.setFixedWidth(400)
4875     right_panel.setStyleSheet("background:#FFF8E1;border-radius:10px;")
4876     right_layout = QVBoxLayout(right_panel)
4877     right_layout.setContentsMargins(36, 28, 36, 28)
4878
4879     self.stack = QStackedWidget()
4880     self.stack.addWidget(self._login_form())
4881     self.stack.addWidget(self._register_form())
4882     right_layout.addWidget(self.stack)
4883
4884     main_layout.addWidget(left_panel)
4885     main_layout.addWidget(right_panel)
4886
4887     def _login_form(self) -> QWidget:
4888         w = QWidget()
4889         layout = QVBoxLayout(w); layout.setSpacing(12)
4890         layout.addSpacing(20)
4891         t = QLabel("      用      户      登      录      ");
4892         t.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
4893         layout.addWidget(t); layout.addSpacing(8)
4894
4895         self.txt_user = QLineEdit(); self.txt_user.setPlaceholderText("用户名");
4896         self.txt_user.setText("admin")
4897         self.txt_user.setStyleSheet("font-size:13px;padding:9px 14px;")
4898         layout.addWidget(self.txt_user)
4899
4900         self.txt_pwd = QLineEdit(); self.txt_pwd.setPlaceholderText("密 码");

```

```

4901 self.txt_pwd.setText("admin123")
4902     self.txt_pwd.setEchoMode(QLineEdit.EchoMode.Password)
4903     self.txt_pwd.setStyleSheet("font-size:13px;padding:9px 14px;")
4904     layout.addWidget(self.txt_pwd)
4905
4906     row = QHBoxLayout()
4907     self.chk_remember = QCheckBox("    记    住    密    码    ");
4908 row.addWidget(self.chk_remember)
4909     row.addStretch()
4910     link_forgot = QPushButton("忘记密码?"); link_forgot.setFlat(True)
4911
4912 link_forgot.setStyleSheet("color:#FF8F00;border:none;font-size:11px;text-decora
4913 tion:underline;min-width:0;background:transparent;")
4914     row.addWidget(link_forgot)
4915     layout.addLayout(row)
4916
4917     btn_login = QPushButton("登  录")
4918     btn_login.setObjectName("primary");
4919 btn_login.setCursor(Qt.CursorShape.PointingHandCursor)
4920     btn_login.clicked.connect(self._do_login)
4921     layout.addWidget(btn_login)
4922
4923     layout.addSpacing(6)
4924     link_reg = QPushButton(" 还  没  有  账  号  ?  立  即  注  册  → ");
4925 link_reg.setFlat(True)
4926
4927 link_reg.setStyleSheet("color:#795548;border:none;font-size:11px;min-width:0;ba
4928 ckground:transparent;")
4929     link_reg.clicked.connect(lambda: self.stack.setCurrentIndex(1))
4930     layout.addWidget(link_reg)
4931     layout.addStretch()
4932     return w
4933
4934     def _register_form(self) -> QWidget:
4935         w = QWidget()
4936         layout = QVBoxLayout(w); layout.setSpacing(10)
4937         layout.addSpacing(12)
4938         t = QLabel("    账    号    注    册    ");
4939 t.setStyleSheet("color:#4E342E;font-size:20px;font-weight:bold;")
4940         layout.addWidget(t); layout.addSpacing(2)
4941
4942     self.reg_user = QLineEdit(); self.reg_user.setPlaceholderText("用户名 (2
4943 位以上)")
4944     layout.addWidget(self.reg_user)
4945     self.reg_pwd = QLineEdit(); self.reg_pwd.setPlaceholderText("密码 (4 位
4946 以上)")
4947     self.reg_pwd.setEchoMode(QLineEdit.EchoMode.Password)
4948     layout.addWidget(self.reg_pwd)
4949     self.reg_pwd2 = QLineEdit(); self.reg_pwd2.setPlaceholderText("确认密码
4950 ")

```

```

4951         self.reg_pwd2.setEchoMode(QLineEdit.EchoMode.Password)
4952         layout.addWidget(self.reg_pwd2)
4953
4954         self.reg_role = QComboBox()
4955         for role in Role: self.reg_role.addItem(role.value, role)
4956         self.reg_role.setCurrentText(Role.GUEST.value)
4957         layout.addWidget(self.reg_role)
4958
4959         btn_reg = QPushButton("注 册"); btn_reg.setObjectName("primary")
4960         btn_reg.clicked.connect(self._do_register)
4961         layout.addWidget(btn_reg)
4962
4963         link_back = QPushButton("← 返回登录"); link_back.setFlat(True)
4964
4965         link_back.setStyleSheet("color:#795548;border:none;font-size:11px;min-width:0;b
4966         ackground:transparent;")
4967         link_back.clicked.connect(lambda: self.stack.setCurrentIndex(0))
4968         layout.addWidget(link_back)
4969         layout.addStretch()
4970         return w
4971
4972     def _do_login(self):
4973         username = self.txt_user.text().strip()
4974         password = self.txt_pwd.text().strip()
4975         if not username or not password:
4976             QMessageBox.warning(self, "提示", "请输入用户名和密码"); return
4977         user = auth_engine.authenticate(username, password)
4978         if user:
4979             self.hide(); self.login_success.emit(user)
4980         else:
4981             QMessageBox.critical(self, "登录失败", "用户名或密码错误, 或账号已被
4982             锁定")
4983
4984     def _do_register(self):
4985         username = self.reg_user.text().strip()
4986         pwd = self.reg_pwd.text().strip()
4987         pwd2 = self.reg_pwd2.text().strip()
4988         if len(username) < 2 or len(pwd) < 4:
4989             QMessageBox.warning(self, "提示", "用户名至少 2 位, 密码至少 4 位");
4990         return
4991         if pwd != pwd2:
4992             QMessageBox.warning(self, "提示", "两次密码不一致"); return
4993         user = auth_engine.register(username, pwd, self.reg_role.currentData())
4994         if user:
4995             QMessageBox.information(self, "成 功", f"注册成功! 角色:
4996             {user.role.value}\n 请返回登录。")
4997             self.stack.setCurrentIndex(0)
4998         else:
4999             QMessageBox.critical(self, "失败", "用户名已存在")
5000

```

```

5001     def _animation_entrance(self):
5002         self.setWindowOpacity(0)
5003         self.anim = QPropertyAnimation(self, b"windowOpacity")
5004         self.anim.setDuration(400); self.anim.setStartValue(0)
5005         self.anim.setEndValue(1);
5006     self.anim.setEasingCurve(QEasingCurve.Type.OutCubic)
5007         self.anim.start()
5008
5009     def mousePressEvent(self, e):
5010         if e.button() == Qt.MouseButton.LeftButton and e.position().y() < 40:
5011             self._drag_pos = e.globalPosition().toPoint()
5012
5013     def mouseMoveEvent(self, e):
5014         if self._drag_pos:
5015             self.move(self.pos() + e.globalPosition().toPoint() - self._drag_pos)
5016             self._drag_pos = e.globalPosition().toPoint()
5017
5018     def mouseReleaseEvent(self, e):
5019         self._drag_pos = None
5020
5021     # ==== ui\main_window.py (219 行) ====
5022     """主窗口 - 13 模块菜单 + 状态栏 + 热更新支持"""
5023     from PyQt6.QtWidgets import (QMainWindow, QMenuBar, QMenu, QStatusBar, QTabWidget,
5024         QToolBar, QDockWidget, QListWidget, QLabel, QMessageBox, QWidget, QVBoxLayout,
5025         QApplication)
5026     from PyQt6.QtCore import Qt, QTimer, pyqtSignal
5027     from PyQt6.QtGui import QAction, QFont
5028     from core.auth import Role, OpAction, auth_engine
5029     from ui.styles import load_stylesheet
5030     import importlib, sys, os
5031
5032     APP_TITLE = "低代码驱动的人工智能模型快速构建与可视化编排平台"
5033
5034     class MainWindow(QMainWindow):
5035         def __init__(self, user):
5036             super().__init__()
5037             self.user = user
5038             self._modules = {}
5039             self._module_widgets = {}
5040             self.setWindowTitle(f"{APP_TITLE} | 用户: {user.username} | 角色:
5041 {user.role.value}")
5042             self.setMinimumSize(1100, 700)
5043             self.resize(1280, 820)
5044             self._setup_menu()
5045             self._setup_toolbar()
5046             self._setup_statusbar()
5047             self._setup_central()
5048             self._setup_quick_nav()
5049             self._hot_reload_timer = QTimer(self)
5050             self._hot_reload_timer.timeout.connect(self._check_hot_reload)

```

```

5051         self._hot_reload_timer.start(3000)
5052         self._file_mtimes = {}
5053         self.statusBar().showMessage(f"✔ 欢迎 {user.username} | 系统就绪 | 13
5054 个业务模块已加载", 5000)
5055
5056     def _setup_menu(self):
5057         mb = self.menuBar()
5058         sys_menu = mb.addMenu("⚙ 系统")
5059         sys_menu.addAction("    刷新模块", self._refresh_modules)
5060         sys_menu.addAction("    切换账号", self._switch_account)
5061         sys_menu.addSeparator()
5062         sys_menu.addAction("    退出系统", self.close)
5063
5064         menus = [
5065             ("    工作台", "dashboard", "项目总览与快速导航"),
5066             ("    模型设计器", "model_designer", "可视化画布编排 AI 流程"),
5067             ("    低代码编辑器", "lowcode_editor", "Python 脚本低代码编写"),
5068             ("    模型训练", "model_training", "AI 模型训练启停控制"),
5069             ("    视觉实验室(OpenCV)", "vision_lab", "图像预处理与视觉推理"),
5070             ("    3D 可视化", "vision_3d", "3D 点云与模型结构渲染"),
5071             ("    规则配置器", "rule_config", "业务规则可视化配置"),
5072             ("    数据管理", "data_manager", "多源数据接入与事务管理"),
5073             ("    任务管控", "task_manager", "任务队列与运行日志"),
5074             ("    项目管理", "project_manager", "项目资源分类管理"),
5075             ("    组件市场", "component_market", "节点/脚本/规则组件"),
5076             ("    报表中心", "report_center", "模型指标与训练报表"),
5077             ("    系统管理", "system_admin", "用户/权限/日志管理"),
5078         ]
5079         for name, mod_key, desc in menus:
5080             menu = mb.addMenu(name)
5081             menu.addAction(f"        打    开    {name[2:]}", lambda k=mod_key:
5082 self._open_module(k))
5083             menu.addAction(f"        {desc}", lambda k=mod_key: self._open_module(k))
5084             menu.addSeparator()
5085             menu.addAction("    刷新", lambda k=mod_key: self._reload_module(k))
5086
5087         help_menu = mb.addMenu(" ? 帮助")
5088         help_menu.addAction("    使用手册", lambda: QMessageBox.information(self,
5089 "帮助",
5090 f"{APP_TITLE}\n\n1. 从工作台新建项目或选择模板\n2. 在模型设计器中拖
5091 拽节点编排流程\n3. 使用低代码编辑器编写节点逻辑\n4. 配置业务规则后启动训练\n5.
5092 在视觉实验室查看处理结果\n6. 报表中心查看训练指标"))
5093         help_menu.addAction("i 关于系统", lambda: QMessageBox.about(self, "关于
5094 ",
5095 f"{APP_TITLE}\n\n核心技术栈: Python3 + PyQt6 + OpenCV + OpenGL\n核
5096 心引擎: RBAC 权限调度 | 状态流转 | 规则引擎 | 数据一致性"))
5097
5098     def _setup_toolbar(self):
5099         tb = QToolBar("快捷工具栏"); tb.setMovable(False); self.addToolBar(tb)
5100         actions = [

```

```

5101         (" 工作台", "dashboard"), (" 设计器", "model_designer"), (" 编辑
5102 器", "lowcode_editor"),
5103         (" 训练", "model_training"), (" 视觉", "vision_lab"), (" 3D",
5104 "vision_3d"),
5105         (" 规则", "rule_config"), (" 数据", "data_manager"), (" 任务",
5106 "task_manager"),
5107         (" 项目", "project_manager"), (" 市场", "component_market"), ("
5108 报表", "report_center"),
5109     ]
5110     for label, mod_key in actions:
5111         act = QAction(label, self)
5112         act.triggered.connect(lambda checked, k=mod_key:
5113 self._open_module(k))
5114         tb.addAction(act)
5115
5116     def _setup_statusbar(self):
5117         self._perm_label = QLabel(f" 角色: {self.user.role.value}")
5118         self._perm_label.setStyleSheet("color:#E65100;padding:2px 8px;")
5119         self.statusBar().addPermanentWidget(self._perm_label)
5120
5121     def _setup_central(self):
5122         self.tab_widget = QTabWidget()
5123         self.tab_widget.setTabsClosable(True)
5124         self.tab_widget.tabCloseRequested.connect(self._close_tab)
5125         self.tab_widget.setDocumentMode(True)
5126         self.setCentralWidget(self.tab_widget)
5127         self._open_module("dashboard")
5128
5129     def _setup_quick_nav(self):
5130         dock = QDockWidget(" 模块导航", self)
5131         nav_list = QListWidget()
5132         modules_list = ["工作台", "模型设计器", "低代码编辑器", "模型训练",
5133 "视觉实验室", "3D 可视化", "规则配置器", "数据管理",
5134 "任务管控", "项目管理", "组件市场", "报表中心", "系统管
5135 理"]
5136         nav_list.addItem(modules_list)
5137         nav_list.itemClicked.connect(lambda item:
5138 self._open_module_by_name(item.text()))
5139         nav_list.setMaximumWidth(160)
5140         dock.setWidget(nav_list)
5141         dock.setFeatures(QDockWidget.DockWidgetFeature.DockWidgetMovable |
5142 QDockWidget.DockWidgetFeature.DockWidgetFloatable)
5143         self.addDockWidget(Qt.DockWidgetArea.LeftDockWidgetArea, dock)
5144
5145     def _open_module(self, mod_key: str):
5146         if not auth_engine.check_permission(self.user, mod_key, OpAction.VIEW):
5147             QMessageBox.warning(self, "权限不足", f"您没有访问 [{mod_key}] 模块
5148 的权限")
5149             return
5150         for i in range(self.tab_widget.count()):

```

```

5151         if self.tab_widget.tabText(i).startswith(self._mod_name(mod_key)):
5152             self.tab_widget.setCurrentIndex(i); return
5153         widget = self._load_module_widget(mod_key)
5154         if widget:
5155             idx = self.tab_widget.addTab(widget, f"{self._mod_icon(mod_key)}
5156 {self._mod_name(mod_key)}")
5157             self.tab_widget.setCurrentIndex(idx)
5158
5159     def _open_module_by_name(self, name: str):
5160         name_map = {"工作台": "dashboard", "模型设计器": "model_designer", "低代码编
5161 辑器": "lowcode_editor",
5162             "模型训练": "model_training", "视觉实验室": "vision_lab", "3D 可视化
5163             ": "vision_3d",
5164             "规则配置器": "rule_config", "数据管理": "data_manager", "任务管控
5165             ": "task_manager",
5166             "项目管理": "project_manager", "组件市场": "component_market", "报表中
5167             心": "report_center",
5168             "系统管理": "system_admin"}
5169         key = name_map.get(name)
5170         if key: self._open_module(key)
5171
5172     def _load_module_widget(self, mod_key: str):
5173         if mod_key in self._module_widgets:
5174             return self._module_widgets[mod_key]
5175         try:
5176             module = importlib.import_module(f"modules.{mod_key}")
5177             if hasattr(module, 'get_module_widget'):
5178                 widget = module.get_module_widget(self.user)
5179                 self._module_widgets[mod_key] = widget
5180                 self._modules[mod_key] = module
5181                 self._track_file(mod_key, module)
5182                 from PyQt6.QtWidgets import QScrollArea
5183                 scroll = QScrollArea()
5184                 scroll.setWidgetResizable(True)
5185                 scroll.setWidget(widget)
5186
5187             scroll.setStyleSheet("QScrollArea{background:transparent;border:none;}")
5188             return scroll
5189         except Exception as e:
5190             import traceback
5191             QMessageBox.critical(self, "模块加载失败", f"模块 [{mod_key}] 加载异
5192 常:\n{str(e)[:200]}")
5193             return QLabel(f"模块加载失败: {e}")
5194
5195     def _reload_module(self, mod_key: str):
5196         if mod_key in self._modules:
5197             importlib.reload(self._modules[mod_key])
5198             if mod_key in self._module_widgets:
5199                 del self._module_widgets[mod_key]
5200             self._open_module(mod_key)

```



```

5201         self.statusBar().showMessage(f"    模块 [{mod_key}] 已热更新", 3000)
5202
5203     def _refresh_modules(self):
5204         for mod_key in list(self._modules.keys()):
5205             self._reload_module(mod_key)
5206
5207     def _check_hot_reload(self):
5208         for mod_key, module in list(self._modules.items()):
5209             try:
5210                 mod_file = module.__file__
5211                 if mod_file and os.path.exists(mod_file):
5212                     mtime = os.path.getmtime(mod_file)
5213                     if mod_key in self._file_mtimes and
5214 self._file_mtimes[mod_key] != mtime:
5215                         self.statusBar().showMessage(f"× 检测到模块 [{mod_key}]
5216 变更, 自动热更新...", 2000)
5217                         self._reload_module(mod_key)
5218                         self._file_mtimes[mod_key] = mtime
5219             except: pass
5220
5221     def _track_file(self, mod_key: str, module):
5222         try:
5223             mod_file = module.__file__
5224             if mod_file and os.path.exists(mod_file):
5225                 self._file_mtimes[mod_key] = os.path.getmtime(mod_file)
5226         except: pass
5227
5228     def _close_tab(self, index: int):
5229         self.tab_widget.removeTab(index)
5230
5231     def _switch_account(self):
5232         self.hide()
5233         from ui.login import LoginWindow
5234         self.login_win = LoginWindow()
5235         self.login_win.setAttribute(Qt.WidgetAttribute.WA_DeleteOnClose)
5236         self.login_win.login_success.connect(self._on_relogin)
5237         self.login_win.destroyed.connect(self._on_login_closed)
5238         self.login_win.show()
5239
5240     def _on_login_closed(self):
5241         if not self.isVisible():
5242             self.show()
5243             self.statusBar().showMessage("已取消账号切换", 3000)
5244
5245     def _on_relogin(self, user):
5246         self.user = user
5247         self.setWindowTitle(f"{APP_TITLE} | 用户: {user.username} | 角色:
5248 {user.role.value}")
5249         self._perm_label.setText(f"    角色: {user.role.value}")
5250         self.tab_widget.clear(); self._module_widgets.clear();

```

```

5251 self._modules.clear()
5252     self._open_module("dashboard"); self.show()
5253     self.statusBar().showMessage(f" ✓ 已切换至 {user.username}
5254 ({user.role.value})", 5000)
5255
5256     def _mod_name(self, key: str) -> str:
5257         names = {"dashboard": "工作台", "model_designer": "模型设计器",
5258 "lowcode_editor": "低代码编辑器",
5259 "model_training": "模型训练", "vision_lab": "视觉实验室",
5260 "vision_3d": "3D可视化",
5261 "rule_config": "规则配置器", "data_manager": "数据管理",
5262 "task_manager": "任务管控",
5263 "project_manager": "项目管理", "component_market": "组件市场",
5264 "report_center": "报表中心",
5265 "system_admin": "系统管理"}
5266         return names.get(key, key)
5267
5268     def _mod_icon(self, key: str) -> str:
5269         icons = {"dashboard": " ", "model_designer": " ", "lowcode_editor": " ", "model_training": " ",
5270 "vision_lab": " ", "vision_3d": " ", "rule_config": " ", "data_manager": " ",
5271 "task_manager": " ", "project_manager": " ", "component_market": " ", "report_center": " ",
5272 "system_admin": " "}
5273         return icons.get(key, " ")
5274
5275 # ==== ui\styles.py (140行) ====
5276 """低代码AI平台统一样式表 - 淡黄色专业主题"""
5277 import os
5278
5279 APP_THEME = """
5280 /* 全局淡黄色主题 */
5281 QMainWindow, QDialog { background-color: #FFF8E1; color: #4E342E; }
5282 QWidget { font-family: "Microsoft YaHei", "Segoe UI", sans-serif; font-size: 12px; }
5283
5284 /* 菜单栏 */
5285 QMenuBar { background: #FFECB3; color: #4E342E; border-bottom: 2px solid #FFD54F;
5286 padding: 2px; }
5287 QMenuBar::item:selected { background: #FFE082; color: #E65100; border-radius:
5288 4px; }
5289 QMenu { background: #FFFDE7; color: #4E342E; border: 1px solid #FFD54F; padding:
5290 4px; border-radius: 6px; }
5291 QMenu::item { padding: 6px 28px; border-radius: 4px; }
5292 QMenu::item:selected { background: #FFECB3; color: #E65100; }
5293 QMenu::separator { height: 1px; background: #FFD54F; margin: 3px 10px; }
5294
5295 /* 工具栏 */

```

```

5301 QToolBar { background: #FFECB3; border-bottom: 1px solid #FFD54F; padding: 2px;
5302 spacing: 4px; }
5303 QToolButton { background: transparent; color: #5D4037; border: 1px solid
5304 transparent; border-radius: 4px; padding: 4px 10px; }
5305 QToolButton:hover { background: #FFE082; color: #E65100; border-color: #FFB300; }
5306 QToolButton:checked { background: #FFE082; color: #E65100; border-color: #FFB300; }
5307
5308 /* 标签页 */
5309 QTabWidget::pane { border: 1px solid #FFD54F; background: #FFF8E1; border-radius:
5310 6px; }
5311 QTabBar::tab { background: #FFFDE7; color: #5D4037; border: 1px solid #FFD54F;
5312 padding: 6px 16px;
5313 margin-right: 2px; border-radius: 6px 6px 0 0; }
5314 QTabBar::tab:selected { background: #FFECB3; color: #E65100; border-bottom: 2px
5315 solid #FF8F00; }
5316 QTabBar::tab:hover { background: #FFE082; color: #E65100; }
5317
5318 /* 按钮 */
5319 QPushButton { background: #FFECB3; color: #5D4037; border: 1px solid #FFD54F;
5320 border-radius: 6px;
5321 padding: 6px 16px; font-weight: bold; min-width: 70px; }
5322 QPushButton:hover { background: #FFE082; border-color: #FF8F00; color: #E65100; }
5323 QPushButton:pressed { background: #FFD54F; }
5324 QPushButton:disabled { background: #F5F5DC; color: #BCAAA4; border-color:
5325 #D7CCC8; }
5326 QPushButton#primary { background: qlineargradient(x1:0,y1:0,x2:1,y2:0,stop:0
5327 #FF8F00,stop:1 #FFB300);
5328 color: white; border: none; font-size: 13px; padding: 8px 24px; }
5329 QPushButton#danger { background: #FFCCBC; color: #D84315; border-color: #FF8A65; }
5330 QPushButton#danger:hover { background: #FFAB91; }
5331 QPushButton#success { background: #C8E6C9; color: #2E7D32; border-color: #81C784; }
5332 QPushButton#success:hover { background: #A5D6A7; }
5333 QPushButton#warning { background: #FFECB3; color: #F57F17; border-color: #FFB300; }
5334 QPushButton#warning:hover { background: #FFE082; }
5335
5336 /* 输入框 */
5337 QLineEdit, QTextEdit, QPlainTextEdit { background: #FFFFFF; color: #4E342E; border:
5338 1px solid #FFD54F;
5339 border-radius: 6px; padding: 6px; selection-background-color: #FFE082; }
5340 QLineEdit:focus, QTextEdit:focus { border-color: #FF8F00; }
5341 QSpinBox, QDoubleSpinBox { background: #FFFFFF; color: #E65100; border: 1px solid
5342 #FFD54F;
5343 border-radius: 6px; padding: 4px; font-weight: bold; }
5344
5345 /* 表格 */
5346 QTableWidget, QTableView { background: #FFFDE7; color: #4E342E; border: 1px solid
5347 #FFD54F;
5348 gridline-color: #FFECB3; border-radius: 6px; selection-background-color:
5349 #FFE082; }
5350 QHeaderView::section { background: #FFECB3; color: #E65100; border: 1px solid

```

```

5351 #FFD54F;
5352     padding: 6px; font-weight: bold; }
5353 QWidget::item:hover { background: #FFF8E1; }
5354
5355 /* 列表 */
5356 QListWidget, QListView { background: #FFFDE7; color: #4E342E; border: 1px solid
5357 #FFD54F;
5358     border-radius: 6px; selection-background-color: #FFE082; outline: none; }
5359 QListWidget::item { padding: 4px 10px; border-radius: 3px; }
5360 QListWidget::item:selected { background: #FFE082; color: #E65100; }
5361 QListWidget::item:hover { background: #FFF8E1; }
5362
5363 /* 树形控件 */
5364 QTreeWidget, QTreeView { background: #FFFDE7; color: #4E342E; border: 1px solid
5365 #FFD54F;
5366     border-radius: 6px; selection-background-color: #FFE082; }
5367 QTreeWidget::item { padding: 3px; }
5368 QTreeWidget::item:selected { background: #FFE082; color: #E65100; }
5369
5370 /* 滚动条 */
5371 QScrollBar:vertical { background: #FFF8E1; width: 12px; border-radius: 6px; }
5372 QScrollBar::handle:vertical { background: #FFD54F; border-radius: 6px; min-height:
5373 30px; }
5374 QScrollBar::handle:vertical:hover { background: #FF8F00; }
5375 QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical { height: 0; }
5376 QScrollBar:horizontal { background: #FFF8E1; height: 12px; border-radius: 6px; }
5377 QScrollBar::handle:horizontal { background: #FFD54F; border-radius: 6px; min-width:
5378 30px; }
5379 QScrollBar::handle:horizontal:hover { background: #FF8F00; }
5380 QScrollBar::add-line:horizontal, QScrollBar::sub-line:horizontal { width: 0; }
5381
5382 /* 分组框 */
5383 QGroupBox { color: #E65100; border: 1px solid #FFD54F; border-radius: 8px;
5384 margin-top: 14px;
5385     padding-top: 14px; font-weight: bold; font-size: 13px; }
5386 QGroupBox::title { subcontrol-origin: margin; left: 14px; padding: 0 6px;
5387 background: #FFF8E1; }
5388
5389 /* 滑块 */
5390 QSlider::groove:horizontal { background: #FFECB3; height: 6px; border-radius:
5391 3px; }
5392 QSlider::handle:horizontal { background: #FF8F00; width: 16px; height: 16px; margin:
5393 -5px 0;
5394     border-radius: 8px; }
5395 QSlider::sub-page:horizontal { background:
5396 qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 #FFB300, stop:1 #FF8F00);
5397     border-radius: 3px; }
5398
5399 /* 进度条 */
5400 QProgressBar { background: #FFFFFF; border: 1px solid #FFD54F; border-radius: 6px;

```

```

5401 text-align: center;
5402     color: #4E342E; font-weight: bold; }
5403 QProgressBar::chunk { background: qlineargradient(x1:0,y1:0,x2:1,y2:0,stop:0
5404 #FF8F00,stop:1 #FFB300);
5405     border-radius: 5px; }
5406
5407 /* 复选框/单选框 */
5408 QCheckBox, QRadioButton { color: #4E342E; spacing: 6px; }
5409 QCheckBox::indicator, QRadioButton::indicator { width: 16px; height: 16px; border:
5410 2px solid #FFD54F;
5411     border-radius: 3px; background: #FFFFFF; }
5412 QCheckBox::indicator:checked { background: #FF8F00; border-color: #FF8F00; }
5413 QRadioButton::indicator:checked { background: #FF8F00; border-color: #FF8F00; }
5414
5415 /* 下拉框 */
5416 QComboBox { background: #FFFFFF; color: #4E342E; border: 1px solid #FFD54F;
5417 border-radius: 6px;
5418     padding: 4px 10px; min-width: 100px; }
5419 QComboBox:hover { border-color: #FF8F00; }
5420 QComboBox QAbstractItemView { background: #FFFDE7; color: #4E342E; border: 1px
5421 solid #FFD54F;
5422     selection-background-color: #FFE082; }
5423 QComboBox::drop-down { border: none; padding-right: 6px; }
5424
5425 /* 停靠窗口 */
5426 QDockWidget { color: #E65100; titlebar-close-icon: none; }
5427 QDockWidget::title { background: #FFECB3; border-bottom: 2px solid #FFD54F; padding:
5428 6px; }
5429
5430 /* 状态栏 */
5431 QStatusBar { background: #FFECB3; color: #5D4037; border-top: 1px solid #FFD54F; }
5432
5433 /* 滚动区域 */
5434 QScrollArea { border: none; background: transparent; }
5435
5436 /* 分割线 */
5437 QSplitter::handle { background: #FFD54F; width: 2px; }
5438 QSplitter::handle:hover { background: #FF8F00; }
5439
5440 /* 链接标签 */
5441 QLabel#link { color: #E65100; text-decoration: underline; }
5442 QLabel#title { color: #E65100; font-size: 18px; font-weight: bold; }
5443 QLabel#subtitle { color: #795548; font-size: 11px; }
5444 QLabel#badge { background: #FFE082; color: #E65100; border-radius: 10px; padding:
5445 2px 8px; font-size: 10px; }
5446 ""
5447
5448 def get_icons_dir():
5449     return os.path.join(os.path.dirname(os.path.dirname(__file__)), "resources")
5450

```

```
5451 def load_stylesheet() -> str:
5452     return APP_THEME
5453
5454
```